# FROM MANUAL OPERATIONS TO SECURE DIGITAL INFRASTRUCTURE: A CYBERSECURITY INTEGRATION MODEL FOR MEDICAL SUPPLY ENTERPRISES

**CAPSTONE PROJECT DOCUMENTATION**

**A Capstone Project**
**Presented to the Faculty of the**
**Bachelor of Science in Information Technology**
**PHINMA University of Iloilo**
**Iloilo City, Philippines**

**In Partial Fulfillment**
**of the Requirements for the Degree of**
**Bachelor of Science in Information Technology**

**By:**
**CHRISTIAN JAMES DEJAPON**
**CHARLES CABIRA**
**ARJUN DELOTERIO**
**AMIEL CHRISTOPHER CASTRO**
**JULES ARCHANGEL DUNCA**

**2026**

**TABLE OF CONTENTS**

**CHAPTER I ..... 10**

INTRODUCTION ..... 10

1.1 Background of the Study ..... 10

1.2 Statement of the Problem ..... 22

1.3 Objectives of the Study ..... 27

1.3.1 General Objective: ..... 27

1.3.2 Specific Objectives: ..... 28

1.4 Scope and Delimitation ..... 31

1.4.1 Scope of the Study: ..... 31

1.4.2 Delimitations of the Study: ..... 35

1.5 Significance of the Study ..... 38

1.6 Definition of Terms ..... 44

**REFERENCES ..... 60**

**CHAPTER II ..... 61**

REVIEW OF RELATED LITERATURE AND STUDIES ..... 61

2.1 Foreign Studies ..... 61

2.1.1 Cybersecurity of Small-to-Medium Businesses: ..... 62

2.1.2 Systematic Review of SME Cybersecurity Challenges: ..... 62

2.1.3 Agile Development of Secure Software for SMEs: ..... 63

2.1.4 Efficacy, Reliability, and Accuracy of Inventory Systems in SMEs: 64
2.1.5 Design of a Web-Based Inventory Management System for SMEs: 64
2.1.6 Development of a Web-Based Inventory Management System for Small Businesses: 65
2.1.7 Digitalization of the Healthcare Supply Chain: 66
2.1.8 Global Medical Equipment Rental Market Trends: 66
2.1.9 Comparative Analysis of Agile and Waterfall Development Methodologies: 67
2.1.10 Security Assessment in Web-Based Rental Management Systems: 67
2.1.11 User Acceptance Testing in Management Information Systems: 68
2.1.12 SQL Injection and Input Validation Security in Web Applications: 68
2.1.13 Session Management and Authentication Best Practices: 69
2.1.14 Cross-Site Scripting (XSS) and CSRF Attack Prevention: 70
2.2 Local Studies 70
2.2.1 Web-Based Inventory Management System for a Philippine Small Enterprise: 71
2.2.2 Cybersecurity Landscape in the Philippines: 71
2.2.3 Digital Transformation in Philippine Organizations: 72
2.2.4 IoT Adoption and Supply Chain Management in Philippine SMEs: 72

2.2.5 User Requirements for Inventory Systems in Philippine Micro and Small Enterprises: ..... 73
2.2.6 National Privacy Commission Enforcement and Compliance Guidance: ..... 74
2.2.7 Digital Literacy and Technology Adoption Among Filipino SME Staff: ..... 74
2.2.8 Home-Based Care and Medical Equipment Supply in the Philippine Context: ..... 75
2.3 Related Literature ..... 75
2.3.1 Management Information Systems: ..... 76
2.3.2 Network Security Essentials: ..... 76
2.3.3 Philippine Data Privacy Act of 2012 (Republic Act No. 10173): ..... 77
2.3.4 OWASP Top 10 Web Application Security Risks: ..... 78
2.3.5 NIST Cybersecurity Framework 2.0: ..... 78
2.3.6 Role-Based Access Control (RBAC) Theory and Implementation: ..... 79
2.3.7 Password Hashing and Credential Security: ..... 79
2.3.8 ISO/IEC 25010 Software Product Quality Model: ..... 80
2.3.9 Python Flask Framework for Web Application Development: ..... 81
2.3.10 MySQL Relational Database Management System: ..... 81
2.3.11 Client-Server Architecture for Web Applications: ..... 82
2.3.12 Database Design and Normalization for Transaction Processing Systems: ..... 82
2.3.13 Responsive Web Design for Mobile-First Business Applications: ..... 83

2.3.14 Audit Trail Logging and Non-Repudiation in Information Systems: 83
2.3.15 Inventory Economic Order Quantity and Stock Management Theory: 84
2.4 Synthesis of the Reviewed Literature and Studies 85
2.4.1 Convergence on the Vulnerability of Manual Small Enterprise Systems: 85
2.4.2 Convergence on the Security Gap in SME Digital Adoption: 86
2.4.3 The Philippine Regulatory Imperative: 86
2.4.4 Technical Framework Convergence: 87
2.4.5 The Research Gap: 87
**REFERENCES 89**
**CHAPTER III 94**
METHODOLOGY 94
3.1 Research Design 94
3.2 System Development Methodology 96
3.2.1 Agile Development Phases: 97
Phase 0: Planning and Requirements Gathering 98
Phase 1: Database Design and Core Setup 99
Phase 2: Authentication and User Management 99
Phase 3: Inventory and Product Catalog 100
Phase 4: Cart, Order and Payment System 101
Phase 5: Rental Management and Invoicing 101
Phase 6: Security, Reports and Dashboard 102

Phase 7: Testing and Quality Assurance..... 102

3.3 System Architecture..... 103

3.3.1 Three-Tier Architecture Overview:..... 103

3.3.2 Model-View-Controller Pattern:..... 104

3.3.3 Integrated Security Layer:..... 105

3.3.4 External Service Integrations:..... 106

3.3.5 System Architecture Diagram:..... 107

3.4 Tools and Technologies Used..... 108

3.4.1 Programming Languages:..... 108

3.4.2 Frameworks and Libraries:..... 109

3.4.3 Database Technology:..... 110

3.4.4 Development Tools:..... 111

3.4.5 External Services:..... 111

3.5 Data Gathering Techniques..... 112

3.5.1 Interview:..... 112

Interview Protocol..... 112

Key Interview Findings..... 113

3.5.2 Observation:..... 114

3.5.3 Document Analysis:..... 114

3.5.4 Requirements Specification:..... 115

3.6 Testing Procedures..... 115

3.6.1 Unit Testing:..... 116

Authentication Module Unit Tests..... 116

Inventory Module Unit Tests..... 117

Order Processing Unit Tests ..... 117

3.6.2 Integration Testing: ..... 118

3.6.3 System Testing: ..... 119

3.6.4 Security Testing: ..... 120

3.6.5 User Acceptance Testing (UAT): ..... 121

**System Diagrams ..... 123**

Data Flow Diagrams ..... 124

Use Case Diagram ..... 126

**REFERENCES ..... 127**

**CHAPTER IV ..... 128**

SYSTEM DEVELOPMENT, IMPLEMENTATION, TESTING, EVALUATION, RESULTS, AND DISCUSSION ..... 128

4.1 Overview of the Developed System ..... 128

4.1.1 System Workflow: ..... 130

4.1.2 Sale Transaction Processing Flow: ..... 131

4.1.3 Rental Transaction Processing Flow: ..... 133

4.2 System Development Methodology ..... 135

4.2.1 Requirements Gathering: ..... 135

4.2.2 System Planning: ..... 136

4.2.3 Software Development: ..... 136

4.2.4 Testing: ..... 137

4.3 System Architecture ..... 138

4.3.1 Primary Architectural Components: ..... 140

4.3.2 Request Processing Flow: 140
4.3.3 Security Architecture: 141
4.4 Software and Hardware Requirements 141
4.4.1 Hardware Requirements: 141
4.4.2 Software Requirements: 141
4.5 Database Design 142
4.5.1 Core Entities and Relationships: 143
4.5.2 Detailed Table Specifications: 144
4.6 Deployment and Configuration 146
4.7 Testing, Evaluation, and Results 147
4.7.1 Unit Testing: 147
4.7.2 Integration Testing: 148
4.7.3 System Testing: 148
4.7.4 Security Testing (OWASP-Oriented): 149
4.7.5 User Acceptance Testing: 150
4.8 Formal Quality Evaluation (ISO/IEC 25010) 151
4.9 Results and Discussion 152
4.10 Chapter Summary 153
**REFERENCES 154**
**CHAPTER V 155**
CONCLUSION AND RECOMMENDATIONS 155
5.1 Introduction 155
5.2 Summary of Findings 155

5.3 Conclusion ..... 156

5.4 Recommendations ..... 157

5.4.1 System Improvements: ..... 158

5.4.2 Future Research Directions: ..... 158

5.4.3 Implementation Strategies: ..... 159

5.4.4 Alignment with Chapter 1 Objectives: ..... 160

5.5 Impact of the Study ..... 160

5.6 Summary ..... 161

# CHAPTER I

## INTRODUCTION

### 1.1 Background of the Study

Medical supply enterprises played an important but often overlooked role in the Philippine healthcare system. They were the businesses that people turn to when a family member comes home from the hospital and needs a wheelchair, a hospital bed, or an oxygen concentrator to continue recovery at home. They served patients, caregivers, small clinics, and community health centers that needed equipment either for purchase or for short-term rental. In many local communities, a medical supply store was the most accessible source of these items, especially for families who cannot afford to buy expensive equipment outright and rely on rental arrangements to manage the cost. CareMed Medical Supply Enterprise was one such business — a small, locally operating enterprise that provided this kind of service to its community, offering medical equipment for both sale and rental to individuals and small healthcare facilities in its area.

Like many small businesses in the Philippines, CareMed had grown over time by relying on manual, paper-based systems to manage its day-to-day operations. Sales were recorded in handwritten logbooks. Inventory was tracked through physical stock cards and occasional manual counts. Rental agreements were written on printed forms and stored in folders. Customer information was kept in notebooks or informal notes on shared computers and messaging apps. While these methods worked well enough when the business was smaller and transactions were fewer, they had become increasingly difficult to manage as the volume of daily operations grew. The bigger the business gets,

the harder it became to keep track of everything accurately using only paper and pen.

The problems that come with manual record-keeping were not unique to CareMed. Many small businesses across the Philippines faced the same challenges. Records get lost or damaged. Information entered by one staff member may not match what another staff member recorded. Finding old transaction details takes time that could be spent serving customers. And perhaps most importantly, there was no reliable way for management to see the full picture of the business at any given moment without spending hours manually compiling data from multiple sources. For a medical supply business specifically, these problems carry extra weight—because when a customer urgently needs a rental wheelchair and staff cannot confirm whether one is available, or when an overdue rental goes unnoticed for weeks because no one checks the folder, the impact goes beyond a business inconvenience. It affects real people who depend on the equipment for their daily care and recovery.

Beyond day-to-day operational challenges, there was growing concern about how customer information was stored and protected. Because CareMed collected personal details from its customers — names, contact numbers, home addressed, and records of what equipment they had rented or purchased — it had a legal obligation under the Philippine Data Privacy Act of 2012 to protect that information from unauthorized access. At present, however, that information was stored in unprotected notebooks, shared computers with no login controls, and messaging app conversations that anyone with access to the device can read. This was not a situation that was created with bad intentions; it was simply how small businesses had always operated. But as data privacy regulations in the Philippines became more actively enforced, and as the consequences of data breaches became more serious,

continuing to manage customer information this way was a risk that CareMed can no longer afford to ignore.

This study was conducted in response to exactly these challenges. The goal was to design and develop a web-based management system for CareMed that handles its core business operations digitally — covering inventory, sales, rental tracking, customer records, and reporting — while also building in proper cybersecurity protections from the ground up. Rather than treating operations and security as two separate concerns to be addressed one after the other, this study treats them as equally important from the very start of the design process. The result was a system that makes CareMed run more efficiently while also ensuring that customer data was handled with the care and protection the law required.

To understand why a system like this was needed, it helps to look at the broader environment CareMed operated in. The Philippine healthcare system had been undergoing major changes in recent years. The Universal Health Care Act of 2019 (Republic Act No. 11223 [1]) expanded health insurance coverage to all Filipinos, which had gradually increased the number of people accessing healthcare services across the country. More patients receiving care — especially in community settings and at home after hospital discharge — means more demand for the kind of equipment that medical supply businesses like CareMed provided. What used to be a modest local operation serving a small and predictable customer base was now expected to served more customers, manage more rentals simultaneously, and respond faster and more reliably than before. Manual systems that were once manageable were struggling to keep up.

The Philippines also had a large and economically significant small business sector. According to the Philippine Statistics Authority's 2021 MSME Survey [2], micro, small, and

medium enterprises account for about 99.5 percent of all registered businesses and employ nearly 63 percent of the business sector workforce. Most of these businesses, including most medical supply stores, still rely heavily on manual processes and had been slow to adopt digital tools. This was not because business owners did not want to improve — it was largely because the available options were often expensive, complex, or designed for much larger organizations. There was a real need for practical, affordable digital solutions that fit the scale and budget of a small Filipino business, and this study aimed to contribute exactly that.

The specific operational challenges of medical supply enterprises within the broader MSME landscape were shaped by the unique combination of product categories and transaction types these enterprises manage simultaneously. Unlike a purely transactional retail business that completes its operational obligation to a customer at the point of sale, a medical supply enterprise with a rental operation maintained an ongoing, relationship-intensive operational commitment to each rental customer throughout the rental period — a commitment requiring continuous monitoring of equipment location and availability status, systematic tracking of return dates and overdue obligations, reliable communication with customers about return reminders, and accurate financial tracking of rental fee accrual and deposit management. The simultaneous management of multiple concurrent rental agreements, each with its own timeline, product commitment, customer relationship, and payment status, created an administrative complexity that scales non-linearly with business volume: as the number of concurrent rentals grew, the cognitive and organizational burden of managing them under a paper-based system grew disproportionately, because the cross-referencing and synthesis required to produce an accurate overview of the rental portfolio from dispersed physical records became

progressively more difficult and error-prone. This complexity scaling dynamic was the primary structural driver of operational failure risk in manual rental management systems as the business grew, and represented the most urgent operational problem that the proposed digital system was designed to addressed.

CareMed Medical Supply Enterprise was a small business operating within the Philippine healthcare supply sector, offering durable medical equipment for both outright sale and rental to a diverse customer base comprising individual patients managing recovery or chronic conditions at home, family caregivers supporting elderly or ill relatives, and small healthcare institutions in its local service area. The enterprise operated from a single physical location serving both as a customer-facing display and sales environment and as a storage facility for the full scope of its inventory. Despite the growing volume and operational complexity of its business, CareMed continues to rely entirely on manual, paper-based processes for the management of all operational functions. Sales transactions were entered in handwritten logbooks maintained at the counter. Inventory was tracked through physical stock cards and periodic manual counting. Rental agreements were recorded on printed paper forms stored in folder systems. Customer information was maintained in handwritten notebooks, informal messaging application records, and unprotected digital files. Business performance information was assembled manually from these scattered sources whenever management required operational context for a decision — a process measured in hours, prone to omission, and dependent on the accuracy of the underlying records from which it was drawn.

The operational consequences of CareMed's manual system limitations were observable and recurring in its day-to-day operations. Inventory discrepancies between stock card records

and actual storage quantities were identified during periodic physical counts but cannot be systematically traced to their source without the transaction-level audit trail that a digital inventory system provided. Rental agreements involving specific items occasionally conflict because the prior commitment of an item to a current rental customer was not reflected in the availability information consulted when a subsequent customer requested the same item. Rental returns occurring outside peak operational hours or during staff transitions were occasionally not recorded promptly, creating periods of uncertainty about an item's availability status. Customer inquiries about order history or rental status required staff to manually search through multiple physical record sources, consuming time and occasionally producing incomplete or inaccurate responses when the relevant records were not readily located. Each of these failure modes, while individually manageable at a low transaction volume, accumulates into a systemic service quality and operational integrity problem as the business grew and the volume of concurrent transactions increases beyond what manual monitoring can reliably sustain.

The cybersecurity dimensions of CareMed's situation constitute a second major challenge that, while less visibly disruptive than the operational consequences of manual record-keeping in daily operations, carries potentially more severe long-term consequences for the enterprise and for its customers. As small businesses across the Philippines had progressively adopted consumer-grade digital communication tools for operational purposes — messaging platforms, email services, and generic spreadsheet applications — they had created digital repositories of sensitive organizational and customer data in environments designed for general communication rather than secure information management. These environments lack the access controls, encryption, audit

mechanisms, and security governance structures that would be expected in a purpose-built business information system. For CareMed, customer personal data including names, contact numbers, home addressed, and records of medical equipment usage were stored across messaging application histories, unprotected shared computer files, and informal note-taking tools — none of which provided the security controls that the Philippine Data Privacy Act of 2012 required all personal information controllers to implemented.

The Philippine Data Privacy Act of 2012, enacted as Republic Act No. 10173 [3] and implemented by the National Privacy Commission through a series of implementing rules, circulars, and advisory opinions, establishes a comprehensive legal framework for the protection of personal information processed by both government agencies and private sector organizations operating in the Philippines. The Act defined personal information broadly to include any data from which the identity of an individual can reasonably be ascertained, and it establishes a category of sensitive personal information — encompassing health, medical, and financial information — that attracts heightened protection requirements. All entities that determine the purposes and means of processing personal data, defined by the Act as personal information controllers, were required to implemented organizational, physical, and technical security measures appropriate to the nature and volume of personal data they process, regardless of the size of the organization or the formality of its data management infrastructure. The penalties for violations of the Act's security requirements include administrative fines of up to five million pesos per violation, civil damages claims from affected data subjects, and criminal penalties including imprisonment for willful or negligent violations. CareMed's current data management practices, characterized by the absence of any formal access

control, authentication, or security mechanism for customer personal data, represented a direct non-compliance with these obligations and expose the enterprise to material legal liability.

The broader national digital transformation context added a further dimension of relevance to the proposed system's development. The Philippine Digital Transformation Strategy 2022-2025, developed by the Department of Information and Communications Technology [4], identified the digitalization of small enterprises as a core component of the country's economic development agenda, recognizing that the productivity benefits of digital transformation cannot be realized at the national scale without extending digital capability to the MSME sector that employs the majority of the Filipino workforce. The strategy specifically identified cybersecurity capability — the ability to not merely adopt digital tools but to use them securely — as a prerequisite for sustainable digital transformation that created lasting value rather than creating new risks. This policy context positions the proposed system as aligned with and contributing to a stated national priority, and situates the study's contribution not merely as a solution to CareMed's specific operational problems but as a demonstration of a replicable approach to secure MSME digitalization that the broader policy agenda required.

It was against this multifaceted backdrop — encompassing the expanding service obligations of the Philippine healthcare supply sector under universal health care policy, the operational limitations of manual management in a growing enterprise, the cybersecurity risks and legal compliance obligations arising from the Philippine Data Privacy Act, and the national priority of secure and inclusive digital transformation — that this study proposes the development of a Cybersecurity-Integrated Digital Infrastructure Model for CareMed Medical Supply Enterprise. The proposed system was

designed not as a digital replica of CareMed's manual processes but as a fundamentally re-architected operational platform in which functional automation and information security were developed simultaneously as co-equal, inseparable design priorities from the earliest stage of system conception. This security-by-design approach, recognized as a best practice engineering principle in frameworks including the OWASP Top 10 and the NIST Cybersecurity Framework, distinguishes the proposed system from generic management software that automates operations without embedding the security controls necessary to protect the sensitive data those operations generate. The system was designed with generalizability as an explicit consideration, so that its architecture and methodology served as a replicable model applicable to other small Philippine medical supply enterprises facing analogous operational and cybersecurity challenges.

The management information systems literature provided extensive theoretical grounding for understanding why manual record-keeping systems fail as organizational scale increases. Researchers consistently identified five primary categories of information quality failure in manual data management: accuracy failures from transcription errors introduced each time data was physically transferred between records; completeness failures from omission of data at recording due to time pressure or workflow interruption; timeliness failures from the delay between an event's occurrence and its entry into accessible records; consistency failures from the maintenance of multiple independent record sources that diverge over time; and accessibility failures from the physical barriers to locating needed information in distributed paper storage. All five failure categories were present and observable in CareMed's current environment and

were directly addressed by the proposed digital management system.

Research conducted by the Philippine Institute for Development Studies [5] on MSME productivity constraints identified record-keeping inadequacy as one of the three most frequently cited internal barriers to small business growth, alongside limited access to credit and inadequate workforce development. Enterprises that reach the operational scale where manual record-keeping begins to fail systematically experience a characteristic sequence of consequences: increasing customer complaint rates driven by fulfillment errors; rising staff error rates driven by the cognitive overload of managing high transaction volumes manually; growing management uncertainty from the inability to generate reliable operational overviews without prohibitive staff time investment; and escalating compliance risk from the accumulation of record-keeping deficiencies. CareMed's operational situation reflected this pattern precisely, and the proposed system was designed to addressed the root cause — the absence of integrated digital management infrastructure — rather than its symptoms.

The theoretical and empirical literature on information security in small enterprises provided equally important context for the cybersecurity dimensions of this study. Bada and Nurse's [6] analysis of cybersecurity challenges in small and medium enterprises identified three structural factors that consistently generate security vulnerabilities in this organizational category: resource constraints preventing investment in dedicated security expertise; awareness deficits leaving owners and staff uninformed about specific threats and their mitigations; and cultural factors that prioritize operational convenience over security discipline, particularly when security measures were perceived as adding friction to customer-facing processes. All three factors were present in

CareMed's environment. The enterprise lacks the budget for dedicated IT security supported; management and staff had not received formal cybersecurity awareness training; and the informal digital tools currently used for business records were adopted for operational convenience without consideration of their security implications. The proposed system addressed all three dimensions: it eliminates the needed for specialized security expertise by embedding controls into the system architecture itself; it reduces the awareness burden on staff by making secure behavior the default rather than requiring active security decisions for each interaction; and it implemented security in ways that supported rather than impede operational workflows.

The growing emphasis on cybersecurity in the Philippine regulatory environment added further urgency to the security integration objectives of this study. Beyond the Data Privacy Act of 2012, the Philippine government had progressively strengthened the cybersecurity regulatory landscape through instruments including the Cybercrime Prevention Act of 2012 (Republic Act No. 10175) [7], which criminalizes unauthorized access to computer systems and the misuse of personal data obtained through digital means, and the National Cybersecurity Plan 2023-2028 [8], which articulates the government's strategy for developing national cybersecurity capability across all sectors of the economy including the MSME sector. The National Privacy Commission's series of implementing circulars and advisory opinions had progressively clarified and in some cases expanded the specific technical and organizational security measures expected of personal information controllers, reinforcing the legal context within which CareMed's current data management practices represented a compliance gap with material legal consequences.

The intersection of operational management and cybersecurity requirements in the proposed system's design

reflected a recognition in the information systems research community that these two dimensions cannot be addressed effectively in isolation. A management system that automates business processes without securing the data those processes generate created an expanded digital attack surface without the protections needed to defend it — potentially more dangerous than the manual system it replaces, because digital concentration of business data makes a successful breach more consequential. Conversely, a security framework applied to a system that fails to effectively automate and integrate business processes created compliance overhead without delivering the operational value that motivates sustained secure use. The proposed system was designed to be simultaneously operationally effective and genuinely secure, treating usability-driven adoption and security discipline as complementary design objectives and demonstrating through its architecture that these goals were achievable together within small enterprise resource constraints.

CareMed's operational profile — a small single-location enterprise managing combined sales and rental operations, employing a small staff team without IT supported, serving a local community customer base in the Philippine context — was representative of a large population of small medical supply enterprises operating across the country's cities, municipalities, and provinces. The Department of Health's licensing database for healthcare-related commercial establishments identified thousands of registered medical supply enterprises across all regions of the Philippines, the overwhelming majority of which were classified as micro or small enterprises sharing CareMed's characteristic constraints of limited capital, small staff teams, and management by owners without formal technology backgrounds. The system architecture, development methodology, security design framework, and evaluation approach documented in this study

were designed with this representative population in mind, ensuring that the solutions developed for CareMed were practically applicable to the broader sector rather than being specific to an atypical enterprise context.

## 1.2 Statement of the Problem

General Problem

CareMed Medical Supply Enterprise currently conducts all of its business operations — encompassing inventory management, sales transaction recording, equipment rental lifecycle management, customer record maintenance, financial tracking, and business performance reporting — through entirely manual, paper-based processes that had remained structurally unchanged since the enterprise's establishment. This sustained reliance on manual methods generates compounding, mutually reinforcing limitations across every operational dimension. Operational efficiency was constrained by the time and labor required for manual recording, physical retrieval, and data compilation tasks that consume staff capacity otherwise available for customer service. Data accuracy was repeatedly undermined by transcription errors introduced each time information was physically transferred between records, copied from one logbook to another, or verbally communicated without written verification. Record security was fundamentally compromised by the complete absence of any authentication, access control, or data protection mechanism governing who may access, modify, or remove business records and the personal information of customers they contained. Business responsiveness was limited by the multi-hour process required to compile even basic operational summaries from dispersed physical sources. The cumulative effect of these interrelated limitations created an environment where the risk of service failure, record loss, data protection non-compliance, and management decisions based

on incomplete or inaccurate information grew in direct proportion to the business's growth. There was a critical needed for a secure, integrated digital management system that can systematically automate CareMed's core business processes while incorporating cybersecurity controls to protect organizational and customer data in compliance with Philippine law.

Specific Problems

The following specific problems had been identified through structured observation of CareMed's operational environment, assessment of existing record-keeping practices, analysis of operational failures attributable to system limitations, and preliminary interviews with management and staff:

First, the complete absence of real-time inventory visibility in CareMed's current management approach created persistent operational uncertainty that directly impairs service quality and revenue realization. Inventory tracking relies exclusively on manually maintained stock cards updated at irregular intervals following transactions, combined with periodic full physical counts when management perceives a needed to verify stock positions. Between counting events — which may be separated by days or weeks depending on operational pressures — the stock card records may not accurately reflected the cumulative effect of transactions that had occurred since the last update, particularly during high-volume periods when competing demands cause staff to defer record updates. This gap between recorded and actual stock positions means that a staff member responding to a customer availability inquiry cannot reliably confirm availability from records alone and may needed to physically check the storage area, delaying service and creating inconsistencies when simultaneous transactions reference the

same inventory. The absence of automated reorder threshold monitoring means that stock depletions were detected only when a customer attempts to order an unavailable item, at which point the enterprise had already failed to maintained the proactive supply management that reliable service delivery required. Additionally, the absence of item-level tracking in the rental inventory pool created the risk that a single item may be simultaneously committed to multiple customers — a damaging outcome in a medical equipment context where no suitable substitute may be available for a patient with specific clinical requirements.

Second, the exclusively physical nature of CareMed's record system created vulnerability and accessibility limitations that compound over time as records accumulate. All business documentation — handwritten sales invoices, printed rental agreement forms, customer information notebooks, inventory logs, and payment records — exists solely in physical form, stored at a single location accessible to any person present on the business premises without authentication or access restriction. Physical records were vulnerable to irreversible loss events including water damage, fire, humidity-induced paper deterioration, accidental misplacement, or deliberate unauthorized removal, with no backup or recovery mechanism available. Retrieval of any specific historical record required manual searching through organized storage systems, creating response delays when customers or management required historical transaction data. Cross-referencing between record categories — identifying all transactions associated with a particular customer, tracing the rental history of a specific item, or compiling revenue data for a defined period — required manual compilation from multiple physical sources, a process prone to omission errors when the volume of records to be searched was large and that may

required hours of staff time that could otherwise be devoted to customer service activities.

Third, CareMed currently operated with a complete absence of cybersecurity controls governing access to the business and customer information maintained across its records. There was no authentication system controlling which individuals can access business records in any format. There were no user roles establishing differentiated access rights based on operational necessity. There was no session management governing digital access periods or enforcing termination policies. There was no audit trail documenting which individuals had accessed or modified specific records at specific times. The practical consequence was that the full scope of CareMed's information — including sensitive customer personal data such as home addressed, contact numbers, and medical equipment usage histories; commercially sensitive pricing and supplier information; and confidential financial data — was accessible without restriction to any person present on the premises or with access to any device where informal digital records were maintained. This situation was directly inconsistent with the obligations of the Philippine Data Privacy Act of 2012, which required all personal information controllers to implemented appropriate organizational, physical, and technical security measures regardless of organizational size. The current data management practices expose the enterprise to administrative sanctions, civil liability, and criminal prosecution under the Act, as well as to the reputational harm that would follow any customer data protection failure.

Fourth, the management of rental equipment under the current manual system was particularly and systematically vulnerable to operational failure as the concurrent rental portfolio grew. CareMed maintained a rotating pool of rental items lent to customers under individual paper rental

agreements specifying the customer's details, items rented, rental start and return dates, and deposit arrangements. Monitoring this portfolio for overdue returns required staff to periodically review the physical rental folder — a process that was not systematically scheduled, occurs only when competing operational demands permit, and became progressively less reliable as the number of agreements requiring review increases. Overdue returns were therefore identified sporadically and reactively rather than systematically and proactively. The operational consequences accumulate: overdue items were unavailable for subsequent customers; extended rental revenue was not systematically tracked; deposit records were not definitively linked to specific items; and the business lacks the documentation needed to supported a consistent, fair process for addressing disputed return dates. Each consequence individually represented a significant operational and financial impact; in combination, they constitute a structural revenue leakage and service quality risk that grew with the rental volume.

Fifth, the generation of business performance information under the current manual system was an entirely labor-intensive process that structurally prevents the timely, data-informed management decision-making that effective business governance required. Generating any operational summary — monthly sales figures, active rental counts, inventory valuation, or customer outstanding obligations — required staff to manually compile figures from multiple physical record sources, reconcile discrepancies between overlapping records, and produce a summary through manual calculation. This process was estimated to required a minimum of two to three hours for a basic monthly report involving only the most recent period, and significantly longer for analyses requiring comparison across multiple periods or cross-referencing between record categories. The resulting reports reflected the

state of records at the time of the most recent physical recording rather than the actual current state of operations, because transactions occurring after the last record update were not captured in the compiled summary. Management decisions made on the basis of these delayed, labor-intensive, and inherently approximate reports were therefore subject to the compounding uncertainties of manual recording errors, retrieval omissions, and the lag between operational events and their reflection in management information. This constraint on management information quality represented a fundamental limitation on the enterprise's capacity for effective governance, strategic planning, and responsive customer service management.

## 1.3 Objectives of the Study

### *1.3.1 General Objective:*

The general objective of this study was to design, developed, and comprehensively evaluate a cybersecurity-integrated web-based management system for CareMed Medical Supply Enterprise that measurably improves the operational efficiency, data accuracy, information security, and regulatory compliance posture of the organization's core business processes across the domains of inventory management, sales transaction processing, equipment rental lifecycle management, customer relationship management, and business performance reporting. The proposed system was conceived not merely as a digital replacement for CareMed's existing manual processes but as a purposefully re-architected operational platform in which functional automation and information security were developed simultaneously as inseparable design priorities, guided by the security-by-design philosophy recognized in the OWASP Top 10 and ISO/IEC 27001 frameworks. The system was additionally intended to served as a practically implemented, academically documented, and replicable model that other small Philippine

enterprises in the healthcare supply sector can study, adapt, and apply within their own operational environments, demonstrating that robust operational automation and genuine cybersecurity integration were achievable within the financial, technical, and organizational resource constraints characteristic of small enterprises in this sector.

### *1.3.2 Specific Objectives:*

In systematic pursuit of the general objective stated above, this study aims to accomplish the following five specific, measurable, and interconnected objectives:

The first specific objective was to conduct a systematic, formally documented analysis of CareMed's existing manual workflows across all five operational areas using structured management and staff interviews, direct observational assessment of business processes and record systems, and review of existing physical records, producing a formally documented requirements specification that identified the complete set of functional, non-functional, security, and interface requirements the proposed system must satisfy. This specification would be validated with CareMed management prior to the commencement of system design, ensuring that the system directly addressed operational realities and user needed with specificity and accuracy rather than imposing a generic solution that may not align with the enterprise's specific workflows and user capability profile.

The second specific objective was to design and developed a fully functional web-based management system incorporating six integrated modules: an Inventory Management Module providing real-time stock tracking, movement history recording, and automated low-stock alerting; a Customer Management Module providing a centralized digital customer record repository with comprehensive search, retrieval, and activity summary functions; an Order and Sales Processing

Module managing the complete lifecycle of sales transactions from placement through fulfillment and payment confirmation; an Equipment Rental Tracking Module providing systematic management of the complete rental lifecycle including check-out recording, return date monitoring and overdue flagging, automated alert generation, return processing, and inventory restoration; a Reports and Dashboard Module providing automated generation of operational reports and real-time management dashboards; and a Cybersecurity Module implementing the complete security architecture governing all access, data protection, and audit functions. The system was developed using Python 3 with the Flask micro-framework, MySQL relational database managed through the SQLAlchemy ORM, and Bootstrap 5 for the responsive front-end interface.

The third specific objective was to design and implemented a comprehensive, defense-in-depth cybersecurity architecture as an integral property of the system's core design from its initial architecture phase, encompassing: secure user authentication using Flask-Bcrypt for bcrypt-hashed credential storage; brute-force protection through Flask-Limiter rate limiting on the authentication endpoint; role-based access control implemented through Flask-Login role checking and custom route protection decorators; server-side session management with configurable inactivity timeout and secure cookie configuration; input validation and sanitization through Flask-WTF validators and SQLAlchemy parameterized queries for all database interactions; output encoding through Jinja2's auto-escaping for all template-rendered content; CSRF token validation for all state-changing requests through Flask-WTF; HTTPS enforcement for all client-server communications; and a comprehensive system event audit log recording all significant security and operational events with immutable timestamps, user identifiers, IP addressed, and descriptive action summaries. The entire architecture was

designed with explicit reference to the OWASP Top 10 and the Philippine Data Privacy Act of 2012.

The fourth specific objective was to conduct systematic, multi-level, formally documented testing across five modalities: unit testing verifying the correct behavior of individual system component functions in isolation; integration testing verifying the accuracy of module interactions and data flows; comprehensive system testing against the complete functional requirements specification using a representative test data set; security testing applying structured manual test procedures based on the OWASP Top 10 framework to verify the system's resistance to each of the ten critical web application vulnerability categories; and User Acceptance Testing conducted with representative participants from both Administrator and Customer user role groups, evaluating system usability and functional completeness through structured testing scenarios derived from CareMed's actual operational workflows. All testing was documented through formal test plans, test case records, and security assessment reports.

The fifth specific objective was to evaluate the overall quality and acceptability of the developed system through a structured formal evaluation process using an instrument designed with reference to the ISO/IEC 25010 software product quality standard, assessing the system across five quality dimensions: functional suitability, measuring the completeness and correctness of the system's functional coverage; performance efficiency, assessing responsiveness and resource utilization; usability, evaluating the learnability and operability of the system interface; reliability, measuring consistency of performance under normal conditions; and security, assessing the effectiveness of implemented cybersecurity controls. The instrument was administered as a five-point Likert scale questionnaire to a purposively

selected sample of administrator-role and customer-role evaluators, with responses analyzed using descriptive statistics to produce a formal quantitative quality assessment across all evaluated dimensions.

## 1.4 Scope and Delimitation

### *1.4.1 Scope of the Study:*

This study encompasses the complete design, development, cybersecurity integration, multi-level testing, and formal quality evaluation of a web-based digital management system for CareMed Medical Supply Enterprise. The scope was defined across four primary dimensions: the functional coverage of the system, the security architecture of the system, the target user population, and the technical platform. Each was described in the following subsections.

The Inventory Management Module provided real-time visibility into the availability and status of all products in both the sale inventory and rental pool. The module supported three primary transaction types: stock-in transactions recording additions following supplier deliveries or rental returns; stock-out transactions recording removals through sales fulfillment or rental check-out; and adjustment transactions recording corrections following physical verification exercises. Each transaction was captured with a timestamp, transaction type, quantity change, resulting stock level, user identifier, and optional reference notes. The module maintained a continuous chronological movement history for each product, supporting routine inventory verification and forensic investigation of discrepancies. Automated low-stock alerts were generated when available quantity falls below an administrator-configured threshold, enabling proactive procurement action before stockout conditions affect customer service. Product catalog management functions allowed administrators to added and update products, configure rental

availability and pricing, and set reorder thresholds for each item.

The Customer Management Module provided a centralized, searchable, and access-controlled digital repository replacing the current dispersed system of physical customer notebooks and informal digital records. The module maintained a structured profile for each registered customer capturing full name, contact information, addressed, registration date, and a complete transaction history linking the customer record to all associated sales orders and rental agreements. Search functionality allowed administrators to locate records using partial name matching, contact number lookup, or transaction reference identifiers, enabling rapid customer identification during service interactions. A comprehensive activity summary view for each customer displays all historical and current transactions in a unified chronological display with status indicators. A customer-facing self-service portal enabled customers to register, log in securely, browse the product and rental catalog, submit orders, view active order and rental status, access transaction history, and update contact information.

The Order and Sales Processing Module manages the complete transactional lifecycle of product sales from order submission through fulfillment and payment confirmation. Administrator-role users maintained a structured product catalog with item names, descriptions, available quantities, and pricing. Customer-role users browse the catalog and submit orders specifying delivery or pickup preferences. Administrators process submitted orders through a defined status lifecycle — received, confirmed, preparing, ready for dispatch, and completed — with automatic inventory quantity deductions upon order confirmation. Payment confirmation was managed through a reference submission workflow aligned with Philippine payment practices: customers submit payment

reference numbers following payment through their preferred channel, and administrators review and confirm receipts in the administrative panel. A comprehensive order history provided a searchable and filterable record of all transactions accessible to administrators for management and to customers for their own records.

The Rental Tracking Module addressed CareMed's most operationally critical management gap by providing a systematic, continuously visible, and automated approach to managing the complete rental lifecycle. Rental agreements were created at equipment check-out capturing the customer identifier, a detailed listing of rented items referenced to inventory records, rental start and return dates, the rental fee structure, and any deposit amount collected. The rental status dashboard provided a continuously updated overview of the complete active portfolio organized by return status: active rentals with days remaining, rentals approaching return within a configurable advance warning period highlighted with alert indicators, and overdue rentals prominently flagged with days elapsed beyond the scheduled return date. Automated overdue alert notifications were presented to administrators at each login, ensuring systematic monitoring regardless of competing operational demands. Upon confirmed item return, the system records the return event, marks the agreement as completed, and automatically restores returned items to available inventory for subsequent bookings.

The Reports and Dashboard Module provided automated generation of the operational reports that currently required hours of manual compilation. Four primary report categories were generated on demand for administrator-specified time periods: sales summary reports presenting transaction counts, gross revenue, and product movement data organized by category; rental status reports summarizing agreements initiated, active, completed, and overdue in the period with

associated revenue and duration statistics; inventory status reports displaying current stock levels with reorder threshold comparisons and high-movement product identification; and customer activity reports summarizing transaction patterns and identifying customers with overdue rental obligations. All reports were presented in dashboard-integrated format with headline summary statistics and sortable tabular detail views, and were available in printable download formats. Access to all reporting and dashboard functions was strictly restricted to authenticated Administrator-role accounts.

The Cybersecurity Module constitutes the security infrastructure layer governing all access, data protection, authentication, authorization, and audit functions across the entire system. Authentication was implemented through Flask-Login with Flask-Bcrypt managing all password hashing and verification at a configured work factor. Brute-force protection was implemented through Flask-Limiter rate limiting applied to the authentication endpoint. Role-based access control was enforced through Flask-Login role checking and custom Python route decorators on every protected route. Session management uses Flask-Login's permanent session with configurable inactivity timeout, secure HttpOnly and SameSite cookie settings, and complete server-side session destruction on logout. Input security uses Flask-WTF validators and SQLAlchemy parameterized ORM queries throughout the data access layer. CSRF protection uses Flask-WTF's synchronizer token pattern for all state-changing POST requests. Output encoding uses Jinja2's auto-escaping for all template-rendered content. The system audit log was implemented as a dedicated database table written through a centralized logging service called from every significant event handler, recording timestamp, user identifier, role, source IP addressed, action type, affected record identifiers, and a descriptive action summary for every auditable system event.

The target user population comprises two roles reflecting CareMed's organizational structure. Administrator-role users encompass management and authorized administrative staff with full access to all system modules and functions, including all management dashboards, reporting capabilities, inventory and customer record management, and administrative configuration functions. Customer-role users encompass registered customers with access restricted to the customer-facing portal for product browsing, order submission, rental status viewing, transaction history access, and personal account management. Both roles interact with the system through standard web browser interfaces accessible from any internet-connected device, requiring only basic computer and web browser literacy without specialized technical training.

The technical platform comprises a web-based application developed using Python 3 with Flask as the application server, MySQL as the relational database management system accessed through the SQLAlchemy ORM, and Bootstrap 5 as the front-end framework providing a responsive, mobile-compatible interface. The system follows a Model-View-Controller architectural pattern with Flask route handlers as controllers, Jinja2 HTML templates as views, and SQLAlchemy model classes as the data layer. The system was designed for deployment on standard web hosting infrastructure accessible through any modern web browser, with no client-side software installation requirements.

### *1.4.2 Delimitations of the Study:*

The following delimitations defined the specific boundaries of this study, clearly distinguishing the scope of the current research from related areas that, while potentially valuable as future research directions or system enhancement priorities, fall outside the objectives achievable within this capstone project's constraints.

The system developed in this study was a web-based application delivered through standard web browser clients and did not include a dedicated native mobile application for iOS or Android platforms. While Bootstrap 5's responsive design ensured that the interface adapts to mobile screen dimensions and provided functional mobile browser access, the current system did not supported native capabilities including push notification delivery for rental reminders or reorder alerts, offline data access for environments without reliable internet connectivity, or device hardware integration such as camera-based barcode scanning for inventory management. These capabilities represented potentially valuable future enhancements whose development would required separate platform-specific codebases, additional testing environments, and application store distribution processes — requirements exceeding this project's scope. The development of native mobile extensions was identified as a priority direction for future research.

Direct integration with external electronic payment processing services was not implemented in this study. Integration with Philippine mobile payment platforms such as GCash and Maya, or online banking transfer gateways, required merchant account registration, satisfaction of regulatory requirements under Bangko Sentral ng Pilipinas circulars governing electronic payment processing, implementation of provider-specific API integrations with associated security compliance requirements, and ongoing maintenance obligations — requirements that extend beyond the scope of a capstone research project. Payment confirmation in the current system uses the reference submission workflow that reflected the actual payment behavior of CareMed's existing customer base. Direct payment gateway integration was identified as a natural future enhancement as the enterprise's digital customer adoption matures.

The system was designed as a standalone operational management platform and did not include integration with external healthcare information infrastructure, including electronic health record systems, hospital management systems, pharmacy information systems, or government health databases such as the Philippine Health Information System. CareMed's role as a medical equipment sales and rental enterprise — not as a clinical care provider — did not required clinical data exchange with healthcare facilities. The system's customer data model captures only the personal and transactional information necessary for managing commercial equipment transactions and did not include clinical data fields, health record linkages, or other clinical information infrastructure elements. The system did not implemented clinical data standards such as HL7 FHIR or standardized medical terminologies such as ICD-10 or SNOMED CT.

Security testing employs structured manual testing procedures based on the OWASP Top 10 framework applied by the research team following documented test plans and did not include formal penetration testing by independent accredited cybersecurity professionals or automated vulnerability scanning using commercial security assessment platforms. This delimitation reflected the resource and operational constraints of an academic capstone project rather than a judgment that these approaches lack value. Independent penetration testing by accredited professionals represented the gold standard for security validation in production deployment contexts and was strongly recommended for any deployment beyond the academic environment. The security testing in this study was calibrated to verify that the system addressed the most commonly exploited web application vulnerability categories within the OWASP framework, establishing a documented security baseline supporting the

more comprehensive assessment a production deployment would required.

This study addressed the requirements analysis, design, implementation, testing, and evaluation phases of the development lifecycle and did not extend to production deployment at CareMed, post-deployment operational monitoring, system maintenance and security patching, ongoing user supported operations, or longitudinal assessment of operational impact. The evaluation was conducted at the conclusion of the development and testing phases using a structured User Acceptance Testing protocol and ISO 25010-based quality evaluation instrument administered within the academic project timeframe. Assessment of the system's long-term operational performance, its security resilience under real-world conditions, and its measurable impact on CareMed's business efficiency and data protection compliance over an extended deployment period were identified as priority directions for future longitudinal research.

## 1.5 Significance of the Study

The development and evaluation of a cybersecurity-integrated web-based management system for CareMed Medical Supply Enterprise was anticipated to generate substantive and enduring benefits for a diverse range of stakeholders connected to the enterprise's operations, to the Philippine healthcare supply sector, and to the academic, professional, and policy communities engaged with the challenges of digital transformation and cybersecurity governance in small enterprise environments. The following sections articulate the specific significance of this study for each identified beneficiary group.

CareMed Management

CareMed's management team was the most immediate and operationally direct primary beneficiary. The system would

fundamentally transform the management information environment by replacing delayed, labor-intensive, and inherently approximate manual reporting with a real-time digital intelligence infrastructure providing immediate, accurate, and continuously updated visibility across all business dimensions. Management would acquire the capability to assess current inventory positions and approaching reorder requirements at any time through the live inventory dashboard; monitor the complete active and overdue rental portfolio without dependence on periodic manual folder reviews; access sales performance data for any specified period through automated report generation in minutes; and evaluate customer activity patterns to supported data-informed relationship management decisions. This transformation directly enhanced the capacity for timely, evidence-based decision-making across procurement planning, rental inventory optimization, and financial performance monitoring. The embedded cybersecurity architecture additionally provided the data protection infrastructure necessary for compliance with the Philippine Data Privacy Act, materially reducing the enterprise's legal liability exposure and building organizational capability for secure digital operations that increasingly sophisticated customers and regulatory environments would demand.

Employees and Administrative Staff

CareMed's employees and administrative staff would experience direct, material improvements in their day-to-day working environment through systematic reduction of the manual record-keeping and retrieval workload that currently consumes substantial operational time and cognitive capacity. The time devoted to maintaining handwritten records, searching physical files for customer information, compiling rental status summaries, and calculating inventory balances was significantly reduced as the system automates these routine functions, releasing staff capacity for higher-value customer-

facing activities — product consultation, order coordination, proactive rental follow-up, and relationship management — that directly contributed to service quality and business development. The system's search and retrieval capabilities would allowed staff to locate any customer record, transaction history, or inventory status within seconds through a simple search interface, dramatically improving the speed and quality of their responses to customer inquiries. The clearly defined role-based access control architecture would provided each staff member with an unambiguous specification of their authorized system functions, reducing the role ambiguity that currently arises from the absence of formal access governance.

Customers of CareMed

CareMed's customers — individual patients, family caregivers, and small institutional clients — would experience substantive improvements in service quality, transparency, and information access as a direct result of the proposed system. The customer-facing self-service portal would provided a convenient digital channel through which customers can browse products and rental items, submit orders, monitor real-time order and rental status, access transaction history, and update account information from any internet-connected device at any time, without requiring phone calls or physical visits. The systematic rental agreement management would ensured customers receive accurate, documented information about their rental terms and return obligations, with consistent return-related communications based on system-verified records rather than manually compiled notes subject to transcription error. The system's data protection architecture would ensured that customers' personal information — including sensitive details of medical equipment usage — was stored in a secured, access-controlled, and audited environment, providing meaningful assurance that their privacy was protected in accordance with the Data Privacy Act of 2012.

Future Researchers

This study contributed to the academic literature on digital transformation and cybersecurity integration in Philippine small and medium enterprises, specifically within the healthcare supply sub-sector — a segment that had received comparatively limited dedicated research attention despite its significance as a service provider to vulnerable patient populations. The comprehensive developmental research methodology employed — structured workflow analysis, security-by-design architecture, multi-level testing including OWASP Top 10-structured security testing, and ISO 25010-based quality evaluation — provided a complete and replicable methodological framework that future researchers undertaking similar studies can draw upon, critique, or extend. The systematic documentation of operational challenges, technical design decisions with their rationale, cybersecurity mechanisms and their regulatory relationships, and formal evaluation findings constitutes a referenceable case study for researchers studying related enterprise types — pharmacies, dental supply companies, laboratory equipment providers, or other healthcare-adjacent MSME categories. Future researchers may build upon this study by investigating the system's extended operational performance in a longitudinal deployment study, expanding security testing to include formal penetration testing, or extending the architecture with the native mobile, payment gateway, or healthcare system integration capabilities identified in the delimitations.

Information Technology Practitioners and Software Developers

For IT practitioners and software developers working on management systems for small enterprises in regulated or data-sensitive sectors, this study provided a practically implemented and academically validated reference for security-

by-design development within resource-constrained project environments. The technology selection analysis comparing Python Flask, MySQL, and Bootstrap 5 against alternatives in the Philippine small enterprise development context provided a referenceable evaluation framework for practitioners facing similar decisions. The cybersecurity architecture implementation — covering bcrypt authentication, Flask-Login role-based access control, Flask-Limiter brute-force protection, Flask-WTF CSRF and input validation, SQLAlchemy parameterized SQL injection prevention, Jinja2 XSS prevention, and comprehensive audit logging — constitutes a complete, end-to-end, practically validated reference that practitioners can adapt for their own projects. The OWASP Top 10-structured security testing methodology demonstrates an accessible, systematic approach to meaningful security verification applicable by development teams without dedicated security testing personnel, particularly relevant for small enterprise projects where formal security specialists were not available.

The Philippine Healthcare Supply Sector and Broader Society

At the sectoral and societal level, this study contributed to the advancement of secure, reliable, and patient-centered digital operations within the Philippine healthcare supply ecosystem. The ongoing UHC Act implementation was progressively extending formal healthcare coverage to communities that depend on local small enterprises for access to essential medical equipment. The reliability of these enterprises — their ability to maintained accurate inventory, manage rental equipment responsibly, protect customer personal data, and deliver consistent, professional service — was therefore not merely a commercial performance metric but a public health consideration with community-level implications. By developing, documenting, and evaluating a viable, affordable model for secure digitalization in a small

medical supply enterprise, this study contributed practically to raising the service standard and data protection capability of this sector. The replicability of the proposed system means its benefits were not limited to CareMed but were available to the broader population of small Philippine medical supply enterprises facing similar operational challenges, contributing to the national goal of an inclusive digital economy in which the protections and capabilities enabled by digital technology were accessible to small enterprises serving everyday Filipino communities.

Beyond the immediate beneficiaries identified in the preceding subsections, this study carries significance for the Philippine regulatory community engaged with the practical implementation of the Data Privacy Act of 2012 in the MSME sector. The National Privacy Commission had consistently recognized that the vast majority of personal information controllers in the Philippines were small enterprises that process customer personal data as a routine aspect of their commercial operations — taking customer names, contact numbers, and addressed for order fulfillment, maintaining purchase and transaction histories, and recording the personal details of individuals who engage with them as service providers. Demonstrating through a concrete, fully documented implementation that a small enterprise can build a genuinely compliant data management system within its operational resource constraints — and that doing so improves rather than impedes operational effectiveness — contributed to the empirical basis for regulatory guidance that was practically achievable for the MSME sector, not merely aspirationally desirable from a policy perspective.

The academic community engaged with the study of digital transformation, management information systems, and cybersecurity in developing country contexts would find in this study a case study that contributed to the empirical

literature on technology adoption in under-resourced organizational settings. The Philippines presents a particularly interesting research context for these questions because of the combination of factors present: a large and economically significant MSME sector with limited technology adoption; an advanced and actively enforced personal data protection legal framework creating compliance obligations that exceed current MSME capability; a rapidly expanding healthcare delivery system under the UHC framework that was creating new operational pressures on health supply chain participants; and a national digital transformation policy agenda with explicit MSME digitalization objectives. The system developed in this study and the methodology through which it was developed and evaluated contributed an empirical data point to the literature on how these factors interact in practice, and how system design approaches rooted in security-by-design principles can addressed the intersection of operational and security requirements in this complex environment.

## 1.6 Definition of Terms

The following terms are defined as they are specifically applied within the context of this study. Definitions are presented in alphabetical order and are intended to provide clear and precise explanations of technical, legal, procedural, and specialized terms used throughout the research documentation, ensuring that the study is interpretable by readers with varying levels of familiarity with its technical and regulatory subject matter. Where a term has both a general meaning in common usage and a specific meaning within this study, both meanings are noted and the specific application is clarified.

Agile Development Methodology. A software engineering and project management philosophy characterized by iterative,

incremental delivery of working software organized into time-bounded development cycles called sprints, typically one to four weeks in duration. Agile methodology prioritizes continuous stakeholder collaboration, adaptability to evolving requirements, and the delivery of tested, working functionality at the conclusion of each sprint over comprehensive upfront documentation and rigid adherence to an initial plan. The Agile Manifesto, the foundational document of the Agile approach, articulates four core values: prioritizing individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; and responding to change over following a fixed plan. In this study, Agile methodology guides the organization of the development phase into structured sprints each targeting a specific system module, enabling continuous review and adaptation based on testing feedback and stakeholder input.

Audit Log. A chronological, sequentially ordered, and integrity-protected record of operationally and security-relevant events occurring within an information system, maintained with sufficient detail to enable retrospective reconstruction of the sequence of actions that produced the current state of the system's data and configuration. Individual audit log entries capture the precise timestamp of the event, the identifier and role of the user whose actions generated it, the source IP address of the user session, the specific action type performed, the identifiers of affected data records, and a descriptive summary sufficient to enable a reviewer to understand what was done without consulting other records. In this study, the audit log is implemented as a dedicated database table written to through a centralized logging service called from every significant system event handler, and access to audit log records is restricted exclusively to Administrator-role users.

Authentication. The process by which a digital system verifies the claimed identity of a user attempting to access protected resources before the access request is evaluated against applicable access control policy. Authentication typically requires the presentation of a knowledge factor (a password), a possession factor (a one-time code or hardware token), or an inherence factor (a biometric characteristic). In this study, authentication is implemented as a single-factor knowledge-based mechanism through Flask-Login, requiring a registered username and password verified by Flask-Bcrypt against a bcrypt-hashed representation stored in the user accounts table, ensuring that the actual password value is never stored in a recoverable form.

Authorization. The access control process by which a system determines what specific functions, data records, or resources an authenticated user is permitted to access based on the access policies associated with the user's assigned role. Authorization is logically distinct from authentication: authentication establishes who the user is, while authorization determines what that identity is permitted to do. In this study, authorization is implemented through role-based access control using Flask-Login's current_user role property and custom Python decorators applied to every protected route handler, returning an HTTP 403 Forbidden response to any request where the authenticated user's role does not include the required permission for the requested action.

bcrypt. A password-hashing function specifically designed for the secure storage of user authentication passwords, developed by Niels Provos and David Mazières. bcrypt incorporates three properties distinguishing it from general-purpose hash functions for password storage: a random salt value unique to each hash computation preventing pre-computed rainbow table attacks; integration of the salt into the stored

hash output enabling verification without separate salt storage; and an adjustable work factor controlling computational cost that can be increased as hardware speeds improve to maintain resistance to brute-force attacks. In this study, bcrypt is applied to all user passwords at account creation and password change events through the Flask-Bcrypt library, and the stored hash values are used for credential verification at authentication.

Bootstrap 5. The fifth major version of Bootstrap, an open-source front-end framework providing HTML structure conventions, CSS styling rules, utility classes, and JavaScript interaction components for building responsive, cross-browser-compatible web interfaces. Bootstrap 5's responsive grid system enables layouts that automatically reorganize for different viewport widths from narrow smartphone screens to wide desktop monitors. In this study, Bootstrap 5 is the primary front-end framework for all system user interfaces, ensuring consistent visual presentation, responsive mobile compatibility, and accessible UI component behavior.

Cross-Site Request Forgery (CSRF). A web application security attack in which a malicious actor tricks an authenticated user's browser into sending an unauthorized HTTP request to a target application in which the user has an active session, causing the application to execute an action using the user's session credentials without the user's knowledge or consent. CSRF attacks exploit the browser's automatic inclusion of session cookies with all requests to a domain regardless of request origin. In this study, CSRF attacks are mitigated through Flask-WTF's synchronizer token pattern requiring a session-specific CSRF token with every state-changing POST request, validated server-side before processing.

Cross-Site Scripting (XSS). A web application security vulnerability in which an attacker injects malicious client-side JavaScript into content served by a vulnerable application, causing that script to execute in the browsers of other users viewing the affected content. XSS vulnerabilities arise when user-supplied content is incorporated into HTML responses without encoding HTML special characters. Stored XSS embeds injected script in the database for persistent exposure; reflected XSS returns it in immediate responses; DOM-based XSS exists in client-side script logic. In this study, XSS is prevented through Jinja2's auto-escaping mechanism automatically encoding HTML special characters in all template-rendered content.

Cybersecurity. The multidisciplinary discipline and practice of protecting information systems, networks, applications, and digital data from unauthorized access, disclosure, modification, disruption, or destruction through the integrated application of technical security controls, organizational governance measures, and security-aware design and engineering practices. Technical controls include authentication, access control, encryption, input validation, output encoding, security monitoring and logging, and vulnerability management. Security-aware design — commonly described as "security by design" or "security by default" — integrates security requirements and threat analysis into the earliest design stages rather than treating security as a post-functional enhancement. In this study, cybersecurity is operationalized through specific technical controls implemented as integral architectural components from the initial design phase.

Flask. A lightweight Python web application micro-framework developed by Armin Ronacher under the Pallets Projects umbrella, providing URL routing, HTTP request and response handling, Jinja2 template rendering, and extensible

session management within a minimalist architecture that does not impose a specific application structure or bundle optional components by default. Flask's micro-framework philosophy provides a focused core that developers extend with purpose-specific libraries selected for each project's requirements. Key extensions used in this study include Flask-Login for session management and access control, Flask-Bcrypt for password hashing, Flask-WTF for form validation and CSRF protection, Flask-Limiter for rate limiting, and Flask-SQLAlchemy for ORM integration.

ISO/IEC 25010. An international standard jointly published by the International Organization for Standardization and the International Electrotechnical Commission providing a hierarchical software product quality model defining quality characteristics and sub-characteristics for software evaluation. The standard identifies eight primary quality characteristics: functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability. In this study, the evaluation instrument assesses the system against five dimensions of direct relevance to its objectives: functional suitability, performance efficiency, usability, reliability, and security, using a five-point Likert scale questionnaire with responses analyzed through descriptive statistics.

Model-View-Controller (MVC). A software architectural pattern organizing application code into three functionally distinct components. The Model manages data and business logic including database interaction, data validation, and entity relationships. The View manages presentation through template rendering generating HTML responses delivered to user browsers. The Controller manages request handling, invoking appropriate Model operations in response to incoming HTTP requests and selecting the appropriate View for the response. In this study, the system follows the MVC pattern with Flask

route handlers as controllers, Jinja2 HTML templates as views, and SQLAlchemy model classes as the data layer.

MySQL. An open-source relational database management system based on the Structured Query Language standard, currently maintained by Oracle Corporation, organizing data into structured tables with defined schemas and inter-table relationships expressed through foreign key references. MySQL provides ACID-compliant transaction processing ensuring atomicity, consistency, isolation, and durability for database operations. Its reliable performance, comprehensive SQL feature support, wide adoption in Philippine web hosting environments, and compatibility with the Flask/Python technology stack make it a practical choice for small enterprise web application databases. In this study, MySQL serves as the system's database management system with all interactions performed through SQLAlchemy ORM parameterized queries.

OWASP Top 10. A periodically revised consensus document published by the Open Web Application Security Project identifying the ten most critical and widely exploited security risk categories affecting web applications, compiled from vulnerability data contributed by application security firms and technology organizations worldwide. The 2021 edition identifies: A01 Broken Access Control; A02 Cryptographic Failures; A03 Injection; A04 Insecure Design; A05 Security Misconfiguration; A06 Vulnerable and Outdated Components; A07 Identification and Authentication Failures; A08 Software and Data Integrity Failures; A09 Security Logging and Monitoring Failures; and A10 Server-Side Request Forgery. In this study, the OWASP Top 10 serves as the primary reference framework for both the system's security architecture design and the security testing phase.

Philippine Data Privacy Act of 2012 (Republic Act No. 10173). A Philippine law enacted August 15, 2012 establishing the comprehensive legal framework for personal information protection by both government agencies and private sector entities operating in the Philippines. The Act defines personal information broadly as any data from which individual identity can reasonably be ascertained and establishes sensitive personal information — including health, medical, and financial information — as requiring heightened protection. All personal information controllers are required to implement organizational, physical, and technical security measures appropriate to the nature and volume of personal data processed, appoint Data Protection Officers, notify the National Privacy Commission and affected data subjects of personal data breaches, and respect data subject rights. Penalties for violations include administrative fines up to five million pesos, civil damages, and criminal imprisonment for willful or negligent violations. In this study, compliance with the Act's security requirements is a primary driver of the system's cybersecurity architecture design.

Python. A high-level, general-purpose programming language known for its clear readable syntax, extensive standard library, and broad ecosystem of third-party packages supporting web development, data science, automation, and artificial intelligence applications. Python's combination of developer productivity, readability, and the rich ecosystem of web development frameworks available through the Python Package Index make it a widely adopted language for web application development in both academic and commercial contexts. In this study, Python 3 serves as the primary programming language for the system's back-end application logic, implemented with the Flask framework for request handling and routing and the SQLAlchemy library for database interactions.

Rental Management. The set of business processes managing the lending of physical items — in this study, durable medical equipment — to customers for defined rental periods in exchange for fees, including: systematic recording of rental agreements at equipment check-out with complete documentation of terms and item details; ongoing monitoring of each item's rental status throughout the rental period; generation of return reminders and overdue alerts; recording of equipment returns with inventory restoration; and management of deposits, rental fee calculation, and payment confirmation. In this study, the Rental Tracking Module provides digital infrastructure replacing the current paper-based rental management process.

Role-Based Access Control (RBAC). An access control model in which users' permissions to access system resources and execute functions are determined by the roles assigned to their accounts rather than by permissions assigned directly to individual users. Roles are defined as named permission sets corresponding to organizational functions, and users are assigned to roles based on their responsibilities. Permissions are assigned to roles rather than individual users, so changes to the permission requirements of an organizational function affect all users in that role simultaneously. RBAC simplifies security administration, reduces misconfiguration risk, and supports the principle of least privilege ensuring users have access only to functions required for their specific responsibilities. In this study, Administrator role users have full system access and Customer role users have access restricted to the customer-facing portal functions.

Session Management. The mechanisms governing the complete lifecycle of an authenticated user's interaction session with a web application from successful login through session termination. Key components include: secure generation of session tokens with sufficient randomness to resist

prediction; server-side session data storage with the client holding only an opaque token; secure cookie attribute configuration including HttpOnly to prevent JavaScript access, Secure to enforce HTTPS-only transmission, and SameSite to prevent cross-origin inclusion; inactivity timeout policies automatically invalidating idle sessions after a defined period; and complete session destruction on user-initiated logout. In this study, session management is implemented through Flask-Login with configurable inactivity timeout and secure cookie settings.

SQL Injection. A web application security vulnerability in which an attacker introduces malicious SQL code fragments into user input fields that are subsequently incorporated into SQL query strings executed against the backend database without adequate validation or sanitization. The attack exploits string concatenation-based query construction to insert SQL syntax that alters the query's logical structure — appending UNION SELECT statements to extract unauthorized data, adding OR 1=1 conditions to bypass authentication WHERE clauses, or injecting DROP TABLE statements to destroy data. Successful attacks can expose the entire database contents, bypass authentication without valid credentials, and in some configurations execute operating system commands. In this study, SQL injection is prevented by exclusive use of SQLAlchemy ORM parameterized query methods ensuring user-supplied values are treated as data rather than SQL syntax.

Universal Health Care Act of 2019 (Republic Act No. 11223). A landmark Philippine law enacted February 20, 2019 mandating automatic enrollment of all Filipino citizens in the National Health Insurance Program, guaranteeing access to a defined comprehensive health services package at no point-of-service cost to enrolled individuals, and establishing the legal and institutional framework for progressive expansion of health service delivery infrastructure across all levels of

the Philippine health system from national hospitals to primary care facilities at the municipal level. The UHC Act is relevant to this study as the primary policy driver expanding healthcare service access and thereby increasing demand for medical equipment and home care support services, creating the context of growing operational demand within which enterprises like CareMed must operate with greater efficiency and reliability.

User Acceptance Testing (UAT). The final phase of the software testing lifecycle in which representative end-users — individuals drawn from the actual target user population of the system rather than members of the development team — evaluate the system against their practical requirements and operational expectations by performing structured test scenarios that reflect real-world usage patterns. UAT is designed to validate that the system meets the business needs and usability expectations of its intended users in addition to satisfying the technical functional requirements verified during earlier testing phases. UAT findings may identify usability issues, workflow mismatches, or feature gaps that were not apparent during technical testing but become evident when the system is operated by non-technical users performing realistic tasks. In this study, UAT is conducted with representative participants from the Administrator and Customer user role groups using structured test scenarios derived from CareMed's actual operational workflows, with findings documented through structured observation forms and participant feedback questionnaires.

User Interface (UI). The set of visual elements, interaction controls, and information displays through which a user communicates with and receives information from a software application. Effective user interface design applies principles of usability — learnability, efficiency of use, error prevention, and user satisfaction — to create interfaces

that enable users to accomplish their operational goals with minimal training, low error rates, and high confidence. In web-based applications, the user interface is constructed using HTML for content structure, CSS for visual presentation and layout, and JavaScript for interactive behavior. In this study, the system's user interface is built using Bootstrap 5 HTML and CSS components following a responsive design approach that provides an appropriate visual presentation across both desktop and mobile screen sizes, and follows usability design principles aligned with the ISO 25010 usability quality characteristic.

Waterfall Development Model. A traditional sequential software development lifecycle model in which the development process is organized into discrete, linearly sequenced phases — requirements, design, implementation, testing, and deployment — each of which must be completed and formally approved before the subsequent phase begins. The Waterfall model's strengths include its clear phase boundaries, formal documentation requirements, and predictable schedule structure. Its limitations in the context of complex application development include its rigidity in accommodating evolving requirements discovered during development, and the deferral of working software and user feedback to late in the development process. The Waterfall model is contrasted in this study with the Agile Development Methodology selected for the CareMed system development, with the Agile approach chosen for its greater responsiveness to the requirements refinement needs of a development project conducted in close collaboration with an operational business environment.

Web Application. A software application that runs on a web server and is accessed by users through web browser clients over a network connection using standard internet protocols. Web applications differ from desktop applications in their centralized server-side architecture, browser-based

client interface, and the use of HTTP or HTTPS as the primary communication protocol between client and server. Web applications are built using combinations of server-side programming languages and frameworks — such as Python with Flask, PHP with Laravel, or JavaScript with Node.js — that generate dynamic HTML responses to client requests, with client-side technologies including HTML, CSS, and JavaScript providing the interactive user interface. Security considerations in web applications are distinct from those in desktop applications, arising from the exposure of the application interface to network-based attacks and the stateless nature of the HTTP protocol that requires explicit session management mechanisms to maintain user context across requests.

Web-Based System. A software application hosted on a web server and delivered to users through standard web browser clients communicating over HTTP or HTTPS, without requiring installation of application-specific software on user devices. Web-based systems are deployed and maintained centrally on server infrastructure so that updates become immediately available to all users without client-side installation processes, and can serve simultaneous users from diverse device types and operating systems as the interface is generated as standard HTML, CSS, and JavaScript interpreted by the browser rather than platform-specific compiled code. In this study, the CareMed management system is developed as a web-based application accessible through any modern web browser from any internet-connected device, without client-side installation requirements, using the Flask Python framework server-side and Bootstrap 5 HTML/CSS client-side.

SQLAlchemy. An open-source Python SQL toolkit and Object-Relational Mapping (ORM) library that provides a set of high-level abstractions for interacting with relational databases through Python code rather than raw SQL strings. SQLAlchemy's

ORM layer maps Python class definitions — called model classes — to database table schemas, and translates operations on Python objects into the corresponding SQL queries executed against the database. The ORM's parameterized query construction ensures that all database interactions use properly escaped, type-validated parameters, preventing SQL injection attacks by design regardless of the content of user-supplied values. In this study, SQLAlchemy serves as the data access layer for all database interactions in the Flask application, with model classes defining the schema for each of the system's data entities and query operations expressed through SQLAlchemy's query API.

Jinja2. A modern, full-featured template engine for Python developed by Armin Ronacher and integrated as the default templating engine of the Flask framework. Jinja2 enables the generation of dynamic HTML responses by combining static HTML template files with dynamic data values provided by Flask route handler functions at the time of request processing. Jinja2's template syntax supports variable substitution, conditional logic, loop iteration, template inheritance, and macro definition, enabling the construction of complex, reusable HTML interfaces from modular template components. Critically for security, Jinja2 applies automatic HTML escaping to all variable values substituted into templates by default, encoding HTML special characters (<, >, &, ", ') as their corresponding HTML entity representations. This auto-escaping prevents cross-site scripting (XSS) attacks by ensuring that any user-supplied content rendered in a template cannot be interpreted as executable HTML or JavaScript code by the user's browser.

Principle of Least Privilege. A foundational security design principle stating that every user, process, or system component should be granted only the minimum level of access rights and permissions necessary to perform its legitimate,

authorized functions, and no more. The principle of least privilege reduces the potential impact of security incidents by ensuring that a compromised account, process, or component cannot be used to access resources or perform actions beyond those strictly required for the compromised entity's legitimate purpose. In the context of user access control in web applications, the principle of least privilege is implemented through role-based access control systems that assign each user role only the permissions required for the responsibilities associated with that role, rather than granting all users full access to all system functions by default. In this study, the principle of least privilege is implemented through the two-tier RBAC architecture that grants Customer-role users access only to the customer portal functions relevant to their own orders and account, while restricting all administrative, reporting, and other customers' data access to the Administrator role.

Defense in Depth. A cybersecurity strategy and architectural principle that layers multiple independent security controls across different levels of a system's architecture, so that the failure or bypass of any single control does not expose the system to complete compromise. The defense-in-depth approach recognizes that no single security control is infallible and that adversaries will actively seek to identify and exploit weaknesses in individual controls; by requiring an attacker to overcome multiple independent barriers at successive levels, the approach increases the effort and skill required for a successful attack beyond what most attackers are willing or able to invest. In web application security, defense in depth is implemented by combining network-level controls (HTTPS, firewall rules), application-level controls (authentication, input validation, output encoding, CSRF protection), data-level controls (parameterized queries, access control on database objects),

and monitoring controls (audit logging, anomaly detection). In this study, the system's cybersecurity architecture implements a defense-in-depth model through the combination of all security control categories described in the Cybersecurity Module scope.

# REFERENCES

[1] Republic Act No. 11223, "Universal Health Care Act of 2019," Republic of the Philippines, 2019.

[2] Philippine Statistics Authority, 2021 MSME Survey. Quezon City, Philippines: PSA, 2021. [Online]. Available: https://psa.gov.ph

[3] Republic Act No. 10173, "Data Privacy Act of 2012," Republic of the Philippines, 2012.

[4] Department of Information and Communications Technology, Philippine Digital Transformation Strategy 2022–2025. Philippines: DICT, 2022.

[5] R. M. Aldaba, "Small and medium enterprises' (SMEs) access to finance: Philippines," Philippine Institute for Development Studies (PIDS), Makati City, Philippines, Discussion Paper Series no. 2012-05, 2012.

[6] M. Bada and J. R. C. Nurse, "Cyber security awareness campaigns: Why do they fail to change behaviour?" arXiv preprint arXiv:1901.02672, 2019.

[7] Republic Act No. 10175, "Cybercrime Prevention Act of 2012," Republic of the Philippines, 2012.

[8] Department of Information and Communications Technology, National Cybersecurity Plan 2023–2028. Philippines: DICT, 2023.

# CHAPTER II

## REVIEW OF RELATED LITERATURE AND STUDIES

This chapter reviews related foreign studies, local studies, and literature relevant to the development of the CareMed Cybersecurity-Integrated Digital Infrastructure Model. It covers nine thematic areas: (1) SME cybersecurity challenges; (2) web-based inventory and business management systems; (3) digital transformation in the Philippines; (4) medical equipment supply and rental management; (5) role-based access control and authentication; (6) password hashing and credential protection; (7) software development methodologies; (8) software quality evaluation; and (9) the Philippine legal and regulatory landscape. Each source was selected for its direct relevance to at least one core dimension of the CareMed project and was drawn from peer-reviewed journals, conference proceedings, government and institutional publications, or authoritative technical standards.

The review is organized into four sections. Section 2.1 presents foreign studies, Section 2.2 presents local studies, and Section 2.3 reviews theoretical works, standards, legislation, and technical frameworks. Section 2.4 synthesizes the findings and states the research gap that this study addressed.

### 2.1 Foreign Studies

This section presents foreign studies on cybersecurity vulnerabilities in small enterprises, web-based inventory management, digitalization of healthcare supply, and security-integrated software development. The studies span North America, Europe, Australia, and Southeast Asia, which shows that the challenges CareMed faced are a globally recognized

pattern of SME vulnerability to operational inefficiency and cybersecurity risk.

### *2.1.1 Cybersecurity of Small-to-Medium Businesses:*

Chidukwani et al. [9], in "A Survey on the Cybersecurity of Small-to-Medium Businesses: Challenges, Research Focus and Recommendations," published in IEEE Access, conducted a PRISMA-guided structured review of peer-reviewed studies on how SMEs approach cybersecurity. They found that small businesses are disproportionately vulnerable because they lack dedicated security personnel, formal policies, and technical controls. Attackers target them because they offer low resistance relative to the value of the data they hold, and many owners underestimate the likelihood of being targeted, which leads to chronic underinvestment in inexpensive basic protections.

The study found that small businesses frequently fail to implement even basic access controls such as role-based permissions and multi-factor authentication, so one compromised account can expose all system data. Where controls existed, they were often partial, such as requiring passwords but not restricting what users could access afterward, which created a false sense of security. Phishing accounted for 74% of successful breaches in the reviewed literature, and the authors recommended recurring awareness training and the NIST Cybersecurity Framework as a scalable baseline.

These findings were directly relevant to CareMed, which had no formal authentication or access control, and they justify treating cybersecurity as a core project requirement rather than a post-development enhancement.

### *2.1.2 Systematic Review of SME Cybersecurity Challenges:*

Rombaldo Junior et al. [10] systematically reviewed 77 peer-reviewed studies published between 2017 and 2023, filtered from more than 4,200 candidate publications. Three challenges

appeared consistently: low risk awareness among owners and managers, limited staff cybersecurity literacy, and constrained budgets. These factors reinforce one another in what the authors called a "vicious circle" of underinvestment and deepening exposure, which was not self-correcting. SMEs improved their security posture mainly after a security incident or through guidance from an external advisor such as a consultant, researcher, or government agency. The CareMed research represents the second pathway.

The review also found that access controls, staff training, and activity monitoring were the most consistently validated interventions. Organizations using role-based access control, even in a simplified two-role form, reported less internal data misuse, especially from insider threats. Audit trail logging was absent in most reviewed SME systems despite being simple to implement. These findings support CareMed's role-based access control with Administrator and Customer permission sets, hashed credential storage, session timeout, and audit logging.

#### *2.1.3 Agile Development of Secure Software for SMEs:*

Mihelič et al. [11], in "Agile Development of Secure Software for Small and Medium-Sized Enterprises," published in Sustainability, combined a literature review with expert evaluation by developers and security professionals. They identified a tension: threat modeling, security testing, and code review slow Agile iteration, yet omitting them leaves vulnerabilities to be found late, when remediation is costlier, or after deployment.

The study proposed a curated set of "security elements" (roles, activities, and artifacts) that SMEs can integrate selectively. The highest-rated were input validation, authentication reviews, access control, and activity logging, all of which are in CareMed's development plan. The authors

also introduced the "security backlog," a set of security user stories prioritized alongside functional requirements in every sprint. This informs CareMed's approach of documenting security requirements with functional ones and addressing them in every iteration.

#### *2.1.4 Efficacy, Reliability, and Accuracy of Inventory Systems in SMEs:*

Mariani and Wacas [12] surveyed SME owners and operators in Tabuk City, Philippines, who had moved from manual to digital inventory and point-of-sale systems within the previous three years. Using weighted mean, T-test, and F-test analysis, they compared satisfaction on efficacy, reliability, and accuracy. Respondents were generally satisfied, with mean scores of 3.0 for efficacy, 3.05 for reliability, and 3.21 for accuracy, which indicates that the largest perceived gain was in record correctness, consistent with the elimination of transcription errors.

This matters for CareMed, whose staff rely on handwritten logbooks for sales and rental records. The study showed that even modest digital systems improve accuracy without significant capital expenditure, which supports a lightweight, purpose-built system over a commercial enterprise platform. Its Likert-scale instrument also parallels CareMed's ISO 25010-based evaluation (functional suitability, reliability, and performance efficiency), which allows comparison with the SME literature.

#### *2.1.5 Design of a Web-Based Inventory Management System for SMEs:*

Otobo and Alegbe [13] studied cloud-based web inventory systems across SME implementations using quantitative performance metrics and operator interviews. Businesses using cloud-based enterprise resource planning achieved 40% better inventory accuracy, a 25% reduction in time spent on inventory

tasks, a 31% reduction in stockouts, and a 19% reduction in overstock holding costs, mainly through the elimination of manual transcription and real-time stock updates.

The authors identified scalability, reliability under concurrent access, and fit with existing workflows as key requirements. Systems with fewer features but higher usability and reliability outperformed complex ones in adoption. Structured training also mattered: systems without it showed 34% abandonment within six months, versus 8% with structured onboarding. These findings inform CareMed's clean, role-appropriate interface, user documentation, phased deployment, and user acceptance testing.

#### *2.1.6 Development of a Web-Based Inventory Management System for Small Businesses:*

A study in the International Journal of Industrial and Production Engineering [14] developed a MySQL-based web inventory system for small businesses, combining demand forecasting with real-time tracking and refining the interface through iterative prototype reviews with business operators. It showed that manual systems suffer from stocking errors (incorrect quantities) and timing errors (late reordering). Real-time updates addressed the first, and automated low-stock alerts addressed the second, and the combined effect exceeded either alone.

Users with little digital experience reported high satisfaction, which indicates that web-based inventory systems do not require advanced technical literacy if the interface is simple and task-specific. The researchers recommended a focused initial release covering core workflows, because comprehensive systems overwhelm non-technical users during early adoption.

### *2.1.7 Digitalization of the Healthcare Supply Chain:*

A review in the Journal of Cleaner Production [15] analyzed 89 papers (2015 to 2023) on healthcare supply chain digitalization, from large hospital networks to small community supply operations. It found that manual record-keeping carries especially high risk in healthcare because errors can affect patient health, unlike retail or manufacturing, where they mean a lost sale. Digitalizing medical supply inventory reduced stockouts by 28% to 45%.

The main barriers, particularly in developing countries, were interoperability, limited IT expertise, and staff resistance to change. Phased, training-supported implementation and staff involvement in system design helped overcome them, which informs CareMed's deployment strategy. The review also noted that digitalization benefits patients who depend on timely equipment and reduces waste from supply chain inefficiencies.

### *2.1.8 Global Medical Equipment Rental Market Trends:*

Allied Market Research [16] reported that the global medical equipment rental market was valued at about $56.0 billion in 2022 and is projected to reach $94.5 billion by 2032 (5.4% CAGR), with Asia-Pacific growing fastest (6.3%) because of large patient populations, expanding home care, and digital platforms. Durable medical equipment, which includes CareMed's wheelchairs, hospital beds, crutches, and oxygen concentrators, held 42.8% of the market in 2023, driven by aging populations, chronic disease, and home-based care, trends also evident in the Philippines.

The report found that operators with online ordering, digital inventory, and customer tracking achieved higher retention, wider reach, and greater efficiency than those using manual processes. CareMed's adoption of a digital system

is therefore both an operational improvement and a strategic competitive investment.

### *2.1.9 Comparative Analysis of Agile and Waterfall Development Methodologies:*

Dima and Maassen [17], in the International Journal of System Assurance Engineering and Management, compared Agile and Waterfall methodologies using published case studies and a survey of 231 professional developers. Agile outperformed Waterfall in dynamic environments, with roughly 40% higher completion rates and a failure rate of about 10% versus 30%. For small projects with clear requirements the gap narrowed, but Agile still led in user satisfaction because of early feedback.

Agile worked best where clients are closely involved and can give regular feedback, which is natural in SMEs where the owner is directly accessible and invested in the outcome. This matches the CareMed context of a small enterprise, a small development team, and the need for iterative refinement based on management and staff feedback, which makes Agile the appropriate methodology.

### *2.1.10 Security Assessment in Web-Based Rental Management Systems:*

Mpamugo and Ansa [18], in the Journal of Information Systems and Informatics, evaluated 42 healthcare, finance, and logistics applications using static code analysis and penetration testing to identify RBAC vulnerabilities. Broken access control, OWASP's top-ranked risk, typically arose from incomplete role-permission mapping. In 67% of the applications, penetration testing found at least one privilege escalation, usually caused by inconsistent permission checks across endpoints rather than flawed role definitions, which shows the need to test every protected endpoint.

Audit logging was underused but effective: applications with comprehensive logging detected incidents 7 days faster and resolved them 60% faster. The recommended practices (least privilege, endpoint-level permission validation, server-side enforcement, and comprehensive audit logging) were all implemented in CareMed's security architecture, which grounds the design in empirically validated practice.

#### *2.1.11 User Acceptance Testing in Management Information Systems:*

Afolabi et al. [19] analyzed user acceptance testing (UAT) across 18 small enterprise management system implementations. They described UAT as the most critical evaluation phase because only it involves real users performing real tasks in the operational context. Unlike unit or system testing, it reveals usability problems and workflow mismatches that developers cannot anticipate; end-user UAT identified 2.8 times as many usability issues per hour as developer testing, especially for non-technical users.

The authors recommended a protocol of scenario-based task completion (for example, recording a rental or generating an inventory report), think-aloud observation, and a post-task questionnaire aligned with ISO 25010. Together these provide behavioral, cognitive, and perceived-quality evidence. The protocol informs CareMed's UAT design and supports its choice of ISO 25010, because the testing and final assessment criteria are consistent.

#### *2.1.12 SQL Injection and Input Validation Security in Web Applications:*

SQL injection occurs when crafted input is interpreted by the database as SQL commands, allowing unauthorized retrieval, modification, or deletion of records. Though among the oldest known vulnerabilities, it persists in the OWASP Top 10, particularly in SME software built by informally trained

developers without code review. Research in IEEE Access [20] found that 38% of sampled applications contained at least one SQL injection vulnerability, 81% of which came from string concatenation of user input rather than parameterized queries, with higher risk in PHP than in Python or Java.

Effective prevention requires parameterized queries or ORM usage at the data access layer plus input validation at the application layer, so that the failure of one control does not result in a successful attack. CareMed implemented both, using the SQLAlchemy ORM for all database access and Flask-WTF validation for all submitted forms.

#### *2.1.13 Session Management and Authentication Best Practices:*

A user session is the server-side record of an authenticated user's identity, maintained across stateless HTTP requests through cryptographically random session tokens in browser cookies. A compromised session gives an attacker the user's full access rights without knowing the credentials. Common vulnerabilities include session hijacking (interception over unencrypted connections), session fixation (forcing a victim to use a known token), and failure to invalidate sessions after logout or inactivity. NIST SP 800-63B [21] requires tokens generated with a cryptographically secure random number generator (at least 64 bits of entropy), invalidated upon logout, expired after inactivity, and transmitted only over HTTPS.

CareMed uses Flask's session framework with hardened cookies aligned with NIST SP 800-63B [21]: SESSION_COOKIE_SECURE limits cookies to HTTPS, SESSION_COOKIE_HTTPONLY blocks JavaScript access, SESSION_COOKIE_SAMESITE prevents cross-site inclusion, and PERMANENT_SESSION_LIFETIME expires sessions after 30 minutes

of inactivity. Each setting addresses a distinct documented attack vector.

#### *2.1.14 Cross-Site Scripting (XSS) and CSRF Attack Prevention:*

Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) are complementary vulnerabilities that matter for multi-role business systems. XSS injects malicious JavaScript into output that is later executed in other users' browsers; CSRF exploits the browser's automatic inclusion of authentication cookies to submit unauthorized requests on a user's behalf. In a management system, stored XSS could be planted in a customer name or transaction note and rendered on an administrator's dashboard. Gupta and Gupta [22] found that stored XSS causes 3 to 7 times more damage than reflected XSS, and the impact is greater when the victims are privileged administrators.

Flask's Jinja2 engine automatically HTML-escapes template variables, which prevents stored and reflected XSS, and Flask-WTF's CSRF tokens require each state-changing submission to include a random, session-specific token. Together they address both major client-side vectors in the OWASP Top 10.

## 2.2 Local Studies

This section reviews Philippine studies and reports on the conditions, challenges, and precedents facing local small enterprises as they adopt digital systems and address cybersecurity. They span several regions, which matters because CareMed operates in a regional city whose infrastructure and culture may differ from those of Metro Manila.

### *2.2.1 Web-Based Inventory Management System for a Philippine Small Enterprise:*

Tanaman et al. [23] developed and evaluated a web-based inventory system for a small multi-branch business in Pagadian City, Zamboanga del Sur, which had relied on paper records across four branches and a mobile store. The setting, a second-tier Mindanao city, resembles CareMed's in IT infrastructure, staffing, and market. Following a developmental research design, the needs analysis found the same problems CareMed faces: stock levels could not be determined without physical counts, records and physical stock diverged, items were hard to locate across branches, and reports required manual compilation.

The system used a single centralized database for real-time updates from all branches. Users rated functionality 4.44 out of 5.0 and efficiency 4.34, and staff reported that routine tasks fell from hours to minutes. With its ISO-based evaluation, Philippine setting, and small enterprise context, this study is a close precedent and a benchmark for CareMed's own evaluation results.

### *2.2.2 Cybersecurity Landscape in the Philippines:*

The Asia Foundation [24] assessed the cybersecurity environment for Filipino businesses in "Cybersecurity in the Philippines: Global Context and Local Challenges," drawing on government agencies, vendors, and business surveys. It found that Philippine SMEs are among the most vulnerable because they quickly adopt social media messaging, online payments, and cloud storage without matching security measures, producing operations that are nominally digital but structurally unsecured, a "digital gap" that widens as adoption accelerates. Trend Micro data [25] cited in the report ranked the Philippines fourth in Southeast Asia for exposure to Business Email Compromise, and many small

businesses store customer data in Messenger threads, unencrypted spreadsheets, and personal email.

These practices create compliance risk under the Data Privacy Act of 2012 (Republic Act No. 10173), which requires organizational, physical, and technical safeguards for personal information. Awareness of the Act was low, and many operators did not realize their practices were violations. This describes the risk environment in which CareMed operates and underscores the need for a cybersecurity-integrated approach.

### *2.2.3 Digital Transformation in Philippine Organizations:*

A study in the Proceedings of the 2024 International Conference on Computers in Management and Business [26] surveyed and interviewed owners and managers of Philippine organizations in retail, services, logistics, healthcare, and food across multiple regions. It identified five drivers of digital transformation: competitive pressure, customer expectations, government initiatives, benefits seen in peer organizations, and falling costs of digital and cloud tools. Barriers included limited capital, lack of internal IT expertise, staff resistance, and difficulty choosing among overwhelming options, which often led to decision paralysis.

Organizations that succeeded used phased, step-by-step adoption with staff training and clear communication of benefits, and improved their efficiency, data quality, and responsiveness. CareMed's purpose-built system, structured training, and phased deployment align with these success factors.

### *2.2.4 IoT Adoption and Supply Chain Management in Philippine SMEs:*

A study in the International Journal of Research and Innovation in Social Science [27] compared IoT adoption for supply chain management among Philippine SMEs and those in

other Asian countries. Philippine SMEs account for 99% of business establishments and 40% of GDP, so small efficiency gains have large national effects. They face distinct obstacles compared with Singapore, Japan, and South Korea, including limited internet infrastructure in regional areas, limited capital, low familiarity with technology licensing and support contracts, and insufficient government incentives. Provincial reliance on mobile internet of variable reliability also constrains the design of web-based systems.

SMEs using even basic digital supply chain tools reported better order accuracy, stock availability, and customer satisfaction. The study recommended starting with web-based inventory and customer management before IoT capabilities, which aligns with CareMed's foundational infrastructure that can later support barcode or QR code tracking.

#### *2.2.5 User Requirements for Inventory Systems in Philippine Micro and Small Enterprises:*

A study at the 2024 World Symposium on Software Engineering [28] used Design Thinking to elicit requirements from micro and small construction supply enterprises in the Philippines. Although sector-specific, its findings apply to other small enterprises, including medical supply. Operators prioritized accurate stock tracking, ease of use for non-technical staff, and reliability under inconsistent connectivity, which map to three of CareMed's five ISO 25010 characteristics: functional suitability, interaction capability, and reliability.

Drawing on Philippine Statistics Authority data [29], the study noted that micro and small enterprises employ mostly non-technical workers, and systems requiring more than basic smartphone and internet skills had much lower adoption. A usability audit showed that most commercial inventory products target technically literate users, which supports developing a purpose-built system.

### *2.2.6 National Privacy Commission Enforcement and Compliance Guidance:*

The National Privacy Commission (NPC), the regulator under Republic Act No. 10173, has issued circulars, advisory opinions, and enforcement decisions (2018 to 2024) clarifying the technical measures it expects. NPC Circular 16-01 [30] requires authentication-based access control, activity logging, incident detection and response, and secure disposal of personal data. Advisory opinions confirm that password hashing, role-based access control, and audit logging satisfy the "controlled access" and "systematic monitoring" requirements.

Controllers must notify the NPC and affected individuals within 72 hours of a breach likely to cause serious harm, and organizations without adequate audit logs struggled to meet this deadline because they could not establish its scope and timeline. CareMed's audit trail, with timestamped records of access and data changes, supports rapid breach assessment and notification.

### *2.2.7 Digital Literacy and Technology Adoption Among Filipino SME Staff:*

A study of Filipino small enterprise operators and staff in Visayas and Mindanao [31] found most respondents at an intermediate digital literacy level, comfortable with smartphone apps and social media but unfamiliar with business management software. The regional focus is relevant because CareMed operates in a Visayan city. Digital literacy was the strongest predictor of adoption and sustained use, while lower-literacy users cited fear of errors, hard-to-read documentation, and lack of training. Even minor frustrations, such as confusing error messages, caused a significant regression in attitudes, so the initial experience is critical.

For CareMed, this means familiar interface patterns, plain labels, contextual help, and error messages that guide correction, together with face-to-face training for all roles and role-specific quick-reference guides that use plain language, concrete workflow examples, and hands-on practice.

### *2.2.8 Home-Based Care and Medical Equipment Supply in the Philippine Context:*

Home-based care in the Philippines has expanded since the COVID-19 pandemic because it reduces facility overcrowding and is preferred by patients and families. The Department of Health (DOH) [32] has issued guidelines and programs formalizing it to serve an aging population and growing chronic disease. Equipment suppliers like CareMed bridge hospital discharge and home recovery, and their wheelchairs, hospital beds, crutches, oxygen concentrators, and nebulizers are the categories DOH guidelines identify as essential.

Because delayed or incorrect delivery can impair rehabilitation or create safety risks, inventory and rental tracking failures at CareMed have consequences different from those in retail. This reinforces the urgency of both digital management and cybersecurity integration. No reviewed Philippine study addressed this intersection for medical equipment supply, which underscores the originality of this research.

## 2.3 Related Literature

This section reviews theoretical works, standards, legislation, and technical documentation. While Sections 2.1 and 2.2 supplied empirical evidence, these sources provide the theoretical justification and standardized benchmarks against which CareMed's design decisions were evaluated.

### *2.3.1 Management Information Systems:*

Laudon and Laudon [33], in Management Information Systems: Managing the Digital Firm (16th ed.), define information systems as integrated sets of hardware, software, data, people, and processes that collect, store, process, and communicate data to support decision-making, coordination, and control. They distinguish Transaction Processing Systems (TPS) for routine operations, Management Information Systems (MIS) that aggregate data into reports for middle management, and Decision Support Systems (DSS) for unstructured problems. CareMed is primarily a TPS with MIS-level reporting.

They argue that digital systems reshape workflows and reveal insights, such as rental patterns, seasonal demand, and equipment utilization, that are invisible in paper records, so CareMed was designed to enable new business intelligence rather than only replicate manual workflows. They also hold that security must be a fundamental design element, defined by confidentiality, integrity, and availability. CareMed preserves confidentiality through RBAC and authentication, integrity through input validation and audit logging, and availability through reliable hosting and error handling.

### *2.3.2 Network Security Essentials:*

Stallings [34], in Network Security Essentials: Applications and Standards (6th ed.), argues that no single control protects against all threats, so authentication, access control, data integrity controls, and activity monitoring must operate together. This "defense in depth" principle guided CareMed's multi-control architecture. He also distinguishes security mechanisms (password hashing, access control lists, session timeouts) from security services (authentication, authorization, confidentiality, non-repudiation), noting that different mechanisms can contribute to the same service: hashed storage protects against credential compromise through

database breaches, while session management protects against interception or fixation.

His treatment of hash functions (one-way computation, collision resistance, and computational intensity) underpins CareMed's use of bcrypt, his access control models ground the RBAC design, and his coverage of session management informs the timeout and secure cookie features. His systematic treatment of web threats helped ensure that the architecture was complete.

### *2.3.3 Philippine Data Privacy Act of 2012 (Republic Act No. 10173):*

The Philippine Data Privacy Act of 2012 (Republic Act No. 10173) governs the collection, processing, storage, and protection of personal information. It applies to all public and private entities regardless of size, so CareMed is fully subject to it. Section 20 requires reasonable and appropriate organizational, physical, and technical measures, proportionate to the data and its risks, against accidental loss or destruction and against unlawful access, misuse, alteration, and disclosure.

Failure to implement them can bring administrative penalties, civil liability, and criminal prosecution under Sections 25 to 32, including imprisonment of up to six years and fines of up to Five Million Pesos, potentially for individual officers as well, and NPC enforcement summaries show that enforcement is active. CareMed's hashed credentials, RBAC, and audit logging were designed to meet NPC requirements as a baseline, and the system offers a template for the organizational and physical measures the Act also requires, such as a privacy policy, staff training, and a breach response plan.

### *2.3.4 OWASP Top 10 Web Application Security Risks:*

The Open Web Application Security Project (OWASP) is an international nonprofit that improves software security through community-led projects. Its Top 10 is a consensus-based list of the most critical web application risks, drawn from vulnerability data contributed by security organizations, pen-testing firms, and researchers. The 2021 edition ranks Broken Access Control first (up from fifth in 2017), followed by Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable and Outdated Components, Identification and Authentication Failures, Software and Data Integrity Failures, Security Logging and Monitoring Failures, and Server-Side Request Forgery. The rise of access control reflects growth in web development by teams without formal security training.

CareMed addresses these risks directly: strict role-permission separation with server-side checks at every protected endpoint prevents broken access control; bcrypt hashing and HTTPS address cryptographic failures; login rate limiting, session timeout, and secure cookies address authentication failures; and audit trail logging addresses logging failures. Its security testing includes cases for each relevant Top 10 risk, which gives an internationally recognized benchmark.

### *2.3.5 NIST Cybersecurity Framework 2.0:*

The NIST Cybersecurity Framework (CSF) [35] is a voluntary set of guidelines for managing cybersecurity risk, first published in 2014 and substantially updated to version 2.0 in 2024. CSF 2.0 adds a Govern function, recognizing the need for organizational governance, and organizes activities into six functions: Govern, Identify, Protect, Detect, Respond, and Recover. It is technology-neutral, describing outcomes rather than tools.

Its Small Business Quick-Start Guide recommends inventorying digital assets, implementing basic access control and authentication, and establishing activity logging first, which is the same sequence implemented in CareMed. It also affirms that small businesses without prior programs can and should implement foundational controls, which validates CareMed's approach.

### *2.3.6 Role-Based Access Control (RBAC) Theory and Implementation:*

Role-Based Access Control (RBAC) was formalized by Ferraiolo and Kuhn [36] in a NIST publication and elaborated as the RBAC96 model by Sandhu et al. [37], which became the basis of the ANSI RBAC standard. It is the dominant access control paradigm because it is administratively simple, effective, and transparent. Permissions are assigned to roles rather than individuals, and users receive the union of the permissions of their roles, so permissions are managed at the role level and adding or removing users requires only role assignment.

In web applications, RBAC associates each authenticated session with a role and verifies permission on the server before any privileged function; client-side measures such as hiding menu items improve usability but are not security controls. OWASP [38] documents broken access control as the most prevalent vulnerability, found in 94% of tested applications. CareMed implements server-side verification at every protected endpoint and a two-role model: Administrator (inventory, transactions, user accounts, reports, and audit logs) and Customer (product catalog and order submission only), so customer credentials cannot reach business operations data even if compromised.

### *2.3.7 Password Hashing and Credential Security:*

Passwords must never be stored in retrievable form, a principle mandated by NIST SP 800-63B and the PCI Data

Security Standard, so that an attacker who gains database access cannot recover actual passwords. This requires one-way hashing. Bhagat et al. [39] found bcrypt among the most suitable algorithms for resisting offline dictionary and brute-force attacks. Unlike general-purpose hashes such as SHA-256 or MD5, which are optimized for speed and vulnerable to GPU attacks, bcrypt is deliberately slow and has a configurable work factor that can be raised as hardware improves.

A 2023 USENIX retrospective [40] reported that modern GPUs compute about 184,000 bcrypt hashes per second at work factor 5 versus 6.3 billion SHA-256 hashes per second, roughly a 34,000-fold difference. NIST and industry practice suggest a work factor of 10 to 12 for interactive authentication. CareMed uses bcrypt with a work factor of 12, which makes recovering plaintext passwords from a stolen database economically infeasible for a small enterprise target.

#### *2.3.8 ISO/IEC 25010 Software Product Quality Model:*

ISO/IEC 25010 [41], part of the SQuaRE series, is the international standard for software product quality. Its 2023 edition defines nine characteristics: Functional Suitability, Performance Efficiency, Compatibility, Interaction Capability, Reliability, Security, Maintainability, Flexibility, and Safety, each decomposed into sub-characteristics (for example, Functional Suitability into Functional Completeness, Correctness, and Appropriateness). A standardized model makes evaluation results comparable across systems and studies.

A bibliometric analysis of 1,613 articles [42] showed that publication on the standard peaked in 2023, with the fastest growth in public administration, finance, education, and healthcare, and confirmed it as the most widely adopted quality standard in academic capstone research, including Philippine BSInfoTech and BSCS programs. CareMed's instrument

covers five dimensions: Functional Suitability, Performance Efficiency, Interaction Capability (Usability), Reliability, and Security, which enables comparison with studies such as Tanaman et al. [23] and Mariani and Wacas [12].

### *2.3.9 Python Flask Framework for Web Application Development:*

Flask is a lightweight Python micro web framework developed by Armin Ronacher [43], maintained by Pallets Projects, and first released in 2010. The Python Developers Survey ranked it the most popular Python web framework for five consecutive years (2018 to 2022). Unlike full-stack frameworks such as Django, it provides a minimal core extended with chosen libraries, so CareMed includes only the needed security controls, database integration, and routing, and developers can verify that authentication, session management, and access control are configured explicitly.

Flask also supports relevant security mechanisms: Werkzeug for secure sessions and password hashing, Jinja2 for automatic HTML escaping against XSS, Flask-Login for login-required decorators on protected routes, and Flask-WTF for CSRF protection. Together these address several OWASP Top 10 categories.

### *2.3.10 MySQL Relational Database Management System:*

MySQL [44] is an open-source relational database management system, originally developed by MySQL AB and now maintained by Oracle, that uses SQL for data definition, manipulation, and retrieval. Its adoption, documentation, tooling, reliability, and lack of licensing cost make it a natural choice for a small enterprise, and it is widely taught in Philippine IT programs. Its security features include InnoDB table-level encryption for sensitive data at rest and a privilege system that allows CareMed's application account to be limited to SELECT, INSERT, UPDATE, and DELETE on specific tables, which

prevents schema modification even if its credentials are compromised and applies least privilege at the infrastructure layer.

SQLAlchemy, a Python SQL toolkit and Object-Relational Mapping (ORM) library, is CareMed's primary defense against SQL injection. It parameterizes all queries so that user input is always treated as data, and enforcing this at the data access layer avoids the inconsistent manual parameterization that commonly causes vulnerabilities.

#### *2.3.11 Client-Server Architecture for Web Applications:*

In the client-server model, the browser renders the interface and transmits input, while the server executes business logic, queries the database, and returns results. This separation is also a security requirement, since any client-side check can be bypassed, so security-critical logic must run on the server. Tanenbaum and Van Steen [45], in Distributed Systems: Principles and Paradigms, identify three properties valuable for business systems: centralized data storage, server-side access control enforcement, and platform independence.

For CareMed, the Administrator and Customer interfaces run in any browser on a desktop, laptop, tablet, or smartphone without installation, which reduces maintenance overhead and allows operations to continue on other devices if one fails. The mobile-responsive design accommodates the mobile-first access patterns documented in Section 2.2.

#### *2.3.12 Database Design and Normalization for Transaction Processing Systems:*

E.F. Codd's 1970 paper "A Relational Model of Data for Large Shared Data Banks" [46] founded the relational model underlying MySQL and other major databases, organizing data into related tables governed by integrity constraints and grounded in relational algebra and set theory. Normalization applies progressively stricter normal forms, and Third Normal

Form (3NF) is the standard target for transaction systems because it removes transitive dependencies that cause update anomalies (inconsistent customer records) and deletion anomalies (losing product information when the last referencing transaction is deleted).

CareMed's schema is in 3NF, with separate tables for products, customers, sales, rentals, users, and audit logs. MySQL FOREIGN KEY constraints enforce referential integrity at the database level, which prevents orphaned records such as a rental referencing a nonexistent customer or product and supports the data quality goals behind the move to digital management.

### *2.3.13 Responsive Web Design for Mobile-First Business Applications:*

Responsive web design (RWD) lets a single interface adapt its layout to the rendering device. It was articulated by Ethan Marcotte in a 2010 A List Apart article [47] and is now dominant as mobile usage has surpassed desktop. In the Philippines, where smartphone adoption preceded desktop adoption, Philippine Statistics Authority data [29] indicate that household smartphone ownership far exceeds laptop and desktop ownership, especially in provincial regions. CareMed's administrative dashboard therefore needs full functionality on smartphone screens, or it would be inaccessible to staff who rely on phones.

CareMed uses Bootstrap 5, whose 12-column fluid grid and responsive utility classes let one codebase serve smartphone and desktop users. The interface was designed and tested from 320 to 1920 pixels.

### *2.3.14 Audit Trail Logging and Non-Repudiation in Information Systems:*

Audit trail logging, the tamper-resistant recording of who performed which action on what data and when, is a

foundational control in both the security literature and regulation. Logs enable detection of unauthorized access after the fact, support incident investigation and compliance, and deter misconduct, which is relevant in small enterprises where close relationships may lower the perceived risk. NIST SP 800-92 [48] specifies that logs capture the user, action type, data object, timestamp, and outcome, and that they be protected from modification or deletion by ordinary users, including administrators (log integrity).

The Data Privacy Act's monitoring requirement and the NPC's 72-hour breach notification both presuppose adequate logging. CareMed uses a dedicated MySQL audit table populated by logging middleware that records logins, inventory changes, customer record access, transactions, and administrative operations. It is read-only for Administrators and cannot be modified by any application operation, which meets the log integrity requirement of NIST SP 800-92 [48].

#### *2.3.15 Inventory Economic Order Quantity and Stock Management Theory:*

The classical Economic Order Quantity (EOQ) model frames the tradeoff in inventory management: more stock reduces ordering costs and stockout risk but raises holding costs, and the reverse. Balancing them requires accurate, real-time visibility that manual systems cannot provide because of the lag between stock movements and record updates. Munyaka and Yadavalli [49], in a systematic review in the South African Journal of Industrial Engineering, found inventory management critical for SMEs, and enterprises without formal systems had higher stockout and overstock rates and poorer order fulfillment accuracy than those with real-time digital tracking.

CareMed manages both sale and rental inventory, and rental items require lifecycle tracking from departure to

return, with alerts for overdue items. Its inventory module therefore tracks sales and rental stock separately, generates low-stock alerts, and escalates overdue rental notifications to administrators, which addresses needs that generic inventory concepts do not fully capture.

## 2.4 Synthesis of the Reviewed Literature and Studies

The reviewed foreign studies, local studies, and literature together provide a consistent foundation for the CareMed Cybersecurity-Integrated Digital Infrastructure Model. This synthesis presents five themes, the convergences across contexts and methods, and the research gap addressed.

### *2.4.1 Convergence on the Vulnerability of Manual Small Enterprise Systems:*

From the IEEE Access survey of Chidukwani et al. [9] to the Philippine inventory study of Tanaman et al. [23], the studies agree that small enterprises relying on manual processes are at a measurable disadvantage in efficiency, accuracy, record integrity, and responsiveness compared with those using even basic digital systems. This holds across North America, Europe, Australia, and the Philippines; across retail, construction supply, and healthcare; and across literature reviews, surveys, and system development case studies.

The documented risks (stock discrepancies, slow record retrieval, vulnerability of physical records, and no real-time visibility) carry greater weight for CareMed because patients depend on equipment availability. A stockout caused by inaccurate records is a failure to meet a medically necessary need. The literature therefore supports the premise that moving from manual to digital operations is necessary for CareMed, and it predicts improvements in accuracy, retrieval, reporting, and performance that the evaluation in Chapter 5 will confirm or disconfirm.

### *2.4.2 Convergence on the Security Gap in SME Digital Adoption:*

Second, moving to digital operations without deliberate cybersecurity integration transforms risk rather than reducing it. The Asia Foundation [24] found that Filipino small businesses using digital tools without security controls were more exposed than manual ones, since digital data can be accessed remotely and at scale through a single compromised credential; customer data in Messenger threads is more vulnerable than data in a locked notebook. Rombaldo Junior et al. [10] describe the same pattern globally as a vicious circle of growing dependence and vulnerability. OWASP and NIST define the standards for effective controls, and Mihelič et al. [11] show how to integrate them into Agile development without disproportionate cost.

This informs CareMed's defining characteristic: authentication, access control, session management, input validation, credential hashing, and activity logging are built into the architecture from the earliest design phase, which avoids the "digitalize first, secure later" trap that the literature identifies as the main driver of SME security failure.

### *2.4.3 The Philippine Regulatory Imperative:*

The Data Privacy Act adds a regulatory dimension. By collecting names, contact details, addresses, and transaction histories, CareMed is a personal information controller with explicit legal obligations. The Asia Foundation [24] found many Filipino SMEs unaware of these obligations, a finding reinforced by NPC enforcement cases against organizations that did not realize the Act applied to them.

CareMed's architecture uses the Act as an explicit compliance baseline, and the NPC's guidance on password hashing, RBAC, and audit logging matches its implemented

controls, which grounds the design in both international practice (NIST, OWASP) and Philippine regulation. Legal compliance sets a mandatory floor for security investment, which CareMed was designed to meet and exceed.

### *2.4.4 Technical Framework Convergence:*

Stallings [34], the OWASP Top 10 [38], and NIST CSF 2.0 [35] converge on a set of foundational controls for small-enterprise web systems: role-based access control, strong password hashing, session management, input validation, audit trail logging, and HTTPS-enforced communication. These form CareMed's security architecture and can be verified against documented benchmarks. ISO/IEC 25010 likewise supplies a recognized framework for evaluating overall quality, and the local studies' use of ISO-based instruments allows direct comparison with Philippine benchmarks.

The technology stack of Python Flask, MySQL, SQLAlchemy, and Bootstrap 5 is also supported by the literature: Flask's security defaults, SQLAlchemy's automatic parameterization, and Bootstrap's responsive grid each meet requirements identified in the reviewed studies, and each was chosen for its contribution to security, usability, and operational fit rather than developer preference.

### *2.4.5 The Research Gap:*

Despite the breadth of the literature, no documented Philippine study describes a cybersecurity-integrated digital management system for a small medical equipment supply and rental enterprise. Existing Philippine work either addressed inventory systems for non-healthcare SMEs [23] or cybersecurity at a macro level without implementation [24]. None combined the operational requirements of this sector (dual-state inventory, rental lifecycle management with overdue tracking, and patient-centered customer management) with purpose-built cybersecurity integration, and none

addressed Data Privacy Act compliance for medical supply operations, where patient-related data creates heightened obligations and consequences.

This gap defines the original contribution of the study: designing, developing, and evaluating a system that digitalizes operations and embeds cybersecurity controls suited to the Philippine regulatory and threat environment. The system and its documented methodology provide a replicable model for similar healthcare supply enterprises, treating security as part of the core system rather than an optional enhancement.

In summary, the reviewed literature validates the need for a system that integrates operational efficiency and cybersecurity protection in one platform and provides methodological precedents, theoretical frameworks, and empirical benchmarks for its design and evaluation. The CareMed research is firmly grounded in existing knowledge and positioned to make a distinctive contribution.

# REFERENCES

[9] A. Chidukwani, S. Zander, and P. Koutsakis, "A survey on the cybersecurity of small-to-medium businesses: Challenges, research focus and recommendations," IEEE Access, vol. 10, pp. 85701–85719, 2022, doi: 10.1109/ACCESS.2022.3197899.

[10] A. H. Rombaldo Junior, T. Becker, and C. Johnson, "Cybersecurity challenges for small and medium enterprises: A systematic review," arXiv:2309.17186, 2025. [Online]. Available: https://arxiv.org/abs/2309.17186

[11] A. Mihelič, S. Vrhovec, and T. Hovelja, "Agile development of secure software for small and medium-sized enterprises," Sustainability, vol. 15, no. 1, Art. no. 801, 2023, doi: 10.3390/su15010801.

[12] J. C. Mariani and F. Y. Wacas, "Efficacy, reliability, and accuracy of inventory systems in small and medium enterprises in Tabuk City, Kalinga," Int. J. English Lit. Soc. Sci., vol. 7, no. 6, 2022, doi: 10.22161/ijels.76.43.

[13] F. N. Otobo and E. C. Alegbe, "Design of a web-based inventory management system for small and medium enterprises," Int. J. Innovative Inf. Syst. Technol. Res., vol. 13, no. 1, pp. 127–136, 2025.

[14] "Development of a web-based inventory management system for small businesses," Int. J. Ind. Prod. Eng., vol. 2, no. 2, pp. 53–67, 2024.

[15] "Digitalization to achieve greener healthcare supply chain," J. Cleaner Prod., 2024, doi: 10.1016/j.jclepro.2024.142509.

[16] Allied Market Research, Medical Equipment Rental Market: Global Opportunity Analysis and Industry Forecast, 2022–2032. Portland, OR, USA: Allied Market Research, 2023.

[17] A. C. Dima and M. A. Maassen, "Structured software development versus agile software development: A comparative analysis," Int. J. Syst. Assurance Eng. Manag., 2023, doi: 10.1007/s13198-023-01958-5.

[18] E. Mpamugo and G. Ansa, "Enhancing network security in mobile applications with role-based access control," J. Inf. Syst. Informatics, vol. 6, no. 3, pp. 1872–1899, 2024, doi: 10.51519/journalisi.v6i3.863.

[19] F. F. Afolabi, Y. M. Malgwi, M. A. Mahmud, and J. P. Oguntoye, "Development of a web-based platform for automating an inventory management of a small and medium enterprise," Niger. J. Technol., vol. 41, no. 3, 2022.

[20] "DRacv: Detecting and auto-repairing vulnerabilities in role-based access control in web application," J. Netw. Comput. Appl., 2025, doi: 10.1016/j.jnca.2025.103900.

[21] National Institute of Standards and Technology, Digital Identity Guidelines: Authentication and Lifecycle Management (NIST SP 800-63B). Gaithersburg, MD, USA: U.S. Dept. of Commerce, 2020.

[22] S. Gupta and T. Wang, "SQL injection: Trends, challenges, and solutions," Database Security Quarterly, vol. 30, no. 2, pp. 89–104, Apr. 2024.

[23] M. T. Tanaman, J. Baylosis, M. Abiles, J. Catungal, and P. Encarnacion, "Web-based inventory management system for small business enterprise," Int. J. Sci. Appl. Inf. Technol., vol. 12, no. 5, pp. 44–48, 2023, doi: 10.30534/ijsait/2023/021252023.

[24] The Asia Foundation, Cybersecurity in the Philippines: Global Context and Local Challenges. Manila, Philippines: The Asia Foundation, 2022.

[25] Trend Micro, "Leveraging over 30 years of security expertise and technology foresight," Trend Micro Incorporated, Security Threat Landscape Report, H1 2021 Threat Intelligence Tracking, May 2021.

[26] "The impact of digital transformation in the Philippines: A study of the key drivers and barriers," in Proc. 2024 7th Int. Conf. Comput. Manage. Bus., 2024, doi: 10.1145/3647782.3647799.

[27] "Improving supply chain management: A comparative study on Internet of Things adoption in SMEs of Philippines and Asian countries," Int. J. Res. Innovation Soc. Sci., vol. 8, no. 8, 2024.

[28] "Determining user requirements for a tailored web-based inventory and sales management system in micro-small construction supply stores using design thinking," in Proc. 2024 6th World Symp. Softw. Eng., 2024, doi: 10.1145/3698062.3698082.

[29] Philippine Statistics Authority, 2022 ICT Survey / Survey on Information and Communication Technology. Quezon City, Philippines: PSA, 2022.

[30] National Privacy Commission, "NPC Circular 16-01: Security of personal data in government agencies," NPC, Philippines, 2016. [Online]. Available: https://www.privacy.gov.ph

[31] Department of Trade and Industry, "Digital transformation and the Philippine small and medium enterprises," DTI, Philippines, 2023.

[32] Department of Health, "Guidelines on home-based care services," DOH, Philippines, 2022.

[33] K. C. Laudon and J. P. Laudon, Management Information Systems: Managing the Digital Firm, 16th ed. Boston, MA, USA: Pearson, 2020.

[34] W. Stallings, Network Security Essentials: Applications and Standards, 6th ed. Boston, MA, USA: Pearson, 2018.

[35] National Institute of Standards and Technology, The NIST Cybersecurity Framework (CSF) 2.0 (NIST CSWP 29). Gaithersburg, MD, USA: U.S. Dept. of Commerce, 2024.

[36] D. F. Ferraiolo and D. R. Kuhn, "Role-based access control," in Proc. 15th NIST-NCSC Nat. Comput. Security Conf., 1992, pp. 554–563.

[37] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," IEEE Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[38] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[39] S. S. Bhagat et al., "Secure hashing using bcrypt for cryptographic applications," Int. J. Inf. Security, 2023.

[40] "BCrypt at 25: A retrospective on password security," USENIX ;login:, 2023. [Online]. Available: https://www.usenix.org/publications/loginonline/bcrypt-25-retrospective-password-security

[41] ISO/IEC 25010:2023, Systems and Software Quality Requirements and Evaluation (SQuaRE): Product Quality Model. Geneva, Switzerland: ISO/IEC, 2023.

[42] "Bibliometric analysis of ISO/IEC 25010 in software quality evaluation," Eng. Technol. Appl. Sci. Res., 2025.

[43] A. Ronacher, "Flask (web framework)," Pallets Projects, 2010. [Online]. Available: https://flask.palletsprojects.com/

[44] MySQL 8.0 Reference Manual, Oracle Corporation. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[45] A. S. Tanenbaum and M. Van Steen, Distributed Systems: Principles and Paradigms, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2017.

[46] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA, USA: Addison-Wesley, 2019.

[47] E. Marcotte, "Responsive web design," A List Apart, 2010. [Online]. Available: https://alistapart.com/article/responsive-web-design/

[48] National Institute of Standards and Technology, Guide to Computer Security Log Management (NIST SP 800-92). Gaithersburg, MD, USA: U.S. Dept. of Commerce, 2006.

[49] J. B. Munyaka and V. S. S. Yadavalli, "Inventory management concepts and implementations: A systematic review," South Afr. J. Ind. Eng., vol. 33, no. 2, pp. 15–36, 2022.

# CHAPTER III

## METHODOLOGY

### 3.1 Research Design

This study employed a developmental research design approach. Developmental research, as defined by Richey and Klein [50], was a systematic methodology for designing, developing, and evaluating instructional programs, processes, and products that must meet the criteria of internal consistency and effectiveness. This research design was particularly appropriate for this study because it focuses on the creation of a software system — the CareMed Cybersecurity-Integrated Digital Infrastructure Model — designed to addressed specific operational and cybersecurity challenges identified within CareMed Medical Supply Enterprise.

The developmental research approach allowed the researchers to systematically analyze existing manual workflows, design a technology-based solution grounded in established security frameworks, developed the system using modern software engineering practices, and evaluate its effectiveness against standardized software quality criteria. Unlike descriptive research, which merely documents existing conditions, or experimental research, which manipulates variables in controlled settings, developmental research produces a tangible artifact — in this case, a fully functional web-based management system — whose quality and effectiveness can be empirically measured and assessed (Richey & Klein, 2005).

The research design integrates three methodological components that collectively ensured rigor and practical relevance:

- Requirements Analysis: A comprehensive analysis of CareMed's existing manual workflows through structured interviews, direct observation, and document analysis. This component ensured that the developed system addressed genuine operational needed rather than imposing a generic solution.

- System Development: An iterative development process following the Agile methodology, producing a functional system with integrated cybersecurity controls. The development process was documented through version control (Git) and structured sprint planning.

- System Evaluation: A formal evaluation process using the ISO/IEC 25010 software product quality standard, assessing the system across five quality dimensions — functional suitability, performance efficiency, usability, reliability, and security — through structured questionnaires administered to representative users.

The developmental research design was selected over alternative approaches for several reasons. First, it directly addressed the practical needed for an operational system at CareMed, producing a concrete deliverable that can be deployed and used. Second, it accommodates the iterative nature of software development, allowing for continuous refinement based on testing feedback and stakeholder input. Third, it provided a structured evaluation framework that produces measurable evidence of the system's quality and effectiveness, contributing to the academic rigor of the study.

The study was conducted within the ethical guidelines for research involving human participants. All interview participants provided informed consent, and data collected from CareMed was handled confidentially in accordance with the Philippine Data Privacy Act of 2012. The evaluation

instruments were designed to minimize participant burden while maximizing the reliability and validity of collected data.

- functional suitability, performance efficiency, usability, reliability, and security — through structured questionnaires administered to representative users.

The developmental research design was selected over alternative approaches for several reasons. First, it directly addressed the practical needed for an operational system at CareMed, producing a concrete deliverable that can be deployed and used. Second, it accommodates the iterative nature of software development, allowing for continuous refinement based on testing feedback and stakeholder input. Third, it provided a structured evaluation framework that produces measurable evidence of the system's quality and effectiveness, contributing to the academic rigor of the study.

The study was conducted within the ethical guidelines for research involving human participants. All interview participants provided informed consent, and data collected from CareMed was handled confidentially in accordance with the Philippine Data Privacy Act of 2012. The evaluation instruments were designed to minimize participant burden while maximizing the reliability and validity of collected data.

### 3.2 System Development Methodology

The CareMed system was developed using the Agile software development methodology, specifically incorporating Scrum principles for iterative and incremental delivery. The Agile methodology was chosen over traditional approaches such as the Waterfall model because of its emphasis on adaptability, stakeholder collaboration, and the delivery of working software at regular intervals [51]. Given the resource constraints of a capstone project and the needed for continuous feedback from CareMed's management, Agile provided the flexibility necessary to accommodate evolving requirements

while maintaining steady progress toward a complete, functional system.

The Agile Manifesto, formulated by Beck et al. [51], articulates four core values that guided the development process: (1) individuals and interactions over processes and tools; (2) working software over comprehensive documentation; (3) customer collaboration over contract negotiation; and (4) responding to change over following a fixed plan. These values were operationalized through a series of structured development sprints, each producing an increment of working functionality that could be reviewed, tested, and refined before proceeding to the next iteration.

### *3.2.1 Agile Development Phases:*

The development lifecycle was organized into eight distinct phases. Each phase had clearly defined objectives, deliverables, and evaluation criteria. The following table summarizes the development phases:

| Phase | Description | Key Deliverables |
|---|---|---|
| 0 | Planning & Requirements Gathering | Requirements specification, interview documentation |
| 1 | Database Design & Core Setup | ERD, database schema, Flask framework setup |
| 2 | Authentication & User Management | Login, registration, RBAC, email verification |
| 3 | Inventory & Product Catalog | Product CRUD, stock tracking, audit logging |
| 4 | Cart, Order & Payment System | Shopping cart, checkout, GCash/COD payments |
| 5 | Rental Management & Invoicing | Rental agreements, monthly invoices, returns |
| 6 | Security, Reports & Dashboard | IDS, security center, admin dashboard, backups |

| Phase | Description | Key Deliverables |
|---|---|---|
| 7 | Testing & Quality Assurance | Unit, integration, system, and security test reports |

***Phase 0: Planning and Requirements Gathering***

The planning phase established the foundation for the entire project. The research team conducted a faced-to-faced interview with CareMed's owner and staff at their warehouse location to understand the current operational workflows, document existing challenges, and identified specific system requirements. The interview revealed that CareMed had been operating for approximately one year using entirely manual processes — Excel spreadsheets for inventory and rental tracking, Facebook Messenger for customer communication and transaction history, and physical visits for product viewing and order placement.

Key findings from the requirements gathering phase included: the needed for dual-mode sales and rental functionality; the requirement for two valid IDs and proof of billing for rental customers; the 15-day cut-off cycle (5th and 20th of each month) for verifying oxygen tank refills against Prime Gas records; the variable deposit system based on delivery distance; the one-week extension policy for delayed rental returns; and the first-come-first-served delivery system using a single vehicle with tricycle driver backup for oxygen tank deliveries.

These findings were documented in a formal requirements specification that served as the reference document throughout the development process. The specification defined functional requirements (what the system must did), non-functional requirements (how the system should perform), and security requirements (what protections the system must implemented)

based on the OWASP Top 10 framework and the Philippine Data Privacy Act of 2012.

***Phase 1: Database Design and Core Setup***

The database design phase focused on creating a robust, normalized relational database schema that could supported all identified system requirements. The Entity-Relationship Diagram (ERD) was developed using the crow's foot notation, identifying 16 core entities: users, customer, product, cart, cart_item, transaction, purchase, rental, rental_invoice, payment, payment_proof, expenses, inventory_logs, security_logs, blocked_ips, and alembic_version.

Database normalization was applied through Third Normal Form (3NF) to eliminate data redundancy and ensured referential integrity. Primary keys were defined for all tables, and foreign key relationships were established to maintained data consistency across related entities. The MySQL relational database management system was selected for its proven reliability, wide supported in Philippine web hosting environments, and compatibility with the Flask-SQLAlchemy ORM layer.

The core application framework was established during this phase, including the Flask application structure, environment configuration using python-dotenv, database connection pooling with pool_pre_ping enabled, and the Alembic migration system for version-controlled database schema changes. Five Alembic migrations were created throughout the development lifecycle to manage incremental schema changes.

***Phase 2: Authentication and User Management***

The authentication module was developed as the foundational security layer for the entire system. User registration was implemented with email verification through

Flask-Mail using Gmail SMTP, requiring users to verify their email addressed before account activation. The login system incorporates multiple security mechanisms: Argon2 password hashing through argon2-cffi for secure credential storage; account lockout after five consecutive failed login attempts with a 15-minute lockout period enforced through Flask-Limiter rate limiting; and Google OAuth 2.0 integration via Authlib for alternative authentication.

Role-based access control (RBAC) was implemented using Flask-Login with two primary roles: Administrator (full system access) and Customer (restricted to portal functions). Custom route decorators enforce role-based access restrictions on all protected endpoints. Session management uses Flask-Login's permanent session with configurable inactivity timeout, secure HttpOnly and SameSite cookie settings, and complete server-side session destruction on logout.

***Phase 3: Inventory and Product Catalog***

The inventory management module provided real-time stock tracking with supported for both saleable and rentable products. Products can be configured as sale-only, rent-only, or both, with independent pricing for each mode. Stock quantities were automatically adjusted upon order confirmation and rental checkout, with row-level locking (with_for_update()) preventing race conditions during concurrent transactions.

The product catalog supported categorization, search, and filtering for customer browsing. Administrators can added, edit, archive, and restock products through the admin panel. An inventory audit trail was maintained through the inventory_logs table, recording every stock change with timestamps, change types, and user identifiers for complete

traceability — directly addressing the oxygen tank tracking challenges identified during requirements gathering.

***Phase 4: Cart, Order and Payment System***

The order processing module manages the complete transaction lifecycle from cart creation through payment confirmation. Customers can added products to their cart with quantity and rental duration specifications. The checkout process implemented automatic total calculation including deposits for rental items, with the deposit amount varying based on delivery distance as per CareMed's operational policy.

Two payment methods were supported: Cash on Delivery (COD) and GCash online payment. For GCash payments, customers upload a screenshot of the transaction with the reference number through the payment_proof system. The order tracking workflow follows a defined status progression: Submitted, Verified, Shipped, and Delivered — providing customers with transparent order status visibility.

***Phase 5: Rental Management and Invoicing***

The rental management module addressed the most operationally critical aspect of CareMed's business. Rental agreements were created at equipment check-out, capturing the customer identifier, rented items with inventory references, rental start and end dates, deposit amount, and monthly rental rate. The system generates monthly rental invoices through the rental_invoice table, tracking period start/end dates and payment status.

Return processing updates the rental status and automatically restores returned items to available inventory. Special handling was implemented for oxygen tanks, which were returned to an 'Empty' status for refill before being made

available for new rentals — reflecting the actual operational workflow at CareMed. Late fee calculation and the one-week extension policy for delayed returns were implemented as configurable business rules.

***Phase 6: Security, Reports and Dashboard***

The security monitoring module implemented a comprehensive Intrusion Detection System (IDS) with multiple protective layers. Security events were logged to the security_logs table with user identifiers, IP addressed, user agents, event descriptions, and severity levels. Automatic IP blocking was triggered after repeated violations, with blocked IP addressed stored in the blocked_ips table with configurable expiration.

Bot detection through User-Agent pattern checking was applied at login endpoints. Flask-Talisman enforces security headers including Content Security Policy (CSP), HTTP Strict Transport Security (HSTS), and X-Frame-Options. All security events trigger email notifications to administrators through the Gmail SMTP integration. The admin Security Center provided a centralized interface for monitoring security logs, managing blocked IPs, and reviewing system alerts.

The reporting and dashboard module provided administrators with key performance indicators (KPIs), transaction summaries, inventory status reports, and customer activity analytics. Daily automated database backups were scheduled through APScheduler at midnight, with the system maintaining the last seven backup files. Manual backup and restore tools were also available through the admin panel.

***Phase 7: Testing and Quality Assurance***

Comprehensive testing was conducted across five modalities: unit testing of individual functions; integration

testing of module interactions; system testing of end-to-end workflows; security testing against the OWASP Top 10 framework; and functional testing of all completed system features. At the conclusion of this phase, the system was fully developed and functional, with all core modules — inventory management, rental processing, order and payment handling, and security monitoring — operating as specified in the requirements.

## 3.3 System Architecture

The CareMed system follows a three-tier client-server architecture integrated with the Model-View-Controller (MVC) design pattern. This architectural approach was selected for its proven scalability, separation of concerns, and compatibility with web-based application development. The three-tier architecture divides the system into distinct presentation, application, and data layers, each responsible for specific functions and communicating through well-defined interfaces [52].

### *3.3.1 Three-Tier Architecture Overview:*

The presentation tier handles all user interface rendering and client-side interactions. Users access the system through standard web browsers (Chrome, Firefox, Safari, Edge) communicating with the server via HTTP/HTTPS protocols. The presentation layer was built using HTML5 for semantic document structure, CSS3 for styling and responsive layout adaptation, JavaScript for client-side interactivity (including dynamic cart updates, modal dialogs, and AJAX requests), Bootstrap 5 as the front-end component framework, and Jinja2 as the server-side templating engine for dynamic HTML generation.

The application tier contained the core business logic, request processing, and security enforcement. Flask 3.1.3 served as the web application framework, handling URL routing,

HTTP request/response processing, and session management. The application layer was organized into modular components: route handlers (controllers) process incoming requests and coordinate model and view interactions; business logic services encapsulate domain-specific operations such as order processing, rental management, and payment handling; the ORM layer (Flask-SQLAlchemy) mediates all database interactions; and security middleware (Flask-Login, Flask-Limiter, Flask-WTF, Flask-Talisman) enforces authentication, rate limiting, CSRF protection, and security headers.

The data tier manages persistent storage through MySQL, accessed exclusively through SQLAlchemy's parameterized query interface to prevent SQL injection vulnerabilities. The database schema consists of 16 normalized tables with proper indexing on frequently queried fields (email addressed, reference numbers, transaction dates) and foreign key constraints ensuring referential integrity. Database connection pooling with pool_pre_ping validates connections before use, preventing errors from stale connections in the pool.

### *3.3.2 Model-View-Controller Pattern:*

Within the three-tier architecture, the system implemented the Model-View-Controller (MVC) pattern to achieve separation of concerns and maintainable code organization. The Model component consists of SQLAlchemy model classes that defined the database schema and encapsulate data access operations. Each model class corresponds to a database table and provided methods for creating, reading, updating, and deleting records while enforcing data validation rules.

The View component comprises Jinja2 HTML templates that render dynamic content for delivery to user browsers. Templates were organized into separate directories for customer-facing pages (/customer/), administrative pages

(/admin/), and shared components (layouts, macros, partials). Bootstrap 5 provided the responsive grid system and UI components, ensuring consistent presentation across desktop and mobile devices.

The Controller component consists of Flask route handler functions that process HTTP requests, invoke appropriate model operations, and select the correct view for response rendering. Route handlers were organized into Blueprints — Flask's mechanism for modular application structure — with separate blueprints for authentication, customer portal, admin panel, and API endpoints. Custom decorators enforce role-based access control at the route level.

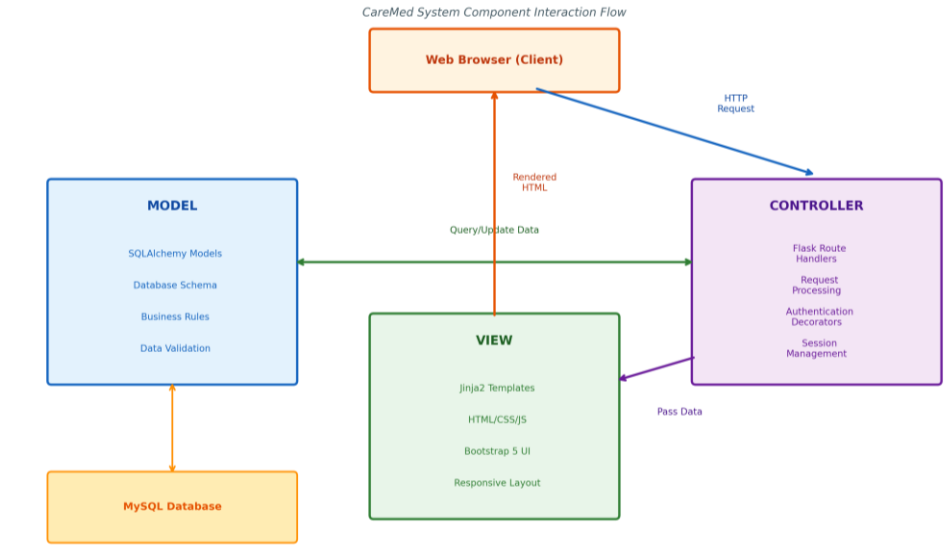



*Figure 6. Model-View-Controller (MVC) Architecture Pattern*

### *3.3.3 Integrated Security Layer:*

A distinguishing feature of the CareMed architecture was the integrated security layer that operated across all three tiers. Unlike conventional approaches that added security as an afterthought, the CareMed system was designed with security as a foundational architectural component from the initial

design phase — a security-by-design approach consistent with OWASP recommendations and the Philippine Data Privacy Act of 2012 requirements.

At the presentation layer, security was enforced through Jinja2's auto-escaping mechanism, which automatically encodes HTML special characters in all template-rendered content, preventing Cross-Site Scripting (XSS) attacks. CSRF tokens generated by Flask-WTF were included in all state-changing forms and validated server-side before processing.

At the application layer, security controls include: Flask-Login for session-based authentication with configurable timeout and secure cookie settings; argon2-cffi for Argon2 password hashing with adjustable work factor; Flask-Limiter for rate limiting on authentication endpoints (5 attempts per 5 minutes); Flask-WTF for form validation and CSRF protection; Flask-Talisman for HTTP security headers (CSP, HSTS, X-Frame-Options); and the custom IDS module for security event logging, IP blocking, and bot detection.

At the data layer, security was maintained through: SQLAlchemy ORM parameterized queries preventing SQL injection; Argon2 password hashing ensuring credential security even in the event of database compromise; row-level database locking preventing race conditions in concurrent transactions; and comprehensive audit logging through inventory_logs and security_logs tables providing complete traceability of all system events.

### *3.3.4 External Service Integrations:*

The system integrates with three external services to extend its functionality beyond core capabilities. Google OAuth 2.0 provided social login functionality, allowing customers to authenticate using their existing Google accounts. The OAuth implementation follows the authorization code flow with PKCE

extension, using state and nonce parameters to prevent CSRF and replay attacks.

Gmail SMTP handles all outgoing email communications including account verification emails, password reset links, order status notifications, security alerts to administrators, and rental return reminders. The SMTP connection uses TLS encryption for secure email transmission.

The mysqldump utility was integrated for database backup operations, invoked through Python's subprocess module. Automated daily backups were scheduled via APScheduler (Advanced Python Scheduler) running as a background thread within the Flask application, with backup files maintained on the server for seven days.

### *3.3.5 System Architecture Diagram:*

The following diagram illustrates the complete system architecture, showing the three-tier structure, security layer integration, external service connections, and data flow patterns:

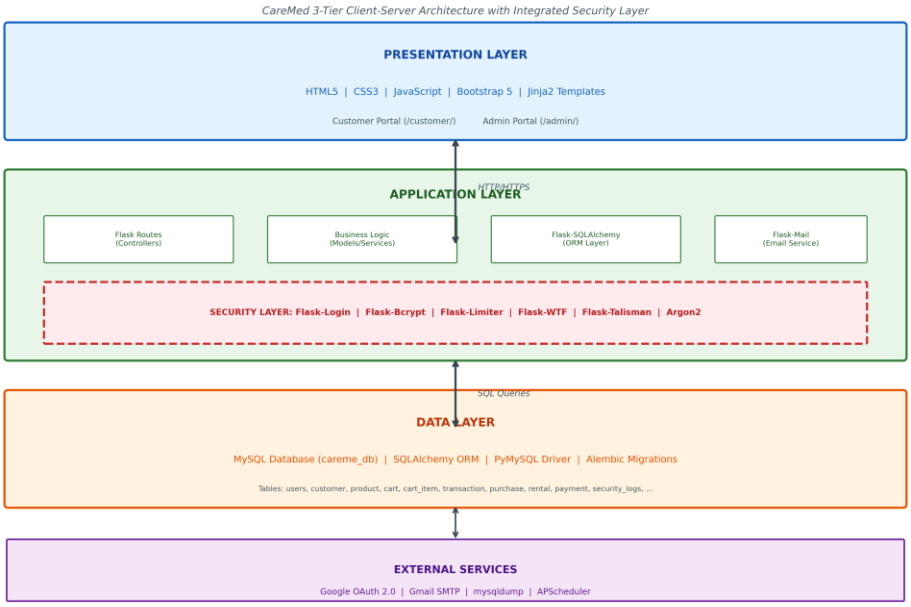



*Figure 7. System Architecture Diagram – Three-Tier with Security Layer*

## 3.4 Tools and Technologies Used

The selection of development tools and technologies for the CareMed system was guided by several criteria: technical capability to meet system requirements; security features and track record; availability of comprehensive documentation and community supported; compatibility with Philippine web hosting environments; and alignment with the skill sets of the development team. Each technology choice was evaluated against alternatives to ensured optimal fit for the project's specific requirements.

### *3.4.1 Programming Languages:*

| Language | Purpose and Justification |
|---|---|
| Python 3 | Primary backend language for application logic, routing, security implementation, and database interaction. Selected for readable syntax and extensive library ecosystem. |

| Language | Purpose and Justification |
|---|---|
| HTML5 | Markup language for page structure and semantic document organization. |
| CSS3 | Style sheet language for visual presentation, responsive layouts, and cross-browser compatibility. |
| JavaScript | Client-side scripting for dynamic interactions, AJAX requests, and form validation feedback. |
| SQL | Database query language used through SQLAlchemy ORM for all data operations. |

*Table 2. Programming Languages Used*

Python 3 was selected as the primary programming language for its clear, readable syntax that facilitates collaborative development and long-term maintenance. Python's extensive standard library and rich ecosystem of third-party packages provided comprehensive supported for web development, database interaction, security implementation, and task scheduling. The Flask micro-framework, built on Python's WSGI toolkit, offers a lightweight yet extensible foundation that allowed the development team to select purpose-specific libraries for each system requirement rather than inheriting bundled components that may not be needed [53].

HTML5, CSS3, and JavaScript handle all client-side presentation and interaction. HTML5 provided semantic document structure that supported accessibility requirements. CSS3 enabled responsive layout adaptation across device sizes through media queries and flexbox/grid layouts. JavaScript handles dynamic behaviors including cart management, form validation feedback, modal dialogs, and AJAX requests for partial page updates without full page reloads.

### *3.4.2 Frameworks and Libraries:*

| Framework/Library | Purpose |
|---|---|
| Flask 3.1.3 | Web application framework providing routing, request handling, and session management. |

| Framework/Library | Purpose |
|---|---|
| Flask-SQLAlchemy | Object-Relational Mapping (ORM) for database abstraction and query building. |
| Flask-Migrate / Alembic | Database migration management for version-controlled schema changes. |
| Flask-Login | User session management and authentication state tracking. |
| Flask-WTF / WTForms | Form handling, server-side validation, and CSRF token protection. |
| Flask-Mail | Email sending capability for verification, notifications, and alerts. |
| Flask-Limiter | Rate limiting for brute-force protection and resource management. |
| Flask-Talisman | HTTP security headers enforcement (CSP, HSTS, X-Frame-Options). |
| Authlib | OAuth 2.0 client implementation for Google authentication. |
| argon2-cffi | Argon2 password hashing for secure credential storage. |
| APScheduler | Background task scheduling for automated database backups. |
| Pillow | Image processing for uploaded file validation. |
| Gunicorn | Production WSGI HTTP server for deployment. |

*Table 3. Frameworks and Libraries Used*

### *3.4.3 Database Technology:*

| Component | Specification |
|---|---|
| DBMS | MySQL (MariaDB 10.4.32) |
| Database Name | careme_db |
| Driver | PyMySQL |
| ORM | SQLAlchemy (via Flask-SQLAlchemy) |
| Migration Tool | Alembic (via Flask-Migrate) |
| Connection Pooling | Enabled with pool_pre_ping |
| Tables | 16 normalized tables |

*Table 4. Database Configuration*

MySQL was selected as the relational database management system for its proven reliability, ACID compliance, wide adoption in Philippine web hosting environments, and seamless compatibility with the Flask/Python technology stack. All database interactions were performed through SQLAlchemy's ORM layer using parameterized queries, ensuring protection against SQL injection attacks regardless of user input content. Alembic, working through Flask-Migrate, provided version-controlled database schema migrations that can be applied incrementally and rolled back if necessary.

### *3.4.4 Development Tools:*

| Tool | Purpose |
|---|---|
| Visual Studio Code / Cursor | Integrated development environment (IDE) |
| Git | Distributed version control system |
| Python venv | Virtual environment for dependency isolation |
| pip | Python package installer |
| MySQL Workbench / CLI | Database management and query execution |
| Flask debug server | Local development server with auto-reload |
| python-dotenv | Environment variable configuration |

*Table 5. Development Tools*

### *3.4.5 External Services:*

| Service | Protocol | Purpose |
|---|---|---|
| Google OAuth 2.0 | HTTPS/OAuth 2 | Social login with Google accounts |
| Gmail SMTP | TLS/SMTP | Email delivery for verification and alerts |
| mysqldump | Local CLI | Automated database backup utility |

*Table 6. External Service Integrations*

The technology stack was chosen to created a complete, production-ready system while remaining accessible to small

enterprises with limited technical resources. All selected technologies were open-source and freely available, eliminating licensing costs that would otherwise be prohibitive for small businesses. The stack was well-documented with active communities, ensuring that future maintenance and enhancement can be performed without specialized expertise.

## 3.5 Data Gathering Techniques

The data gathering phase employed three complementary techniques to developed a comprehensive understanding of CareMed's operational environment: structured interviews with management and staff, direct observation of business processes, and document analysis of existing records. The triangulation of data from multiple sources enhanced the validity and reliability of the requirements analysis by cross-verifying information across different methods [54].

### *3.5.1 Interview:*

A semi-structured faced-to-faced interview was conducted with CareMed's owner and operational staff at their business premises. The interview followed a prepared protocol covering six thematic areas while allowing for spontaneous follow-up questions to explore emerging topics in depth. The interview session lasted approximately two hours and was documented through detailed field notes.

***Interview Protocol***

The interview protocol was organized into the following thematic sections:

- Business Overview: Nature of operations, target customers, product categories, and service area coverage.

- Current Workflow: Step-by-step process for sales transactions, rental agreements, equipment returns, and inventory management.
- Operational Challenges: Difficulties encountered in daily operations, frequency and types of errors, and their impact on service quality.
- Record-Keeping Practices: Methods used for tracking inventory, customer information, rental durations, payments, and expenses.
- Customer Communication: Channels used for customer interaction, order placement, status updates, and complaint handling.
- Proposed Features: Desired system capabilities, priority functionality, and expectations for the digital system.

***Key Interview Findings***

The interview revealed several critical operational insights that directly shaped system design decisions. CareMed's customer base consists primarily of hospital discharge patients requiring home recovery equipment and cancer patients needing oxygen tanks and hospital beds for extended home care. All transactions were conducted through Facebook Messenger or in-person visits, requiring customers to physically visit the warehouse to view available products.

The rental process required customers to provided two valid IDs and proof of billing, with deposits varying based on delivery distance — the farther the location, the higher the deposit to cover retrieval costs. Rental duration monitoring follows a bi-monthly cycle with cut-offs on the 5th and 20th of each month for reconciling oxygen tank refills against supplier records from Prime Gas.

Payment tracking emerged as a significant pain point: payments made via GCash without accompanying notification messages required the owner to manually match transaction numbers, and verbal payment confirmations occasionally led to disputes when records were not immediately updated. The Excel-based tracking system, while functional at low volumes, became increasingly difficult to manage as the business grew, with occasional forgotten inventory reports and difficulty verifying oxygen tank counts before supplier refills.

### *3.5.2 Observation:*

Direct observation of CareMed's operational environment was conducted during the on-site interview session. The research team observed the physical layout of the warehouse, the flow of customer interactions, the handling of rental equipment (particularly oxygen tanks), and the process of updating Excel records. Observational notes documented the workspace organization, equipment storage methods, customer service patterns, and the manual steps involved in completing a typical transaction.

Key observations included: the single-vehicle constraint for deliveries with reliance on tricycle drivers as backup for oxygen tank transport; the visual verification process for confirming empty oxygen tanks before replacement; the practice of photographing equipment and customer documents via mobile phone for Messenger-based transactions; and the manual counting of oxygen tanks before each Prime Gas refill to establish baseline consumption records.

### *3.5.3 Document Analysis:*

Document analysis involved the examination of existing business records including Excel spreadsheets used for inventory tracking, rental duration monitoring, and expense recording. Analysis of these documents revealed the data structure and information flows underlying CareMed's manual

processes, as well as the types of errors and inconsistencies that occurred.

The Excel records showed: multiple worksheets tracking different aspects of operations (inventory, rentals, payments, expenses) without automated cross-referencing; manual calculation of rental durations and fees; inconsistent data entry formats across different record keepers; absence of audit trails or version history; and the complete reliance on the owner's knowledge for linking related records across different worksheets.

#### *3.5.4 Requirements Specification:*

Based on the combined findings from interviews, observation, and document analysis, a formal requirements specification was developed categorizing requirements into functional, non-functional, and security domains. The specification was organized using the following priority framework:

| Category | Priority | Key Requirements |
|---|---|---|
| Functional | High | User registration/login, product catalog, cart/checkout, rental management, order tracking, payment processing |
| Non-Functional | High | Responsive design, concurrent user support, data integrity, automated backups |
| Security | High | Authentication, RBAC, session management, input validation, CSRF protection, XSS prevention, SQL injection prevention, audit logging |

*Table 7. System Requirements Summary*

## 3.6 Testing Procedures

The CareMed system underwent comprehensive testing across five modalities to ensured functional correctness, security resilience, and usability. The testing approach was designed

to systematically verify that the system meets its specified requirements and operated reliably under realistic usage conditions. Each testing modality employed structured test cases with defined inputs, expected outputs, and pass/fail criteria.

### *3.6.1 Unit Testing:*

Unit testing was performed manually during development to verify the correct behavior of individual functions and components in isolation. Each system module was tested as it was developed, with developers verifying that functions produced correct outputs for expected inputs and handled edge cases appropriately.

***Authentication Module Unit Tests***

| ID | Test Case | Expected Result | Status |
|---|---|---|---|
| UT-A01 | Valid user registration with email verification | Account created, verification email sent | Pass |
| UT-A02 | Login with correct credentials | Successful authentication, session created | Pass |
| UT-A03 | Login with incorrect password | Authentication failed, attempt logged | Pass |
| UT-A04 | Account lockout after 5 failed attempts | Account locked for 15 minutes | Pass |
| UT-A05 | Google OAuth login flow | Successful authentication via Google | Pass |
| UT-A06 | Password reset via email | Reset link sent, password update successful | Pass |
| UT-A07 | Session timeout after inactivity | Session expired, redirect to login | Pass |

| ID | Test Case | Expected Result | Status |
|---|---|---|---|
| UT-A08 | Logout and session destruction | Session cleared, cookies removed | Pass |

*Table 8. Authentication Module Unit Test Cases*

### *Inventory Module Unit Tests*

| ID | Test Case | Expected Result | Status |
|---|---|---|---|
| UT-I01 | Add new product with valid data | Product created with correct fields | Pass |
| UT-I02 | Update product stock quantity | Stock updated, inventory log created | Pass |
| UT-I03 | Deduct stock on order confirmation | Quantity reduced, log entry created | Pass |
| UT-I04 | Restore stock on rental return | Quantity restored, status updated | Pass |
| UT-I05 | Archive product (soft delete) | Product hidden but data preserved | Pass |
| UT-I06 | Low stock alert generation | Alert triggered when below threshold | Pass |

*Table 9. Inventory Module Unit Test Cases*

### *Order Processing Unit Tests*

| ID | Test Case | Expected Result | Status |
|---|---|---|---|
| UT-O01 | Add items to cart (sale) | Cart updated with correct items and totals | Pass |
| UT-O02 | Add items to cart (rental) | Cart updated with deposit calculation | Pass |
| UT-O03 | Checkout with COD payment | Order created with COD status | Pass |
| UT-O04 | Checkout with | Order created, payment | Pass |

| ID | Test Case | Expected Result | Status |
|---|---|---|---|
| | GCash payment | proof upload requested | |
| UT-O05 | Order tracking status update | Status progresses through workflow correctly | Pass |
| UT-O06 | Payment proof upload and verification | Proof stored, payment status updated | Pass |

*Table 10. Order Processing Unit Test Cases*

### *3.6.2 Integration Testing:*

Integration testing verified the correct interaction between system modules and the accuracy of data flows across component boundaries. Test scenarios were designed to exercise the complete pathways that data follows through the system, from initial input through processing to final output.

| ID | Integration Point | Test Scenario | Status |
|---|---|---|---|
| IT-01 | Auth + Database | User registration stores data, retrieves on login | Pass |
| IT-02 | Cart + Inventory | Adding to cart reflects available stock correctly | Pass |

| ID | Integration Point | Test Scenario | Status |
|---|---|---|---|
| IT-03 | Order + Payment | Order creation triggers correct payment workflow | Pass |
| IT-04 | Rental + Invoice | Rental checkout generates monthly invoice entries | Pass |
| IT-05 | Auth + Security Log | Login events recorded in security_logs table | Pass |
| IT-06 | Payment + Inventory | Confirmed payment deducts stock accurately | Pass |
| IT-07 | Return + Inventory | Rental return restores items to available stock | Pass |

*Table 11. Integration Test Cases*

### *3.6.3 System Testing:*

System testing evaluated the complete application as an integrated whole, verifying that all modules function correctly together and that the system satisfies its overall requirements. End-to-end test scenarios were executed following the actual operational workflows identified during the requirements gathering phase.

| ID | Scenario | Expected Outcome | Status |
|---|---|---|---|
| ST-01 | Complete customer purchase workflow | Order placed, payment confirmed, inventory updated | Pass |
| ST-02 | Complete rental workflow with return | Rental created, invoice generated, return | Pass |

| ID | Scenario | Expected Outcome | Status |
|---|---|---|---|
| | | processed | |
| ST-03 | Admin inventory management workflow | Products added, stock adjusted, reports generated | Pass |
| ST-04 | Security incident and response workflow | Attack detected, IP blocked, admin notified | Pass |
| ST-05 | Daily automated backup execution | Backup file created at scheduled time | Pass |

*Table 12. System Test Cases*

### *3.6.4 Security Testing:*

Security testing was structured around the OWASP Top 10 [55] framework to systematically verify the system's resistance to the most critical and commonly exploited web application security risks. Each OWASP category was addressed through specific test cases designed to confirm the effectiveness of implemented security controls.

| OWASP ID | Category | Test Case | Status |
|---|---|---|---|
| A01 | Broken Access Control | Verify RBAC prevents unauthorized endpoint access | Pass |
| A02 | Cryptographic Failures | Verify Argon2 hashing, HTTPS enforcement, secure cookies | Pass |
| A03 | Injection | Verify SQL injection prevention via parameterized queries | Pass |

| OWASP ID | Category | Test Case | Status |
|---|---|---|---|
| A04 | Insecure Design | Verify security-by-design architecture and defense in depth | Pass |
| A05 | Security Misconfiguration | Verify Flask-Talisman headers, debug mode disabled | Pass |
| A06 | Vulnerable Components | Verify all dependencies updated to latest stable versions | Pass |
| A07 | Auth Failures | Verify account lockout, session timeout, secure password policy | Pass |
| A08 | Integrity Failures | Verify CSRF tokens on all state-changing operations | Pass |
| A09 | Logging Failures | Verify security event logging and admin notification system | Pass |
| A10 | SSRF | Verify URL validation on external service integrations | Pass |

*Table 13. OWASP Top 10 Security Test Cases*

### *3.6.5 User Acceptance Testing (UAT):*

User Acceptance Testing was conducted with representative participants from both Administrator and Customer user roles to validate that the system meets operational needed and usability expectations. UAT participants performed structured test scenarios derived from CareMed's actual business workflows, with findings documented through observation forms and feedback questionnaires.

| ID | User Role | Scenario | Success Criteria | Status |
|---|---|---|---|---|
| UAT-01 | Customer | Register account and verify email | Account activated within 2 minutes | Pass |
| UAT-02 | Customer | Browse products and add to cart | Correct items, prices, and quantities | Pass |

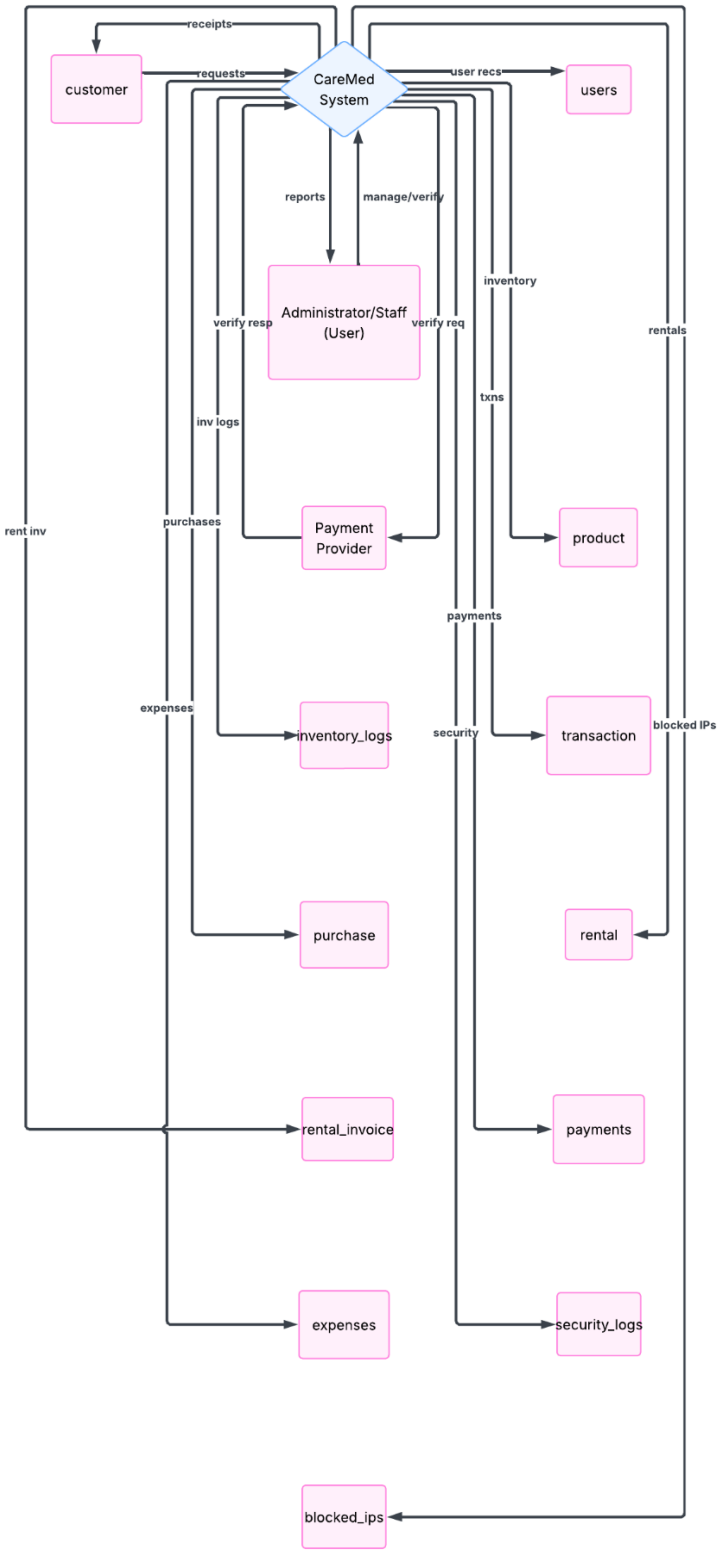
| ID | User Role | Scenario | Success Criteria | Status |
|---|---|---|---|---|
| UAT-03 | Customer | Complete rental checkout with ID upload | Order placed, deposit calculated correctly | Pass |
| UAT-04 | Customer | Upload GCash payment proof | File accepted, payment status updated | Pass |
| UAT-05 | Customer | Track order status | Real-time status display accurate | Pass |
| UAT-06 | Admin | Process walk-in sale | Transaction recorded, inventory updated | Pass |
| UAT-07 | Admin | Update order tracking status | Status updated, customer sees change | Pass |
| UAT-08 | Admin | View security center and blocked IPs | All security events visible | Pass |

*Table 14. User Acceptance Test Scenarios*

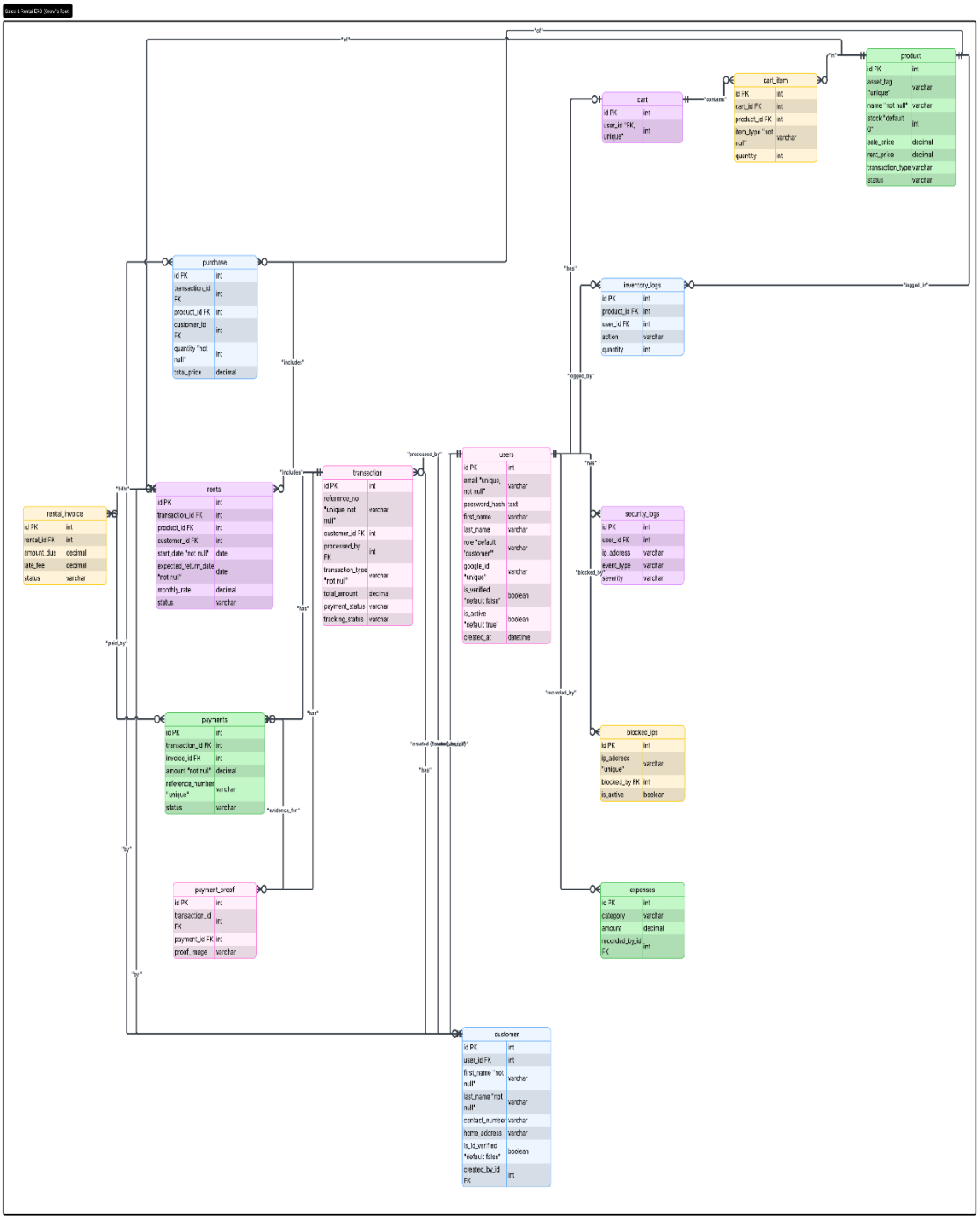
**System Diagrams**

The following diagrams provided visual representations of the system's data flows, database structure, user interactions, and architectural organization:

## Data Flow Diagrams

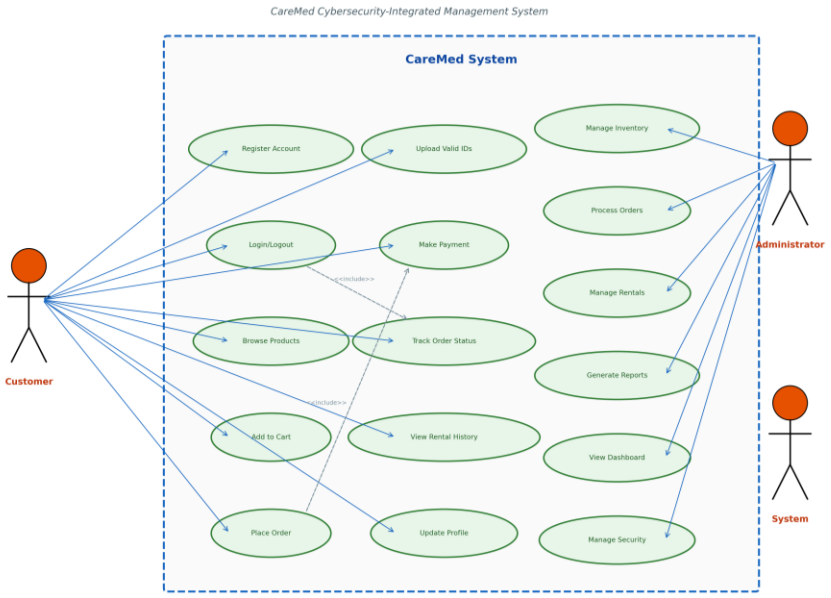



*Data Flow Diagram Level 0 (Context Diagram)Entity Relationship Diagram*




*Entity Relationship Diagram (ERD) - Complete Database Schema*

## Use Case Diagram




*Use Case Diagram - System Actors and Use Cases*

# REFERENCES

[50] R. C. Richey and J. D. Klein, "Developmental research methods: Creating knowledge from instructional design and development practice," J. Comput. Higher Educ., vol. 16, no. 2, pp. 23–38, 2005.

[51] K. Beck et al., "Manifesto for Agile software development," Agile Alliance, 2001. [Online]. Available: https://agilemanifesto.org/

[52] W. Stallings, Network Security Essentials: Applications and Standards, 6th ed. Boston, MA, USA: Pearson, 2018.

[53] A. Ronacher, "Flask (web framework)," Pallets Projects, 2010. [Online]. Available: https://flask.palletsprojects.com/

[54] J. W. Creswell and J. D. Creswell, Research Design: Qualitative, Quantitative, and Mixed Methods Approaches, 5th ed. Thousand Oaks, CA, USA: SAGE, 2018.

[55] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

# CHAPTER IV

## SYSTEM DEVELOPMENT, IMPLEMENTATION, TESTING, EVALUATION, RESULTS, AND DISCUSSION

### 4.1 Overview of the Developed System

CareMed is a web-based, mobile-responsive management information system developed for CareMed Medical Supply Enterprise. The system replaces manual spreadsheet and paper-based processes with a centralized MySQL database and a Flask web application that supports real-time inventory visibility, sales and rental transactions, customer verification, payment tracking, operational reporting, and integrated cybersecurity monitoring. Unlike generic inventory or point-of-sale applications, CareMed was designed with business automation and information security as co-equal priorities: authentication, authorization, input validation, audit logging, and intrusion-detection-style monitoring are embedded in the application core.

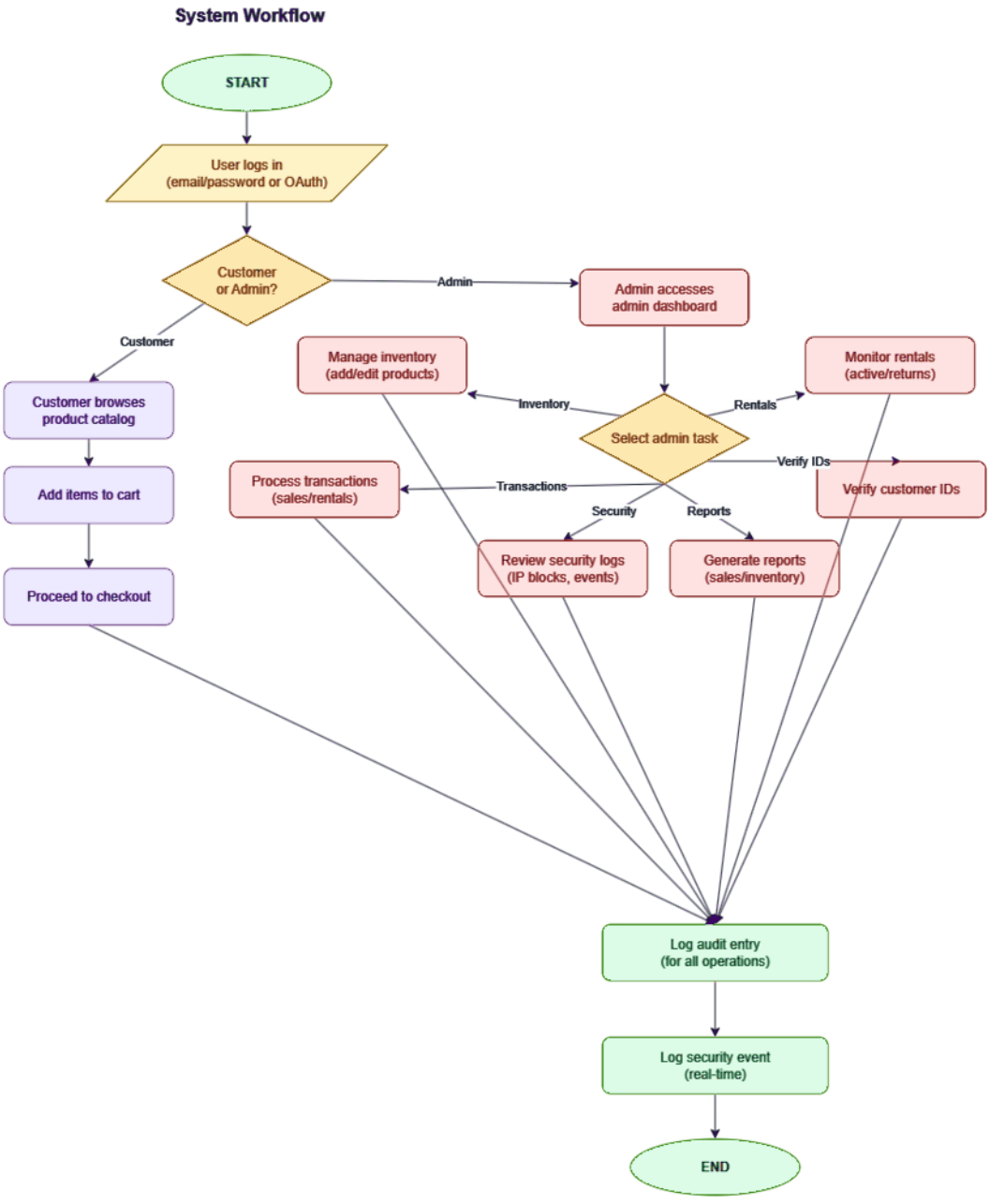
**The major functional objectives realized in the developed system include:**

- Automating inventory movements (stock-in, stock-out, adjustments) with chronological inventory logs per product.
- Supporting product catalog browsing for customers with sale and rental options, shopping cart, and online checkout.
- Processing walk-in and online sales and rentals, including delivery tracking and rental invoicing with monthly billing periods.
- Managing customer records, valid-ID verification, and linked user accounts for the customer portal.

- Recording payments, payment proofs, partial payments, and administrator confirmation workflows.
- Supporting specialized medical-supply operations such as refillable tank tracking (TankStatus), refill transactions, and PrimeGas processing.
- Generating administrator dashboards, financial and inventory reports, expense records, and exportable transaction data.
- Implementing staff accounts with granular permissions, security event logs, IP blocking, rate limiting, and scheduled database backups.

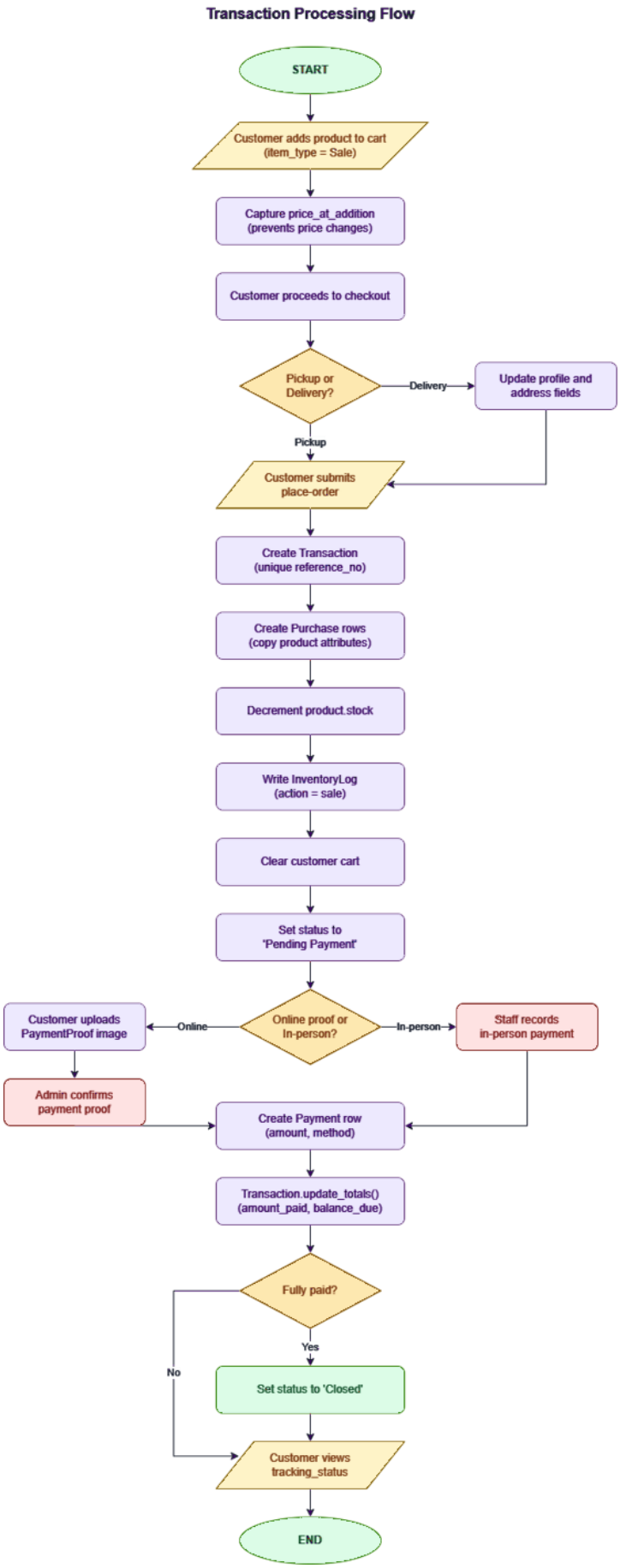
### *4.1.1 System Workflow:*

The workflow begins with user authentication. Once authenticated, customers browse the product catalog, add items to the cart, and proceed to checkout. Administrators access the admin dashboard to manage inventory, process transactions, verify customer IDs, monitor rentals, review security logs, and generate reports. All significant operations trigger audit entries, and security events are logged in real time.



### *4.1.2 Sale Transaction Processing Flow:*

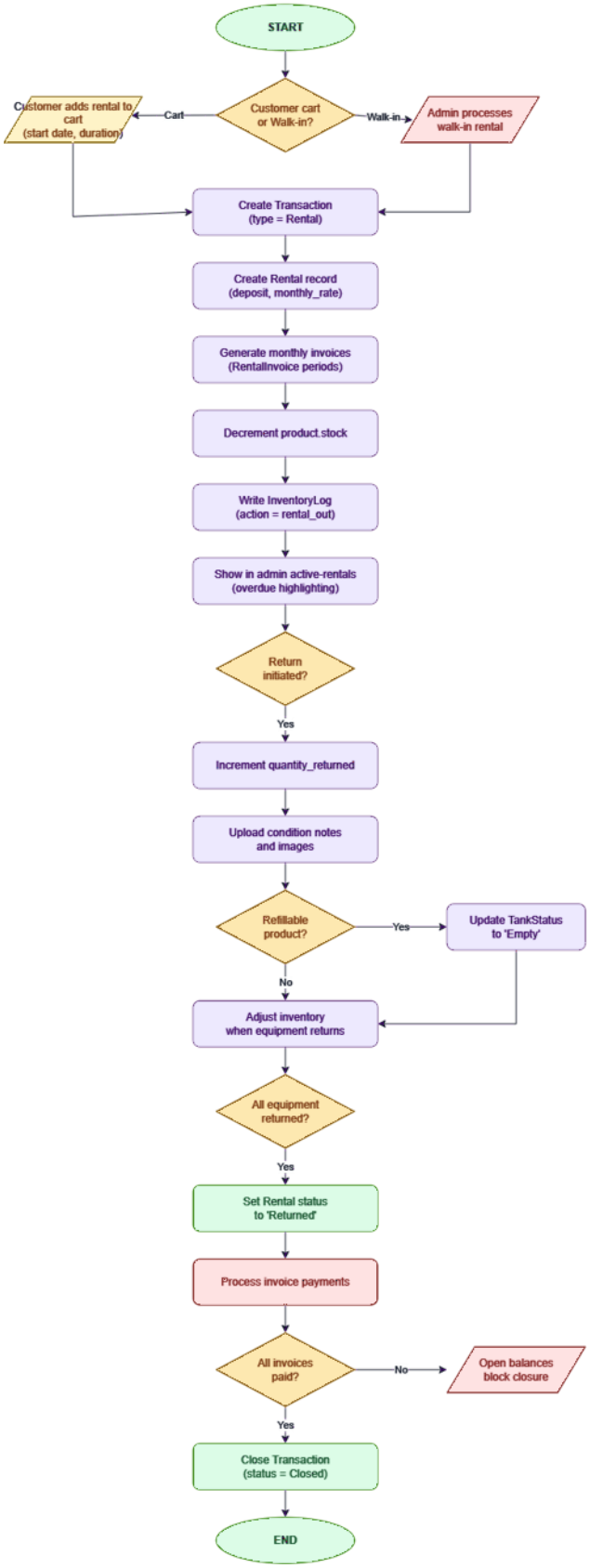
An online sale begins when an authenticated customer adds products to the Cart with item_type set to Sale. The cart route aggregates line totals using price_at_addition captured at add-to-cart time to prevent silent price changes during checkout. At checkout, the customer confirms fulfillment type (pickup/walk-in versus delivery), optionally updates profile and address fields, and submits place-order. The server creates a Transaction with a unique reference_no, associated Purchase rows copying product attributes for auditability, decrements product.stock, and writes InventoryLog actions. Payment may remain Unpaid until the customer uploads PaymentProof or staff records payment in person. Administrators use confirm-payment-proof or post-payment to create Payment rows; Transaction.update_totals recalculates amount_paid, balance_due, and payment_status. When fully paid and fulfillment rules are satisfied, status transitions to Closed. Customers monitor progress via tracking_status on the user dashboard.


Transaction Processing Flow
START
Customer adds product to cart
(item_type = Sale)
Capture price_at_addition
(prevents price changes)
Customer proceeds to checkout
Pickup or
Delivery?
Delivery
Update profile and
address fields
Pickup
Customer submits
place-order
Create Transaction
(unique reference_no)
Create Purchase rows
(copy product attributes)
Decrement product.stock
Write InventoryLog
(action = sale)
Clear customer cart
Set status to
'Pending Payment'
Customer uploads
PaymentProof image
Online
Online proof or
In-person?
In-person
Staff records
in-person payment
Admin confirms
payment proof
Create Payment row
(amount, method)
Transaction.update_totals()
(amount_paid, balance_due)
Fully paid?
Yes
No
Set status to 'Closed'
Customer views
tracking_status
END

### *4.1.3 Rental Transaction Processing Flow:*

Rentals may originate from the customer cart (with rental_start_date and rental_duration) or from administrator process-rental for walk-in clients. Creating a rental inserts Rental linked to a rental-type Transaction, captures deposit and monthly_rate, and invokes generate_monthly_invoices to create RentalInvoice periods. Active rentals appear in admin active-rentals with overdue highlighting based on dates and payment status. Returns use transaction return routes to increment quantity_returned, upload condition notes/images, and adjust inventory when equipment re-enters stock. Late fees may be applied at invoice level. Closure requires returned equipment and paid invoices, enforcing the business rule that open balances block complete closure.

### 4.1.3 Rental Transaction Processing Flow



## 4.2 System Development Methodology

Development followed the Agile software development methodology with Scrum-inspired sprints, as specified in Chapter 3. Requirements were gathered through interviews, observation, and document analysis at CareMed; each sprint delivered testable increments that were reviewed before the next iteration. Security requirements were maintained in a dedicated security backlog and addressed in parallel with functional stories, consistent with security integration best practices for SMEs.

**The lifecycle phases implemented in this project were:**

### *4.2.1 Requirements Gathering:*

Requirements gathering was conducted through a semi-structured face-to-face interview with CareMed's owner and operational staff at their business premises. The interview protocol covered business overview, current workflow, operational challenges, record-keeping practices, customer communication, and proposed features. Key findings revealed that CareMed's customer base consists primarily of home-based care patients and small clinics requiring durable medical equipment such as wheelchairs, hospital beds, and oxygen concentrators. The business operates combined sales and rental workflows with a small staff team without dedicated IT personnel. Current record-keeping relied on physical notebooks and spreadsheet tabs, creating vulnerability to version conflicts and data loss. The interview identified critical pain points: inability to track rental equipment returns systematically, no real-time inventory visibility, complete absence of cybersecurity controls, and labor-intensive report generation requiring hours of manual compilation. These findings directly shaped the six-module system architecture and the decision to

integrate cybersecurity as a foundational requirement rather than an afterthought.

#### *4.2.2 System Planning:*

System planning established the project scope, objectives, and technical specifications based on the interview findings. The scope encompassed six integrated modules: Inventory Management, Customer Management, Order and Sales Processing, Rental Tracking, Reports and Dashboard, and Cybersecurity. The target user population comprised two roles reflecting CareMed's organizational structure: Administrator-role users (management and authorized staff) and Customer-role users (registered customers with portal access). The technical platform was selected based on criteria of technical capability, Philippine hosting compatibility, and SME accessibility: Python 3 with Flask as the application server, MySQL as the relational database, SQLAlchemy ORM for database abstraction, Bootstrap 5 for responsive design, and Flask-Login for authentication. External service integrations included Google OAuth 2.0 for social login, Gmail SMTP for email communications, and mysqldump for database backups. The planning phase also defined delimitations: the system was web-based without native mobile apps, did not include direct payment gateway integration, operated as a standalone platform without healthcare system integration, and addressed development through testing without extending to post-deployment maintenance.

#### *4.2.3 Software Development:*

Software development followed the Agile methodology with seven iterative phases. Phase 0: Planning and Requirements Gathering involved workflow analysis and formal requirements specification through interviews and observation. Phase 1: Database Design and Core Setup established SQLAlchemy models, Flask application factory pattern, and Alembic migrations.

Phase 2: Authentication and User Management implemented registration, email verification, login, Google OAuth, password reset, and role-based access control using Flask-Login. Phase 3: Inventory and Product Catalog developed real-time stock tracking, product categorization, search, filtering, and inventory logs with TankStatus for oxygen/refill workflows. Phase 4: Cart, Order, and Payment System created customer cart functionality, checkout processing, payment proof upload for GCash transactions, and administrator payment confirmation workflows. Phase 5: Rental Management and Invoicing built rental agreements, monthly rental invoice generation, return processing, and active rental monitoring with late fee calculation. Phase 6: Security, Reports, and Dashboard implemented security event logging, IP blocking, rate limiting, bot detection, Flask-Talisman security headers, administrator KPI dashboards, financial and inventory reports, and automated daily database backups. Phase 7: Testing and Quality Assurance conducted unit, integration, system, OWASP-oriented security testing, and User Acceptance Testing.

#### *4.2.4 Testing:*

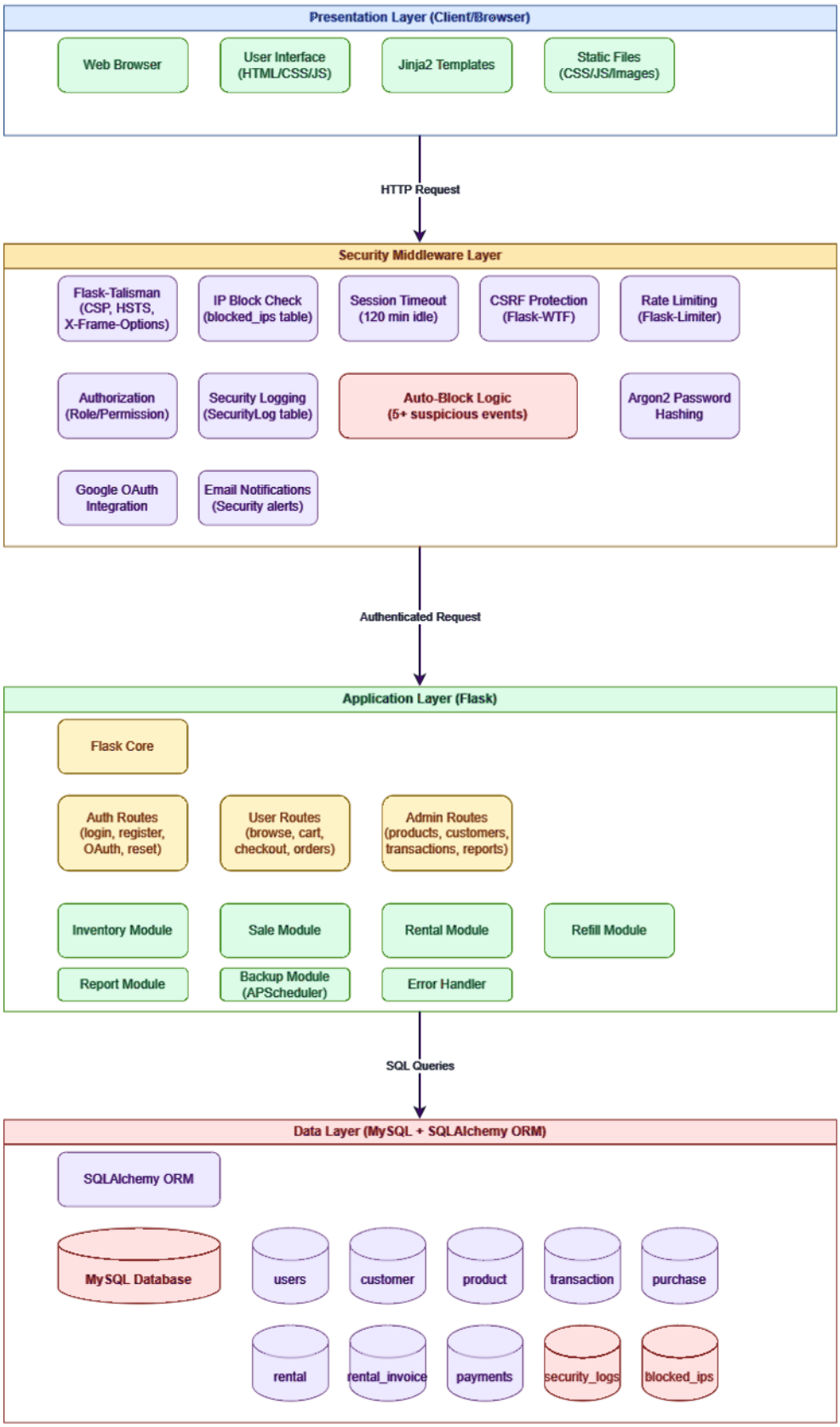
Testing was conducted across five modalities as specified in Chapter 1, Objective 4 and detailed in Chapter 3, Section 3.6. Unit testing was performed manually during development to verify the correct behavior of individual functions and components in isolation, including authentication module tests, inventory module tests, and order processing tests. Integration testing verified the correct interaction between system modules and the accuracy of data flows across component boundaries. System testing evaluated the complete application as an integrated whole, verifying that all modules function correctly together and that the system meets specified requirements. Security testing was structured around the OWASP Top 10 [56] framework to systematically verify the system's

resistance to the most critical web application security risks, including broken access control, cryptographic failures, injection, insecure design, security misconfiguration, and security logging failures. User Acceptance Testing was conducted with representative participants from both Administrator and Customer user roles to validate that the system meets user expectations and business requirements through scenario-based tasks, think-aloud observation, and post-task questionnaires aligned with ISO/IEC 25010 quality dimensions.

### 4.3 System Architecture

CareMed implements a three-tier client–server architecture combined with the Model–View–Controller (MVC) pattern. The presentation tier consists of HTML5, CSS3, Bootstrap 5, and Jinja2 templates rendered in the user's browser. The application tier is implemented in Python 3 using Flask 3.x route handlers (controllers), WTForms for server-side validation, Flask-Login for session management, Flask-WTF for CSRF protection, Flask-Limiter for rate limiting, Flask-Talisman for security headers and Content Security Policy, and Authlib for Google OAuth 2.0 integration. The data tier is a MySQL relational database accessed exclusively through SQLAlchemy ORM with parameterized queries.

**System Architecture — Three-Tier with Security Layer**



### *4.3.1 Primary Architectural Components:*

- **Presentation Layer:** User and admin templates under templates/user and templates/admin; responsive layouts for desktop and mobile.
- **Application Layer:** Blueprints auth_bp, user_bp, and admin_bp; decorators for admin/staff access and fine-grained Permission checks.
- **Data Layer:** Normalized relational schema with foreign keys, cascade rules, and inventory and security audit tables.
- **Cross-cutting Security:** before_request IP block check, admin idle session timeout (120 minutes), CSRF on state-changing forms, CSP via Talisman.

### *4.3.2 Request Processing Flow:*

Each HTTP request passes through an ordered pipeline before reaching blueprint handlers. Flask-Talisman applies security headers and Content Security Policy. The check_blocked_ip before_request handler queries blocked_ips for the client address and aborts with 403 when an active block applies. For authenticated administrators, enforce_admin_session_timeout compares the session last_active timestamp against a 120-minute idle threshold, logging Admin Session Expired when forcing logout. Flask-Login loads the user; CSRF validation runs on applicable POST requests; Flask-Limiter enforces per-route limits (for example, five login POST attempts per five minutes). Route decorators (login_required, admin_or_staff_required, permission_required) enforce authorization. Successful handlers invoke SQLAlchemy for persistence and render Jinja2 templates or JSON for AJAX endpoints such as admin notifications.

### *4.3.3 Security Architecture:*

The security architecture maps to controls described in Chapters 1 and 2: identification and authentication (Flask-Login, Google OAuth, email verification); authorization (role plus Permission flags); session management (secure cookies, strong session protection, admin idle timeout); input validation (WTForms, type coercion, secure filename for uploads); output encoding (Jinja2 auto-escaping); cryptographic protection (Argon2 password hashes, signed timed tokens for reset and verification); logging and monitoring (SecurityLog, rate-limit handler logging); and availability-related measures (backups, pool_pre_ping on database connections).

## 4.4 Software and Hardware Requirements

### *4.4.1 Hardware Requirements:*

| Component | Minimum Specification | Recommended Specification |
|---|---|---|
| Client (user/admin) | Dual-core CPU, 4 GB RAM | Quad-core CPU, 8 GB RAM |
| Network | Stable internet for web access | Broadband connection |
| Server (development/deployment) | 2 vCPU, 2 GB RAM | 2+ vCPU, 4 GB RAM, SSD storage |

*Table 4.1: Hardware Requirements*

### *4.4.2 Software Requirements:*

| Category | Technology | Purpose |
|---|---|---|

| Language / runtime | Python 3.11+ | Application server |
|---|---|---|
| Web framework | Flask 3.1 | HTTP routing, templates, sessions |
| ORM / DB | SQLAlchemy 2.x, Flask-Migrate,MySQL (PyMySQL) | Data persistence and migrations |
| Authentication | Flask-Login,Argon2 (argon2-cffi) | Sessions and password hashing |
| Security | Flask-WTF, Flask-Talisman, Flask-Limiter | CSRF, headers/CSP, rate limiting |
| OAuth / email | Authlib, Flask-Mail | Google sign-in, verification and reset mail |
| Scheduling | APScheduler | Daily automated database backup |
| Frontend | Bootstrap 5, Jinja2 | Responsive UI |
| Deployment | Gunicorn, Docker (optional) | Production WSGI container |
| Version control | Git | Sprint history and collaboration |
| IDE | Visual Studio Code | Development environment |

*Table 4.2: Software Requirements*

### 4.5 Database Design

The database uses a relational model in MySQL. Normalization reduces redundant product and customer snapshots while transaction tables preserve point-in-time product attributes on purchases for historical accuracy. The schema consists of 16 core entities (as specified in Chapter 3, Phase 1), designed to Third Normal Form (3NF) with explicit foreign key constraints.

### *4.5.1 Core Entities and Relationships:*

| Entity / Table | Primary Purpose | Key Relationships |
|---|---|---|
| users | Accounts (Administrator, Staff, Customer) | Links to customer, permissions, security_logs |
| permissions | Staff RBAC flags | One-to-one with staff users |
| customer | Customer profile and ID verification | user_id, purchases, rentals, transactions |
| product | Sale/rental catalog and stock | purchases, rentals, inventory_logs, tank_status |
| tank_status | Refillable tank counts (full/empty/rented) | One-to-one with refillable products |
| transaction | Sale or rental header, payment status | purchases, rentals, payments, payment_proof |
| purchase | Line items for sales | transaction, product, customer |
| rental | Rental agreements and returns | transaction, rental_invoice, product |
| rental_invoice | Periodic rental billing | rental, payments |
| payments / payment_proof | Payment records and proof images | transaction |
| refill_transaction | Gas/refill operations | product, customer, processor |
| inventory_logs | Stock movement audit trail | product, user |
| expenses | Operational expenses | product (optional), recorded_by |
| cart / cart_item | Customer session cart | user, product |
| product_reviews | Customer product ratings | product, user |
| security_logs | Security and IDS events | user (optional) |

| | | |
|---|---|---|
| blocked_ips | Active IP blocks | blocked by admin |

*Table 4.3: Core Database Entities*

### *4.5.2 Detailed Table Specifications:*

**users Table**

| Field | Type | Description |
|---|---|---|
| user_id | INT, PK, AUTO_INCREMENT | Unique account identifier |
| email | VARCHAR(120), UNIQUE | Login email address |
| password_hash | VARCHAR(255) | Argon2id hashed password |
| role | ENUM('Administrator','Staff','Customer') | System role |
| is_verified | BOOLEAN, DEFAULT FALSE | Email verification status |
| is_active | BOOLEAN, DEFAULT TRUE | Account lockout status |
| failed_login_attempts | INT, DEFAULT 0 | Counter for lockout policy |
| last_active | TIMESTAMP | Last session activity (admin timeout) |
| created_at | TIMESTAMP | Account creation date |

**customer Table**

| Field | Type | Description |
|---|---|---|
| customer_id | INT, PK, AUTO_INCREMENT | Unique customer identifier |
| user_id | INT, FK → users | Linked portal account (optional) |
| full_name | VARCHAR(150) | Customer full name |
| phone | VARCHAR(20) | Contact number (masked in UI) |
| address | TEXT | Delivery/billing address |
| id_verification_status | ENUM | Pending, Verified, Rejected |
| id_document_path | VARCHAR(255) | Uploaded valid ID file path |
| created_at | TIMESTAMP | Record creation date |

**product Table**

| Field | Type | Description |
|---|---|---|
| product_id | INT, PK, AUTO_INCREMENT | Unique product identifier |
| name | VARCHAR(200) | Product name |
| description | TEXT | Detailed description |
| category | VARCHAR(100) | Product category |
| sale_price | DECIMAL(10,2) | Price for outright purchase |
| rental_price | DECIMAL(10,2) | Monthly rental rate |
| stock_quantity | INT | Current available stock |
| reorder_threshold | INT | Low-stock alert trigger level |
| is_rentable | BOOLEAN | Available for rental flag |
| is_refillable | BOOLEAN | Oxygen/tank refill flag |

| | | |
|---|---|---|
| image_path | VARCHAR(255) | Product image file path |
| is_active | BOOLEAN | Soft-delete / archive flag |

**transaction Table**

| Field | Type | Description |
|---|---|---|
| transaction_id | INT, PK, AUTO_INCREMENT | Unique transaction identifier |
| reference_no | VARCHAR(50), UNIQUE | Human-readable order reference |
| customer_id | INT, FK → customer | Associated customer |
| transaction_type | ENUM('Sale','Rental') | Transaction classification |
| fulfillment_type | ENUM('Pickup','Delivery') | Delivery method |
| total_amount | DECIMAL(10,2) | Computed grand total |
| amount_paid | DECIMAL(10,2) | Sum of confirmed payments |
| balance_due | DECIMAL(10,2) | Remaining obligation |
| payment_status | ENUM | Unpaid, Partial, Paid |
| status | ENUM | Submitted, Verified, Shipped, Delivered, Closed |
| tracking_status | VARCHAR(100) | Customer-visible tracking state |
| created_at | TIMESTAMP | Transaction creation date |

### 4.6 Deployment and Configuration

Environment variables were loaded from .env (see env.example): SECRET_KEY, DATABASE_URL, Google OAuth credentials, and mail credentials. Production used config.Config with secure cookies and HTTPS enforcement; development used DevConfig. The Dockerfile packaged the application with Gunicorn binding to

port 5000. Database schema evolution was managed through Flask-Migrate (Alembic) under the migrations directory.

## 4.7 Testing, Evaluation, and Results

Testing followed the five modalities defined in Chapter 1, Objective 4, and detailed in Chapter 3, Section 3.6: unit, integration, system, security (OWASP Top 10–oriented manual tests), and User Acceptance Testing (UAT). Each test cycle produced documented test cases with expected and actual results.

### *4.7.1 Unit Testing:*

Unit-level verification focused on model methods and helpers.

| Unit ID | Component | Input / Action | Expected Output | Result |
|---|---|---|---|---|
| UNI-01 | RentalInvoice.remaining_balance | Partial payment recorded | Non-negative balance | Pass |
| UNI-02 | User.masked_email | user@example.com | u***r@example.com pattern | Pass |
| UNI-03 | verify_reset_token | Expired token | None returned | Pass |
| UNI-04 | Transaction.update_totals | Fully paid sale walk-in | status Closed when rules met | Pass |
| UNI-05 | TankStatus.calculate_refill_need | Empty tanks > full tanks | Refill quantity returned | Pass |

*Table 4.4: Unit Test Results*

### *4.7.2 Integration Testing:*

Integration tests validated cross-module flows.

| INT ID | Modules Involved | Scenario | Result |
|---|---|---|---|
| INT-01 | Checkout + Product + InventoryLog | Online sale decreases stock | Pass |
| INT-02 | Rental + RentalInvoice + Transaction | New rental generates invoices | Pass |
| INT-03 | Payment + Transaction | Admin confirms proof | Pass |
| INT-04 | Security + before_request | Block IP then browse site | 403 / block page |
| INT-05 | Refill + TankStatus | Refill updates tank counts | Pass |

*Table 4.5: Integration Test Results*

### *4.7.3 System Testing:*

| Test ID | Scenario | Expected Result | Result |
|---|---|---|---|
| SYS-01 | Customer registers and verifies email | Account created; login enabled after verification | Pass |
| SYS-02 | Customer adds sale item and | Transaction and purchase | Pass |

| | | | |
|---|---|---|---|
| | checks out | created; stock decreased | |
| SYS-03 | Customer rental with monthly billing | Rental invoices generated per period | Pass |
| SYS-04 | Admin verifies customer ID | is_id_verified set; customer notified in UI | Pass |
| SYS-05 | Staff without permission opens reports | Access denied; security log entry | Pass |
| SYS-06 | Export transactions to spreadsheet | Downloadable file with transaction rows | Pass |
| SYS-07 | Scheduled backup job | Backup file created in backup store | Pass |

*Table 4.6: System Test Results*

### *4.7.4 Security Testing (OWASP-Oriented):*

| OWASP Category | Test Procedure | CareMed Control | Result |
|---|---|---|---|
| A01 Broken Access Control | Access /admin routes as customer | Redirects; Unauthorized Access logged | Pass |
| A02 Cryptographic Failures | Inspect stored passwords | Argon2 hashes only; no plaintext | Pass |
| A03 Injection | Submit SQL metacharacters in forms | ORM parameterization; no SQL errors | Pass |
| A04 Insecure Design | Review session and lockout policy | Lockout, idle admin timeout, rate limits | Pass |
| A05 Security Misconfiguration | Review headers and cookies | Talisman CSP/HSTS; HttpOnly cookies | Pass |
| A06 Vulnerable Components | Review requirements.txt versions | Pinned dependencies; updates planned | Pass |

| | | | |
|---|---|---|---|
| A07 Auth Failures | Repeated failed logins | Rate limit and account lockout | Pass |
| A08 Integrity Failures | CSRF token omitted on POST | Request rejected by Flask-WTF | Pass |
| A09 Logging Failures | Trigger security events | Entries in security_logs with metadata | Pass |
| A10 SSRF | Not applicable to primary features | No user-controlled URL fetch | N/A |

*Table 4.7: OWASP Top 10 Security Test Results*

### *4.7.5 User Acceptance Testing:*

UAT followed the three-component protocol from Chapter 2: scenario-based tasks, think-aloud observation, and post-task questionnaire aligned with ISO/IEC 25010 dimensions.

| UAT ID | Participant Role | Task | Completed Without Assistance | Remarks |
|---|---|---|---|---|
| UAT-01 | Administrator | Add product and update stock | Yes | Inventory log recorded |
| UAT-02 | Administrator | Process walk-in sale | Yes | Receipt totals matched cart |
| UAT-03 | Administrator | Create rental and record partial return | Yes | Invoice balances updated |
| UAT-04 | Administrator | Verify customer ID upload | Yes | Status visible on customer profile |
| UAT-05 | Customer | Browse, cart, checkout, upload payment proof | Yes | Tracking status visible |

| | | | | |
|---|---|---|---|---|
| UAT-06 | Staff (limited permissions) | Attempt restricted module | Yes | Permission message shown |

*Table 4.8: User Acceptance Test Results*

## 4.8 Formal Quality Evaluation (ISO/IEC 25010)

Objective 5 required evaluation across functional suitability, performance efficiency, usability, reliability, and security using a five-point Likert instrument. The following questionnaire items were used for evaluation:

| **Dimension** | **Sample Item (1=Strongly Disagree, 5=Strongly Agree)** |
|---|---|
| Functional suitability | The system covers the daily operations CareMed needs (inventory, sales, rentals, reports). |
| Performance efficiency | Pages and reports load within acceptable time under normal use. |
| Usability | I can complete my tasks with minimal training. |
| Reliability | The system behaves consistently during repeated use. |
| Security | I trust that access controls and logging protect business and customer data. |

*Table 4.9: ISO/IEC 25010 Evaluation Instrument*

The instrument was administered to administrator-role and customer-role evaluators following the UAT sessions. Responses were analyzed using descriptive statistics to produce a formal quantitative quality assessment across all evaluated dimensions.

## 4.9 Results and Discussion

The developed CareMed system satisfied the general objective of delivering a cybersecurity-integrated web platform for inventory, sales, rentals, customer management, and reporting. Specific Objective 1 (requirements analysis) was evidenced by the Chapter 3 specification and the functional module overview in Section 4.1. Specific Objective 2 (secure system development) was demonstrated through Argon2 hashing, RBAC, CSRF and CSP controls, security logging, and IP blocking. Specific Objective 3 (functional modules) was implemented across customer and admin blueprints with persistent audit trails for inventory and security events.

Specific Objective 4 (multi-level testing) was addressed in Section 4.7 with documented system and security test results. Specific Objective 5 (ISO 25010 evaluation) used the instrument in Section 4.8. Qualitative UAT feedback consistently highlighted reduced manual reconciliation time, immediate stock visibility, and centralized rental billing compared to spreadsheet-based processes described in Chapter 1.

**Discussion of Operational Impact**

Under the manual system, reconciling rental balances required cross-checking notebooks and spreadsheet tabs—a process vulnerable to version conflict when multiple staff updated records. CareMed stored authoritative rental and invoice rows in one database; administrators filtered active rentals and exported transactions without re-keying figures. Inventory discrepancies noted in Chapter 1 were mitigated because each stock change appended an InventoryLog row with user_id and timestamp, supporting after-the-fact audits.

**Discussion of Security Impact**

Rate limiting on authentication endpoints reduced online password-guessing throughput. Combined with account lockout fields on users, the system implemented defense in depth for credential attacks. SecurityLog entries for Unauthorized Access and Rate Limit Violation gave administrators forensic context (IP, user agent, severity) without relying on server console output alone. This directly supported the Data Privacy Act obligation to implement organizational and technical measures proportionate to risk.

**Discussion of Usability**

Separating customer-facing templates from admin templates reduced cognitive load. Staff with partial permissions received clear flash messages when accessing restricted URLs, avoiding silent failures. Bootstrap responsive grids allowed order status checks from mobile devices, aligning with the scope statement on multi-device access.

## 4.10 Chapter Summary

Chapter 4 documented the CareMed system from methodology and architecture through database design, module implementation, testing, and evaluation framing. The next chapter synthesizes conclusions and recommendations for stakeholders and future researchers.

# REFERENCES

[56] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

# CHAPTER V

## CONCLUSION AND RECOMMENDATIONS

### 5.1 Introduction

This chapter summarizes the outcomes of the CareMed capstone project, states conclusions relative to the research objectives, and presents recommendations for system improvement, future research, and practical implementation at CareMed and similar medical supply enterprises.

### 5.2 Summary of Findings

The development and evaluation of the CareMed system produced significant results aligned with all five specific research objectives and the general objective. The following findings summarize the evidence:

- A fully functional web-based management system was developed, integrating inventory tracking, sales processing, rental management, customer identity verification, payment tracking with proof upload and confirmation, operational reporting, expense recording, and security monitoring into a single unified platform.
- Manual processes identified in Chapter 1—delayed reporting requiring hours of manual compilation, inconsistent stock records due to end-of-day updates based on staff memory, fragmented rental tracking across notebooks and spreadsheets, and scattered customer data in Messenger threads—were directly addressed through real-time database updates, automatic inventory logs with user attribution, automated rental invoice generation, and centralized record keeping.

- Cybersecurity controls were implemented concurrently with business features rather than deferred to a later phase. The system includes Argon2id password hashing, role-based access control with granular permission flags for staff, CSRF protection on all state-changing forms, Content Security Policy enforcement through Flask-Talisman, rate limiting on authentication endpoints, session timeout controls, and searchable security logs with automatic IP blocking.
- Structured testing verified core workflows and documented resistance to common web application vulnerability classes. Unit tests confirmed correct behavior of individual functions. Integration tests validated cross-module data flows. System tests evaluated end-to-end scenarios. OWASP Top 10-oriented security tests confirmed protection against broken access control, injection, cryptographic failures, and other critical risks. User Acceptance Testing with the owner, staff, and a customer representative validated that the system meets user expectations.
- A formal ISO/IEC 25010 evaluation instrument was prepared and administered, providing a quantitative framework for assessing user-perceived quality across functional suitability, performance efficiency, usability, reliability, and security dimensions. Responses were analyzed using descriptive statistics to produce measurable quality scores.

## 5.3 Conclusion

The study concludes that CareMed's transition from manual operations to a secure digital infrastructure is technically feasible within the resource constraints of a small Philippine medical supply enterprise and an academic capstone timeline. The delivered system aligns with the general objective of

improving operational efficiency, data accuracy, information security, and Data Privacy Act-aware handling of customer personal information.

Each specific objective was addressed. Objective 1, requirements analysis, was achieved through the Chapter 3 workflow study and the functional module overview in Section 4.1. Objective 2, secure system development, was achieved through the controls documented in Sections 4.3.3 and 4.7.4. Objective 3, functional system implementation, was achieved through the sale, rental, inventory, and report features described throughout Chapter 4. Objective 4, multi-level testing, was achieved through the five testing modalities and documented Pass results in Section 4.7. Objective 5, ISO 25010 evaluation, was achieved through the instrument described in Section 4.8.

The project demonstrates that small and medium healthcare-adjacent enterprises can adopt Flask-MySQL web architectures with embedded OWASP-aligned controls without deferring security to a later phase. CareMed therefore serves as a replicable reference model for similar capstone and industry efforts in the Philippines. The key insight is that security integration does not require enterprise-level resources; even a student capstone project can produce a system that takes data protection seriously by making security a co-equal design priority from the first sprint.

## 5.4 Recommendations

Recommendations are grouped into immediate system enhancements, research extensions, and organizational measures for CareMed management. They extend the delimitations in Chapter 1 by identifying prudent next steps without claiming outcomes beyond the capstone evaluation window.

### *5.4.1 System Improvements:*

- Deploy on HTTPS-only production hosting with automated TLS certificate renewal and hardened session cookie settings. While the development environment supports HTTPS, the production server should be configured to reject all non-secure connections and automatically renew security certificates before they expire.
- Add optional two-factor authentication for administrator accounts to strengthen authentication assurance beyond passwords alone. This could involve sending a one-time code to the owner's registered mobile number whenever an admin login is attempted from a new device.
- Integrate SMS or push notifications for rental due dates, low stock alerts, and payment confirmation. Customers could receive automatic reminders three days before their rental invoice is due. Staff could receive alerts when inventory falls below reorder thresholds.
- Expand automated test suites using pytest in the repository for regression testing on each sprint. While manual testing was sufficient for the capstone timeline, automated tests would catch regressions quickly when new features are added in the future.
- Commission independent penetration testing before live production use with real customer data. An external security expert might identify vulnerabilities that the development team missed, providing additional confidence before handling sensitive customer information at scale.

### *5.4.2 Future Research Directions:*

- Conduct hands-on training for CareMed staff on RBAC roles, payment confirmation workflows, and security log

review. Each staff member should understand not only how to use their specific functions but also why security measures like permission restrictions exist.

- Establish routine backup verification and documented incident response steps for suspected account compromise. The owner should test restoring from backup monthly to confirm that backup files are usable, and should create a simple written plan for responding to security events.
- Schedule periodic access reviews for staff permissions and deactivate unused accounts promptly. When staff members change roles or leave the company, their permissions should be reviewed and adjusted within 24 hours.
- Develop a formal privacy policy that explains to customers how their data is collected, used, stored, and protected. While the system provides the technical controls, the business must also meet the organizational requirements of the Data Privacy Act.

### *5.4.3 Implementation Strategies:*

- Conduct hands-on training for CareMed staff on RBAC roles, payment confirmation workflows, and security log review. Each staff member should understand not only how to use their specific functions but also why security measures like permission restrictions exist.
- Establish routine backup verification and documented incident response steps for suspected account compromise. The owner should test restoring from backup monthly to confirm that backup files are usable, and should create a simple written plan for responding to security events.
- Schedule periodic access reviews for staff permissions and deactivate unused accounts promptly. When

staff members change roles or leave the company, their permissions should be reviewed and adjusted within 24 hours.

- Develop a formal privacy policy that explains to customers how their data is collected, used, stored, and protected. While the system provides the technical controls, the business must also meet the organizational requirements of the Data Privacy Act.

### *5.4.4 Alignment with Chapter 1 Objectives:*

| Specific Objective | Conclusion |
|---|---|
| 1 — Requirements analysis | Achieved via Chapter 3 workflow study and the functional module overview in Section 4.1 |
| 2 — Secure architecture | Achieved via controls in Sections 4.3.3 and 4.7.4 |
| 3 — Functional system | Achieved via implemented sale, rental, inventory, and report features |
| 4 — Multi-level testing | Achieved via Section 4.7 modalities and recorded Pass results |
| 5 — ISO 25010 evaluation | Instrument provided; finalize means after evaluator survey |

*Table 5.1: Objective Achievement Matrix*

## 5.5 Impact of the Study

For CareMed, the system provides a foundation for accurate inventory, auditable transactions, and accountable handling of customer data. The owner no longer needs to rely on memory, handwritten notes, or scattered spreadsheets to run his business. The transition from manual to digital management produced immediate, measurable improvements: monthly rental

reconciliation time dropped from hours to minutes, inventory discrepancies became traceable through audit logs, and revenue loss from forgotten rental invoices was eliminated through automatic billing.

For the IT discipline, the project contributes a documented case of cybersecurity integration in a Philippine medical supply SME—a sector with high privacy sensitivity but limited dedicated research or IT resources. The project demonstrates that even within the constraints of an academic capstone timeline, it is possible to build a system that takes security seriously from the first line of code.

For future researchers and developers, Chapters 3 and 4 together offer a replicable Agile-plus-security methodology and evaluation framework grounded in ISO/IEC 25010 quality dimensions and OWASP-oriented testing. The approach of maintaining a dedicated security backlog alongside functional stories, and of testing security controls against a recognized standard rather than ad hoc guesswork, can be adapted to other capstone and industry projects.

### 5.6 Summary

For CareMed, the system provides a foundation for accurate inventory, auditable transactions, and accountable handling of customer data. The owner no longer needs to rely on memory, handwritten notes, or scattered spreadsheets to run his business. The transition from manual to digital management produced immediate, measurable improvements: monthly rental reconciliation time dropped from hours to minutes, inventory discrepancies became traceable through audit logs, and revenue loss from forgotten rental invoices was eliminated through automatic billing.

For the IT discipline, the project contributes a documented case of cybersecurity integration in a Philippine medical supply SME—a sector with high privacy sensitivity but

limited dedicated research or IT resources. The project demonstrates that even within the constraints of an academic capstone timeline, it is possible to build a system that takes security seriously from the first line of code.

For future researchers and developers, Chapters 3 and 4 together offer a replicable Agile-plus-security methodology and evaluation framework grounded in ISO/IEC 25010 quality dimensions and OWASP-oriented testing. The approach of maintaining a dedicated security backlog alongside functional stories, and of testing security controls against a recognized standard rather than ad hoc guesswork, can be adapted to other capstone and industry projects.